

포스코그룹

WX Agent 입문과정

- # 크롬에 개인 구글 계정으로 로그인
- # 강의장 뒤편에 있는 출석부에 이름 작성



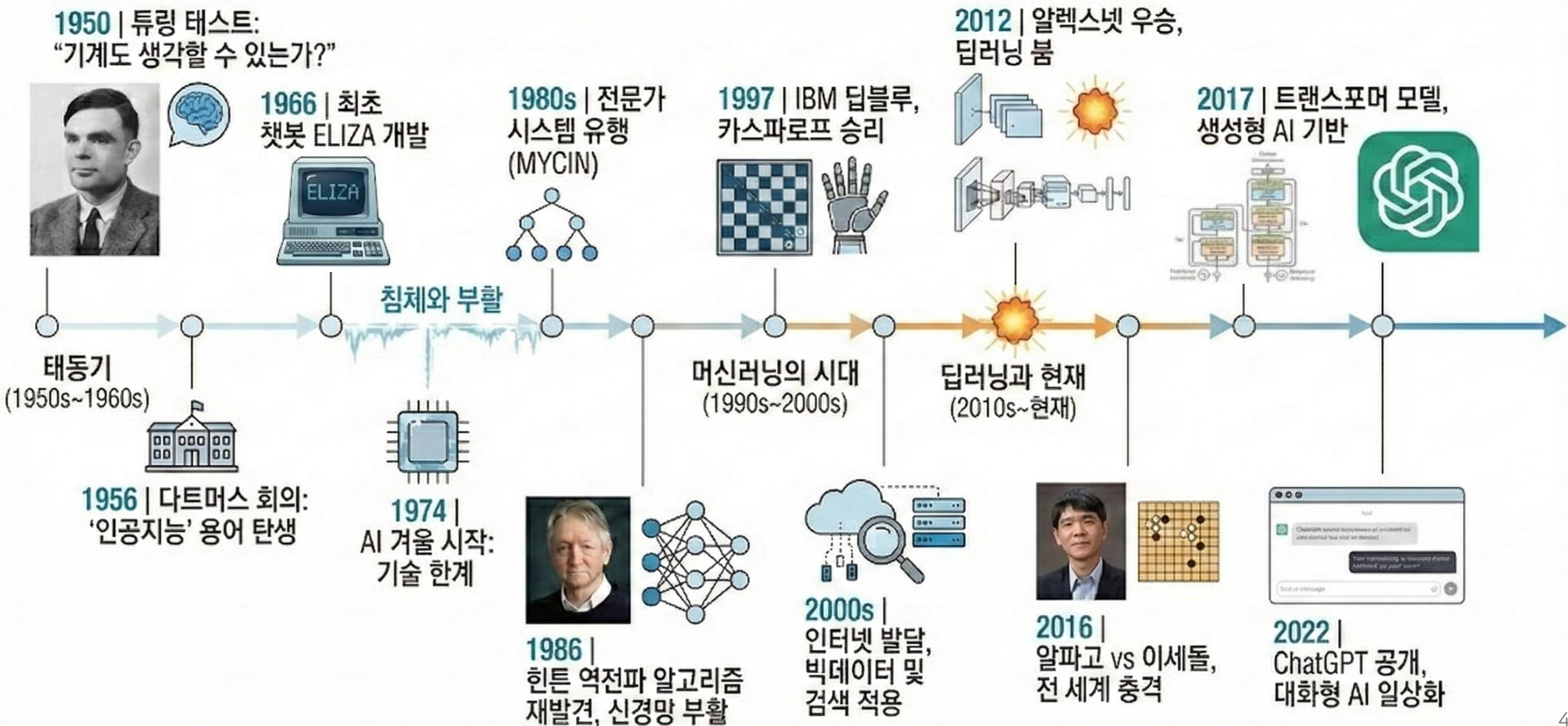
교육커리큘럼

모듈	주제	학습내용	시간
M1. AI Agent 개발 방법론 이해 (2hr)	1. AI 개념 및 기술 이해	- AI 개념, AI Agent 이해 - AI 주요 기술 이해	0.5
	2. 기반 기술 이해	LLM 이해: ChatGPT와 OpenAI의 API, 토큰, 컨텍스트 윈도우	1.5
M2. 바이브코딩 활용 앱/웹 개발 (8hr)	3. 바이브코딩 환경 구축	- Cursor AI 개발 환경 이해 - 프롬프트 작성 방법, Cursor 활용 실습	1
	4. 바이브코딩 기반 앱 개발	- 웹과 앱의 차이 - PDF 분할/병합(PDF 자료 업로드) - 이미지 다운로드, 사이즈 색상 및 크기 조정 - PDF를 PPT로 변환	1.5
	5. 바이브코딩 기반 웹 개발	- 웹 이해 : 웹 개발 절차(기획부터 개발까지) 프론트엔드와 백엔드, 마크다운/API/JSON 이해 - API Key 활용, 웹페이지 개발(PDF번역 웹페이지) * IP주소, 포트이해 - DB활용, 웹페이지 개발(물품공유 웹페이지) - DB 및 파일 활용, 예측 프로그램 개발	3.5
	6. 바이브코딩 스킬 활용	스킬을 활용해서 아이디어 구체화-프롬프트 제작-웹 개발 실습	2
M3. RAG 구현 (3.5hr)	7. LangChain 이해	랭체인 맛보기 : 랭체인 개념과 구성 요소, LLM API 호출하기, PromptTemplate, Output parser	1
	8. RAG 코드리뷰	랭체인 구성 요소 : Prompt Template, Output Parser, Document loader, Text Splitters, Embedding, Vectorstore, Retriever	1
	9. RAG 구현	Cursor AI 활용 Rag 구현 : ① General Rag ② Local LLM, 임베딩 모델 활용 Rag	1.5
M4. 종합실습(2.5hr)	10. 종합실습	예제 3개 중 1개 선택, 아이디어 구체화-프롬프트 작성-시스템 구현까지 완성	2.5

M1. AI Agent 개발 방법론 이해

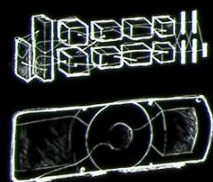
1. AI 개념 및 기술 이해

AI(인공지능)의 과거와 현재



GENERATIVE AI 에서 AGENTIC AI로..

2012 ALEXNET



PERCEPTION AI

SPEECH RECOGNITION
DEEP RECSYS
MEDICAL IMAGING

GENERATIVE AI

DIGITAL MARKETING
CONTENT CREATION

AGENTIC AI

CODING ASSISTANT
CUSTOMER SERVICE
PATIENT CARE

PHYSICAL AI

SELF-DRIVING CARS
GENERAL ROBOTICS

말하는 AI에서, '일하는 AI'로의 거대한 전환

단순한 대화형 인터페이스를 넘어, 실질적인 업무를 수행하는 에이전트(Agent)로의 진화

대화형 AI (Conversational AI)



- 수동적 정보 제공 및 검색
- 단순 질의응답(Q&A) 및 잡담
- '무엇인가요?'에 대한 답변 중심

[인터페이스(Interface) 역할]

진화 (EVOLUTION)

실행형 AI (Action-Oriented AI)



- 능동적 업무 수행 및 워크플로우 자동화
- 복합적 문제 해결 및 도구 활용(Tool Use)
- '해주세요'에 대한 결과 창출 중심

[파트너(Partner) & 에이전트(Agent)]

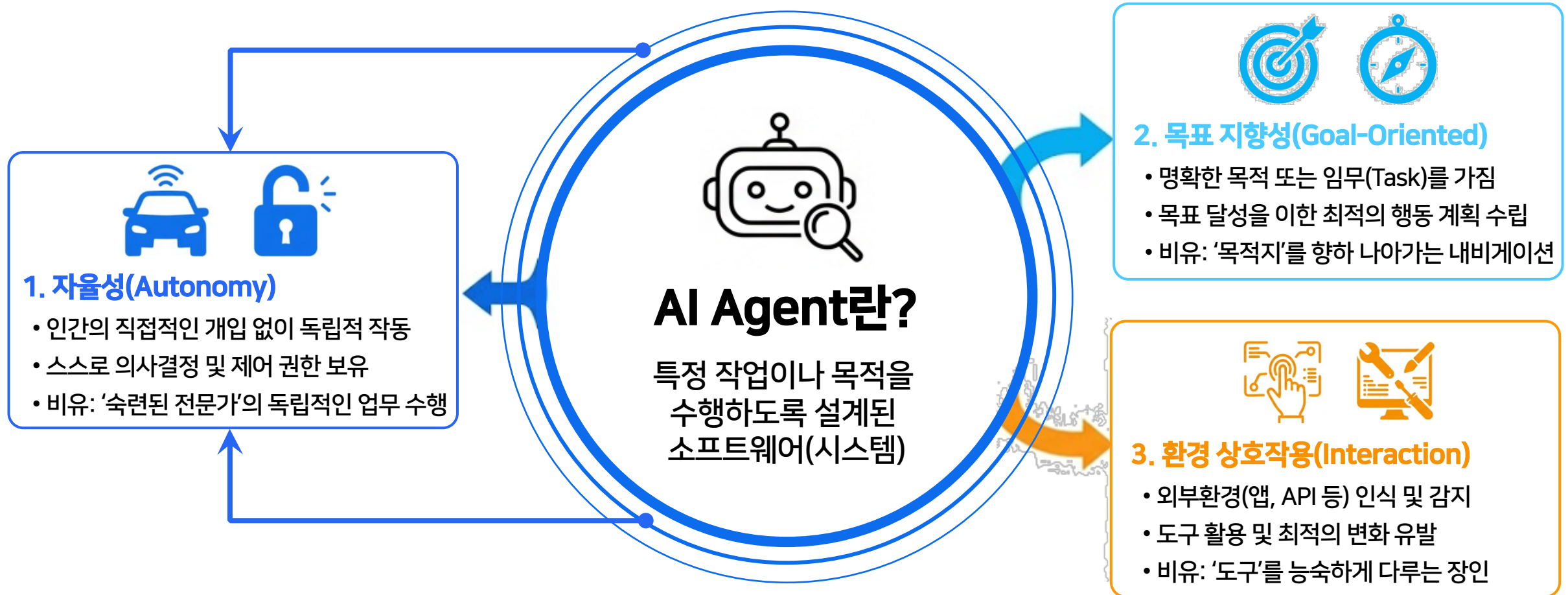
AI는 이제 단순한 '말동무'가 아닌, 비즈니스 성과를 창출하는 가장 강력한 '도구'가 됩니다.

사람·기관·업종별 AI Agent 정의

사람 / 학계 관점	AI 모델·개발 플랫폼	클라우드 / 빅테크	엔터프라이즈 / 산업 활용
Russell & Norvig 환경을 지각하고 목표 달성을 위해 행동하는 시스템	OpenAI LLM이 도구를 사용해 복잡한 작업을 단계적으로 수행하는 시스템	Google Cloud 사용자를 대신해 목표를 추구하고 작업을 완료하는 AI 소프트웨어	IBM 도구를 활용해 워크플로를 만들고 자율적으로 업무를 수행하는 프로그램
Andrew Ng 계획·실행·반성·도구 사용을 반복하며 결과를 개선하는 AI	Anthropic 사용자와 협업하며 계획하고 행동하는 AI 동료	Microsoft 목표를 이해하고 계획하며 사용자를 대신해 행동하는 동적 소프트웨어	Salesforce 고객 맥락을 이해하고 업무를 자동 처리하는 비즈니스 AI 에이전트
	LangChain LLM이 도구를 선택해 반복적으로 문제를 해결하는 구조	AWS 사람이 목표를 주면 달성 방법을 스스로 선택하는 소프트웨어	Amazon Science 채팅을 넘어 실제 세계의 일을 대신 수행하는 AI
		NVIDIA 목표를 받아 추론하고 계획하며 실행하는 디지털 작업자	

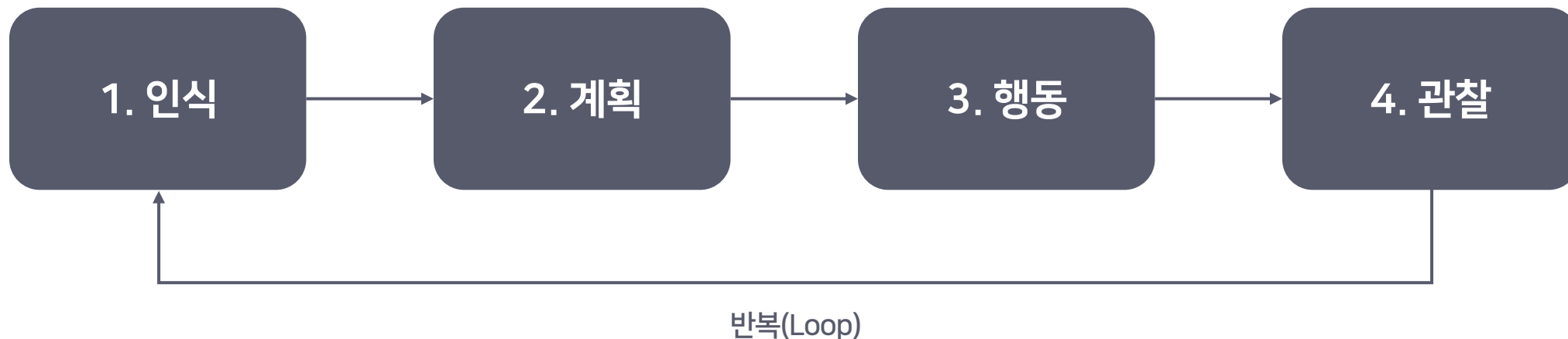
공통 목표를 이해하고, 계획을 세우고, 도구를 사용해 행동하고, 결과를 바탕으로 반복하는 자율 AI 시스템

AI Agent 정의



AI 에이전트란?

- ▶ **정의** 목표를 입력 받아 스스로 계획을 세우고, 도구를 활용해 행동하며, 결과를 평가해 다음 행동을 결정하는 자율형 소프트웨어
- ▶ **챗봇과의 차이** 챗봇은 '대화 응답', 에이전트는 '목표 달성을 위한 다단계 행동'
- ▶ **핵심 동작 루프** 인식 → 계획 → 행동 → 관찰을 반복하며 목표에 도달



- ▶ **'비즈트립'(AI Agent)에** 직장인 A "다음 주 화요일 부산 출장 준비해줘" 입력

동작 흐름: 인식 → 계획

1. 인식 (Perceive)

- 사용자 메시지를 LLM이 해석:
"부산 출장 + 다음 주 화요일 + 준비"라는 의도 파악
- 자연어 속 정보 추출(개체명 인식과 유사):
목적지, 날짜, 작업 범위

2. 계획 (Plan)

- 작업을 세부 단계로 분해
항공권 검색 및 예약
호텔 예약
캘린더에 일정 등록
출장 보고서 초안 작성
- 오케스트레이터가 실행 순서를 관리

동작 흐름: 행동 → 관찰

3. 행동 (Act)

- 항공권 API 호출 → 항공사 예약 시스템에
REST API로 가격·좌석 조회
- 호텔 예약 API 호출 → 가용 객실 검색
- 캘린더 API 호출 → 사용자 일정에 자동 등록
- 백엔드-API 통신(HTTP 요청/응답) 방식으로 동작

4. 관찰 (Observe)

- 항공권 가격이 예산 초과 → 대안 항공편 재검색
- 호텔과 회의 장소 거리가 멀어 → 다른 호텔로 재탐색
- 결과를 평가해 다음 행동을 스스로 수정
(계획 ↔ 행동 반복)

메모리 활용과 전체 흐름

5. 메모리 활용

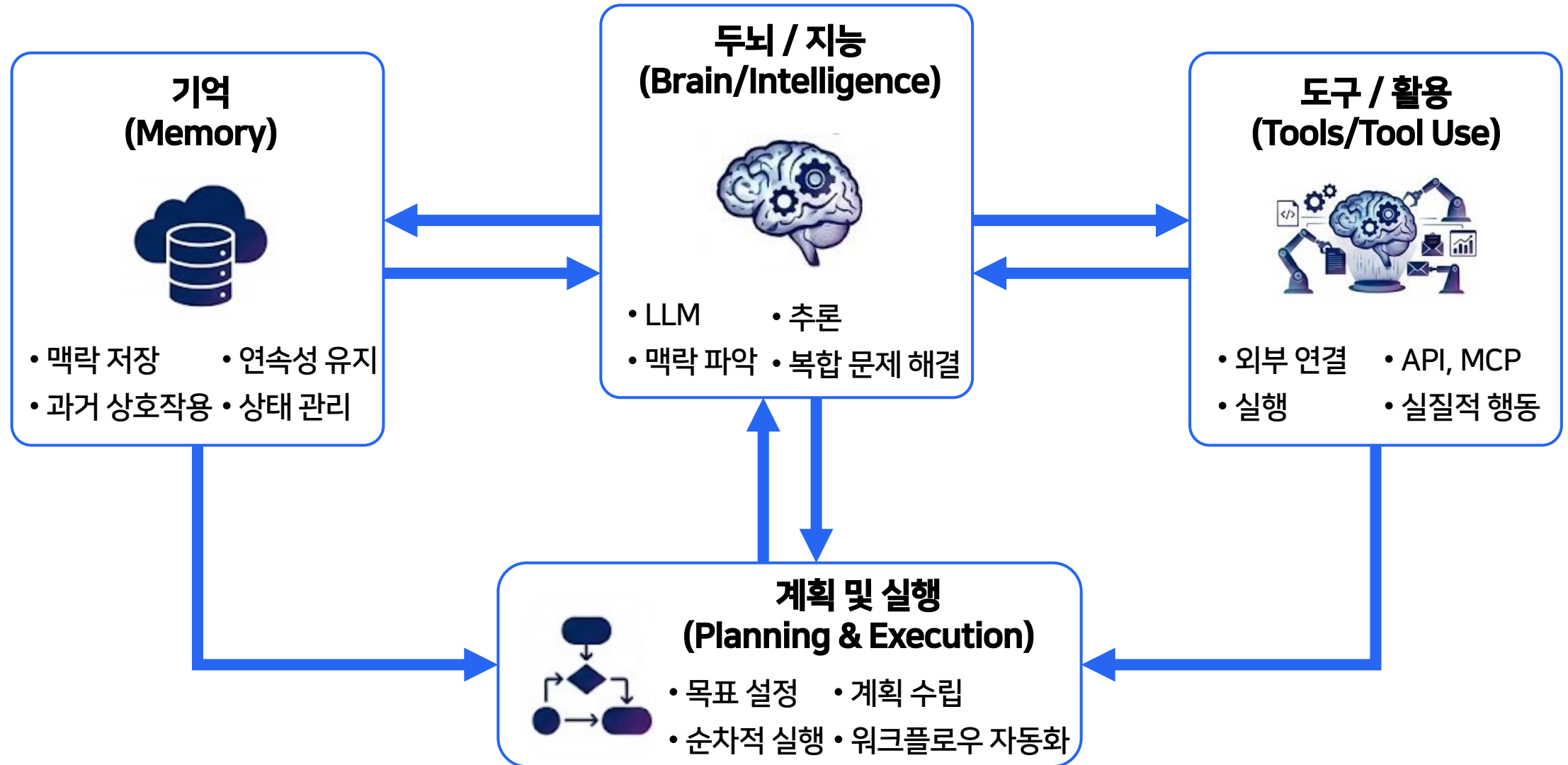
- 직장인 A의 과거 선호(창가석, 4성급 호텔, 저녁 비행기 선호)를 벡터DB/사용자 DB에서 조회
- 이번 예약에도 동일 선호 자동 반영
- 새 선택 결과는 다시 메모리에 저장
→ 다음 출장에 재사용

6. 전체 흐름 요약

- 사용자 요청 (프론트엔드)
- 의도 파악 (LLM)
- 작업 분해 (플래너)
- 외부 API 호출 (백엔드)
- 결과 검증·재시도
- 메모리 저장
- 사용자에게 최종 결과 보고

※ AI 기술(추론·계획)과 전통 웹 기술(API·DB·서버)이 함께 맞물려 동작하는 구조

AI Agent 구성요소



AI 주요기술 _ 컴퓨터 비전

컴퓨터 비전(Computer Vision)은 컴퓨터가 디지털 이미지, 비디오 및 기타 시각적 입력으로부터 정보를 추출하고 이해하도록 하는 기술

【Classification】



【Object Detection】



- 이미지처리 : 디지털 이미지를 변환, 개선 분석
 - 객체탐지 : 이미지나 비디오에서 특정 객체를 탐지, 위치 표시
 - 객체분할 : 이미지에서 객체를 픽셀 단위로 분리
 - 이미지분류 : 이미지를 분석해 특정 클래스나 라벨을 예측
- ※ 파이썬 패키지 : opencv, tensorflow, pytorch, torchvision

컴퓨터 비전 절차

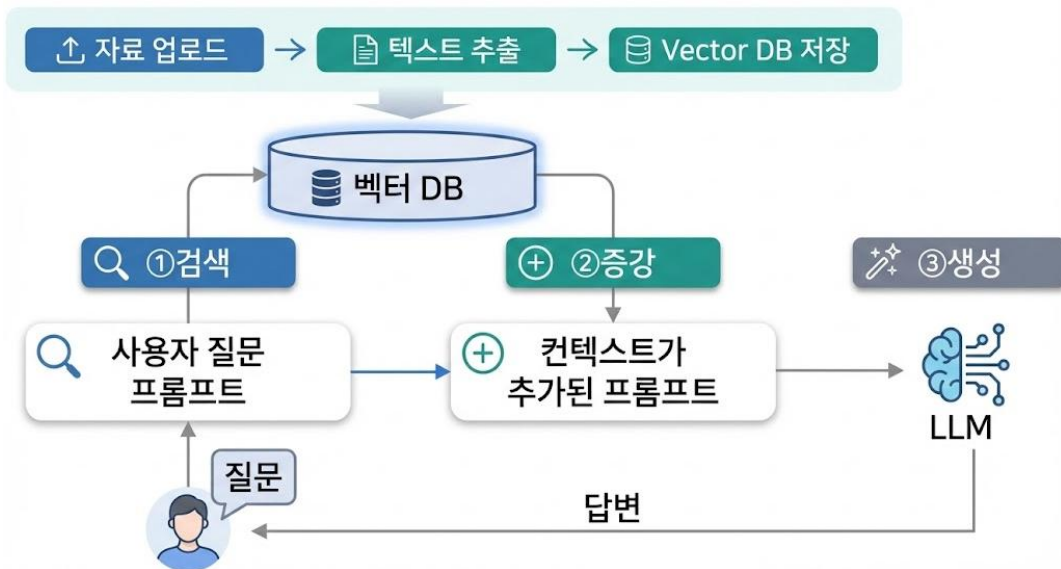


☞ 활용 : 제품 표면 결함 판별, 작업자 안전상태 확인, 자율 주행, 침입자 감지, 의료 영상 판별

AI 주요기술 _ Rag

RAG(Retrieval-Augmented Generation)는 대형 언어 모델(LLM)과 외부 데이터 소스를 결합하여 보다 정확하고 맥락적으로 풍부한 답변을 생성하는 기술

【RAG 시스템】



☞ 활용 : 지식기반 고객상담 챗봇, 맞춤형 튜터링, 특정 법률, 규정, 판례 검색, 맞춤형 보고서 생성

- 검색단계 : 사용자의 질문에 대한 정보를 벡터DB에 저장한 외부 데이터 소스에서 검색
- 증강단계 : 검색된 데이터를 사용자가 입력한 질문과 결합하여 언어모델이 더 풍부하게 답변할 수 있도록 보조
- 생성단계 : 증강된 프롬프트를 바탕으로 사용자의 질문에 대한 최종 답변을 생성

※ 파이썬 패키지 : [Langchain](#), [Langgraph](#), [Openai](#)

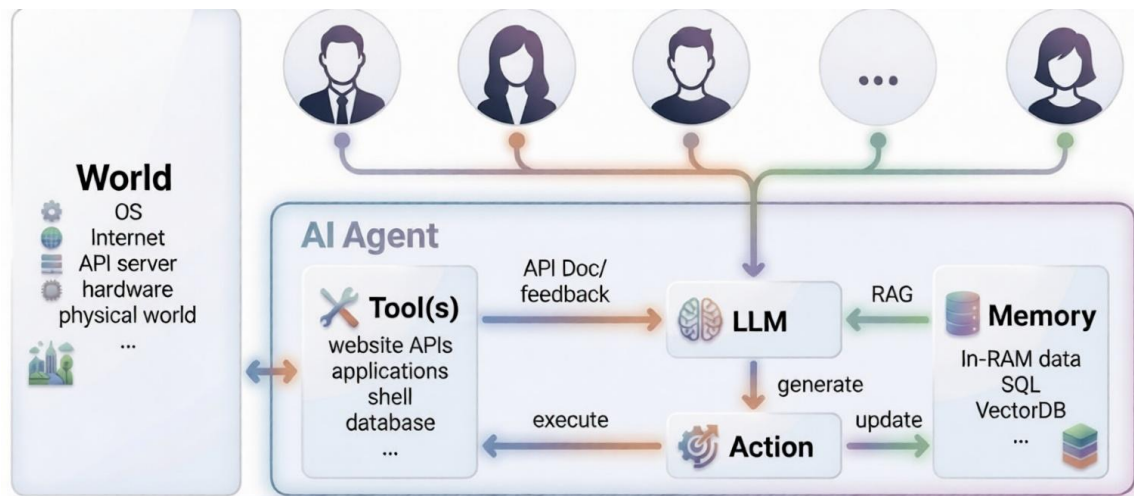
RAG 장점

- 외부 데이터 소스에서 필요한 정보를 실시간으로 가져올 수 있음
- 일반적인 언어 모델보다 더 정확하고 신뢰할 수 있는 답변을 제공

AI 주요기술 _ AI Agent

AI 에이전트(AI Agent)는 인공지능 기술을 기반으로 **특정 작업이나 목적을 수행하도록 설계된 소프트웨어** 또는 시스템을 의미

【AI Agent】



☞ 활용 : 공장 자동화, 로봇 제어, 시장 분석, 의료 상담, 맞춤형 학습경로 제안, 개인화된 제품 추천

- Tool : AI 에이전트가 특정 작업을 수행하거나 외부 시스템과 상호작용하기 위해 사용하는 기능 또는 인터페이스
- Action : AI 에이전트가 도구를 활용하여 실제 작업을 수행하거나 환경 상태를 변경하는 행위
- Memory : AI 에이전트가 과거 경험, 데이터를 저장하고 이를 학습하거나 활용하여 더 나은 의사결정을 할 수 있도록 하는 기능

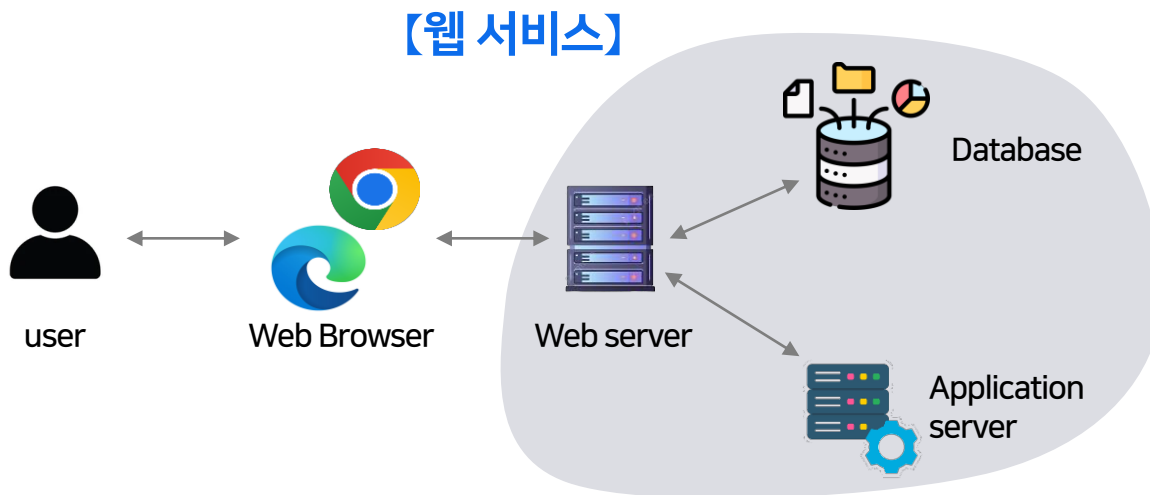
※ 파이썬 패키지 : Langchain, LangGraph, Openai, CrewAI

AI Agent 특징

- 자율적 의사결정과 행동을 통해 복잡한 문제를 해결하거나 인간의 작업을 보조하는 데 있어 뛰어난 성능을 발휘

AI 주요기술 _ 웹 프레임워크

웹 프레임워크는 웹사이트나 서비스를 개발할 때 필요한 기본 구조와 기능 (라우팅, 데이터베이스, 보안 등)을 미리 제공해주는 도구



- 풀 스택 프레임워크
 - 웹 서비스 개발에 필요한 모든 도구와 기능 제공
 - Django(python), Spring(java), Ruby on Rails(ruby)
 - 마이크로 프레임워크
 - 웹 서비스의 필수적인 기능만 제공, 간결하고 유연한 구조
 - Flask(python), FastAPI(python), Node.js(javascript)
- ※ 파이썬 패키지 : Django, FastAPI, flask, streamlit

※ 파이썬으로 간단하게 데이터 시각화 및 대시보드 웹 애플리케이션을 구현하는 “streamlit”

- 추가적인 서버/프론트엔드 개발 없이 Python 코드로 즉시 실행 가능한 애플리케이션 생성

AI 주요기술 _ 데이터베이스

데이터베이스(Database)는 데이터를 체계적으로 저장하고 관리하는 시스템으로 여러 사용자가 효율적이고 안전하게 데이터를 접근하고 활용할 수 있도록 설계

관계형 데이터베이스

- 데이터를 표(Table) 형태로 저장하며 테이블 간의 관계를 키를 통해 표현, 구조적 데이터 처리
- MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server

비관계형 데이터베이스

- 정형화 되지 않은 데이터를 저장하고 처리, 고정된 스키마가 없으며 다양한 데이터 형식을 지원
- MongoDB, Redis, Apache Cassandra, Neo4j *벡터DB : Chroma, Faiss, Weaviate

하이브리드 데이터베이스

- 관계형 데이터와 비관계형 데이터를 혼합하여 저장 및 관리, 유연한 데이터 처리가 필요할 때 적합
- Amazon Aurora, Azure Cosmos DB

※ 파이썬 패키지 : sqlite3(파이썬 표준 라이브러리), mysql-connector-python, psycopg2, cx_Oracle, pymongo, elasticsearch

M1. AI Agent 개발 방법론 이해

2. 기반 기술 이해

LLM(Large Language Model, 대형언어모델)

- 정의 : 수조 개의 문장 데이터를 학습하여 인간처럼 언어를 이해하고 생성하는 AI 모델
- 핵심 원리: '다음에 올 가장 확률 높은 단어'를 예측하는 기계
 - 예 : "대한민국의 수도는 []" -> "[서울]일 확률 99%"

▪ 대표 모델



▪ 한계점과 해결책

한계점 (Problem)	설명	해결책 (Solution)
할루시네이션 (Hallucination)	사실이 아닌 것을 자신 있게 말함	RAG (외부 데이터 참조)
최신성 부족	학습 데이터 시점 이후 정보 모름	웹 검색 및 API 연동
복잡한 업무 수행	한 번에 긴 작업 수행 시 실수함	Workflow (단계별 설계)
보안 문제	기업 내부 데이터 유출 우려	Private LLM / 보안 설정

LLM의 주요 기능



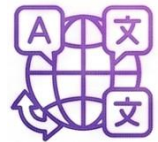
콘텐츠 생성

- 자연스러운 문장, 이메일, 보고서 초안, 시, 소설 등 새로운 콘텐츠 제작
- 예시: "신제품 런칭을 위한 인스타그램 홍보 문구 5개 작성해 줘"



요약 및 추출

- 방대한 양의 텍스트에서 핵심 내용을 추리거나 필요한 정보(날짜, 금액, 이름 등)만 뽑아 냄
- 예시: "1시간 분량의 회의록을 3줄로 요약하고 결정된 실행 과제만 리스트업해 줘"



변환 및 번역

- 언어 간 번역은 물론, 말투 변경(비즈니스 톤 ↔ 반말), 데이터 형식 변경(텍스트 ↔ JSON/CSV)
- 예시: "이 고객 불만 글을 정중한 답변 메일 형식으로 바꿔줘" 또는 "한글 문서를 영어로 번역해 줘"



추론 및 논리 분석

- 복잡한 문제를 단계별로 풀거나, 논리적인 오류를 찾고, 프로그래밍 코드를 작성/디버깅함
- 예시: "이 파이썬 코드에서 에러가 나는 이유를 찾고 수정해 줘" 또는 "수학 서술형 문제 풀이 과정"



분류 및 감성 분석

- 텍스트의 카테고리를 나누거나 작성자의 감정 상태(긍정/부정)를 파악
- 예시: "수천 개의 고객 리뷰를 분석해서 칭찬인지 불만인지 분류해 줘"

ChatGPT 서비스 vs OpenAI API

ChatGPT (웹 서비스, <http://chatgpt.com>)

- 사용자의 과거 대화 기록을 서버가 알아서 기억함
- "아까 말한 코드 수정해줘"가 자연스럽게 통함.
- 미세 제어(온도, 규칙 등)가 어려움.

OpenAI API (개발자용, GPT 5.4)

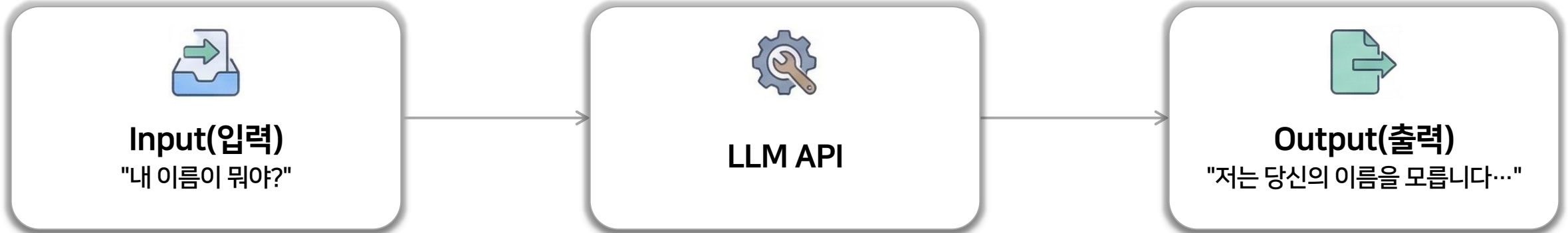
- AI 에이전트 개발의 뼈대이지만, 기본적으로 이전 대화를 전혀 기억하지 못함(무상태성)
- 매번 질문할 때마다 처음부터 끝까지의 모든 대화 기록을 개발자(에이전트)가 직접 묶어서 전송해야 함
- 에이전트 개발 시 '메모리(장부) 관리 프로그램'을 짜는 것이 필수인 이유임

OpenAI API

- OpenAI API를 이용해서 질문하기: 첫번째 질문



- OpenAI API를 이용해서 질문하기: 두번째 질문



- 해결방법: 매번 대화 기록을 messages 배열에 직접 담아 보내야 합니다

컨텍스트 윈도우 (Context Window)

■ 정의

- 모델이 한 번의 API 호출에서 읽고 처리할 수 있는 최대 토큰 수의 한계
- 대화 기록 + 시스템 프롬프트 + 사용자 입력 + 응답 전부 합산

■ 최대 토큰 수를 넘어서면?

- 오래된 대화 내용이 잘리거나 오류 발생
- API가 에러를 반환하거나 앞부분을 무시

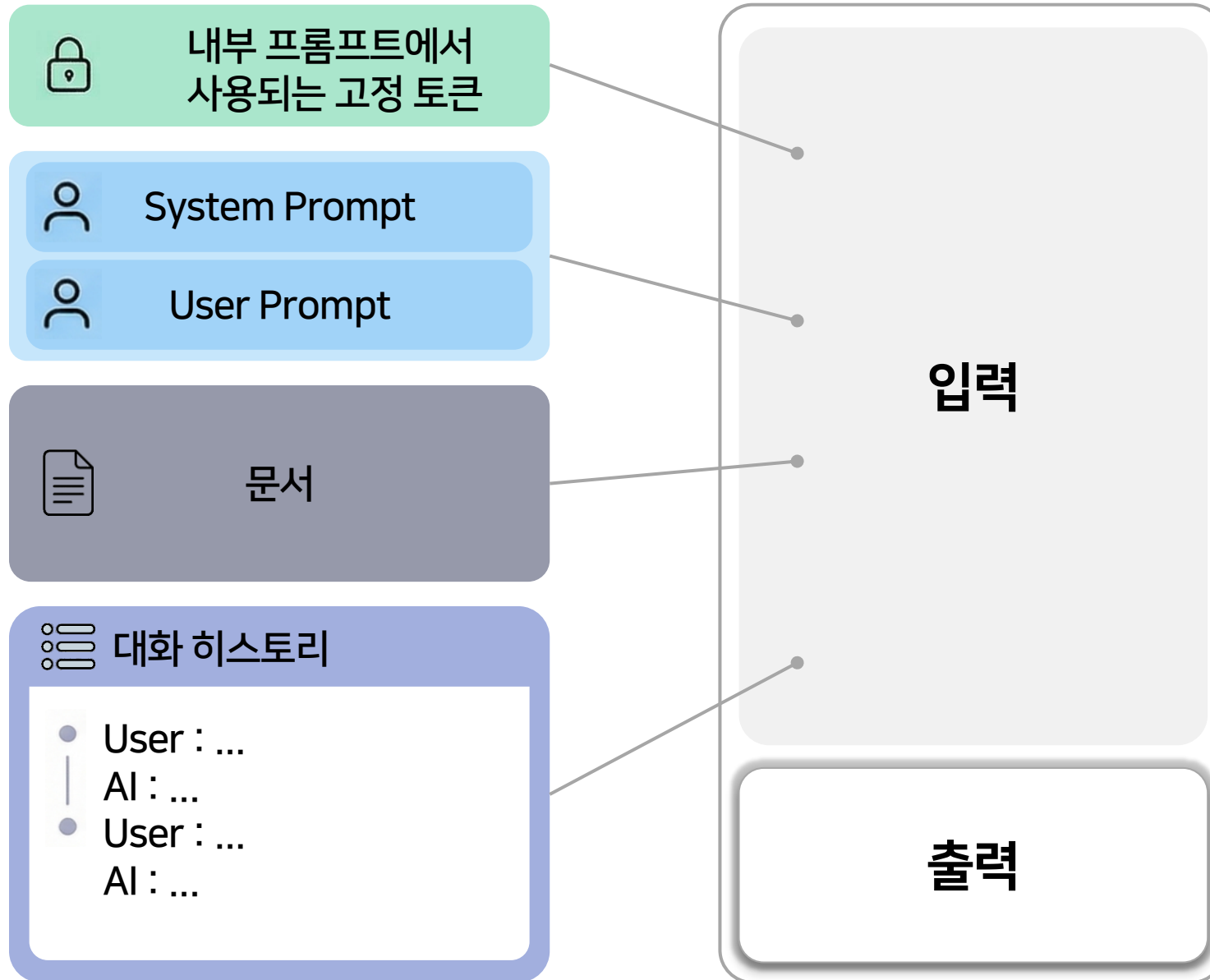
■ 바이브코딩 시 발생하는 문제 (금붕어 현상)

- 전체 소스코드를 계속 처리하다 보면 용량 제한을 초과함
- 초과 시 앞부분의 개발 규칙이나 가이드라인을 까먹음

■ 토큰 수를 최소화 하려면?

- 대화 기록 슬라이딩 윈도우 적용
- 핵심만 요약해서 보내거나, 청크 분할 전략 사용

컨텍스트 윈도우 (Context Window)



주요 모델별 컨텍스트 크기

모델	Context Window	최대 출력
GPT-5.2	128K~400K	128K
Claude Opus 4.6	200K	128K
Gemini 2.5 Pro	1M	65K
Llama 3	128K	?

128K 토큰 환산 분량

- 해리포터 1권
- A4지 100장
- 대학 석사 논문
- 채팅 5000건
- 소스코드 1만줄

토큰(Token)

토큰이란?



AI 모델이 텍스트 데이터를 처리하고 이해하기 위해 나누는 가장 작은 단위

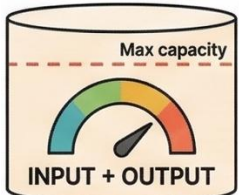


분할 기준은 모델마다 다르며, 일반적으로 단어·문자·형태소 단위로 쪼개짐

【확인방법】 <https://platform.openai.com/tokenizer>



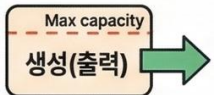
토큰 길이 한계



Context Window (컨텍스트 윈도우)

모델이 한 번에 처리할 수 있는 입력(Input) + 출력(Output)의 최대 합계

⚠ 범위 초과 시 이전 대화를 기억하지 못하거나 오류 발생



Max Tokens (최대 출력 토큰)

모델이 한 번의 요청에 대해 생성(출력)할 수 있는 최대 길이 제한

토큰화 예시

예시문장: I love programming!

"I"

"love"

"programming"

!"

계산: 총 4개의 토큰으로 분할되어 처리

토큰화 예시(질문)



시스템 프롬프트: 모델의 역할 및 지침 설정값



외부 데이터: 업로드한 이미지, 문서(PDF/Excel 등) 파일 내용



대화 이력(History): 이전 단계에서 주고받은 대화내역 전체

Agent에서의 토큰 관리

Multi-turn 대화에서 토큰이 쌓이는 구조

턴 1:
질문(50) + 답변(200) 사용 토큰: 250

턴 2: 이전(250) + 질문(50) + 답변(200) 사용 토큰: 500

턴 3: 이전(500) + 질문(50) + 답변(200) 사용 토큰: 750

...

턴 N: 컨텍스트 한계 도달 **잘림**

Sliding Window

오래된 메시지를 순서대로 제거하여
윈도우 크기 유지

Summarization

이전 대화를 요약 텍스트로 압축하여 삽입

RAG(검색 증강 생성)

필요한 정보만 검색하여 컨텍스트에 삽입

M2. 바이트코딩 활용 앱/웹 개발

3. 바이트코딩 환경 구축

바이브코딩

복잡한 프로그래밍 문법 대신, 자연어로 의도(Vibe)를 설명하면 AI가 코드 작성



Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · 4M Views

1.2K

3.9K

24K

12K



"바이브 코딩(vibe coding)"이라고 부르는 새로운 종류의 코딩이 있습니다. 여기서는 '바이브(vibe)'에 완전히 몸을 맡기고, 기하급수적인(AI의) 발전 속도를 받아들이며, 코드가 존재한다는 사실조차 잊어버립니다.

(중략)

저는 항상 '모두 수락(Accept All)'을 누르고, 이제 더 이상 코드 변경 사항을 읽지 않습니다. 에러 메시지가 뜨면 아무 설명 없이 그냥 복사해 붙여 넣는데, 보통 그러면 해결됩니다.

(중략)

저는 프로젝트나 웹 앱을 만들고 있지만, 이건 사실 코딩이 아닙니다. 그저 보고(see), 말하고(say), 실행하고(run), 복사해서 붙여 넣을(copy paste) 뿐인데, 대부분 잘 작동합니다."

- 2025년 2월, 안드레이 카파시 (Andrej Karpathy)의 언급

바이브코딩 Tools

도구 명칭		주요 특징 및 용도
커서(Cursor)		AI 기반 코드 편집기로, 기존 코드를 이해하고 자연어로 전체 파일 수정 및 생성이 가능
Google AI Studio		구글의 최신 모델(Gemini)을 테스트하고 프롬프트를 설계하여 빠르게 API 프로토타입 생성
Claude Code		엔트로픽(Anthropic)에서 제공하는 터미널 기반 도구로, 명령줄에서 직접 복잡한 코딩 작업을 수행
코덱스(Codex)		OpenAI의 모델로, 자연어를 코드로 변환하는 엔진 역할을 하며 다양한 서비스의 기반 마련
리플릿(Replit)		브라우저 기반 개발 환경으로, 'Replit Agent'를 통해 대화만으로 웹 서비스 전체를 구축
안티그래비티(Antigravity)		AI 에이전트 중심의 통합 개발 환경(IDE). '에이전트 우선(Agent-first)'방식의 플랫폼
제미나이(Gemini)		구글의 AI로, 코드 생성뿐만 아니라 논리적인 구조 설계 및 데이터 시각화 Prototyping에 강점
챗지피티(ChatGPT)		'Canvas'나 'GPTs' 기능을 통해 인터랙티브 한 앱 프로토타입과 문서, 코드를 즉각 생성
오파(Opal AI)		시각적인 노드 기반이나 자연어 흐름을 통해 AI 앱의 로직을 설계하고 결과물을 확인

실습준비



시스템 및 프로그램 설정



① 파이썬 설치 : 3.11.9



② 패키지 설치 : requirements.txt 파일 작성 및 설치
`pip install -r requirements.txt`



③ Cursor AI 설치



④ Git 설치 * 2일차 실습, Skill 설치시 필요



필수 패키지 목록(requirements.txt)

```
easyocr>=1.7.0
email-validator==2.1.0
fpdf2==2.7.9
Flask==3.0.2
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Flask-WTF==1.2.1
matplotlib==3.8.2
numpy==1.26.2
openai
opencv-python>=4.8.0
openpyxl==3.1.2
pandas==2.1.4
pdf2image==1.17.0
pdfplumber==0.11.0
pytesseract==0.3.10
Pillow==10.2.0
PyMuPDF>=1.23.0
pypdf==4.0.1
PyQt5
python-pptx>=0.6.21
scikit-learn==1.3.2
scipy==1.11.4
seaborn==0.13.0
statsmodels==0.14.1
torch>=2.0.0
torchvision>=0.15.0
WTFForms==3.1.1
Werkzeug==3.0.1
xlrd==2.0.1
yt-dlp==2024.12.23
.
```

실습준비 _ ① 파이썬 설치

①크롬에서 파이썬 다운로드 검색 → ②Download Python 클릭 → ③3.11.9 검색 → ④클릭

The image consists of two side-by-side browser screenshots illustrating the steps to download Python.

Left Screenshot (Google Search):

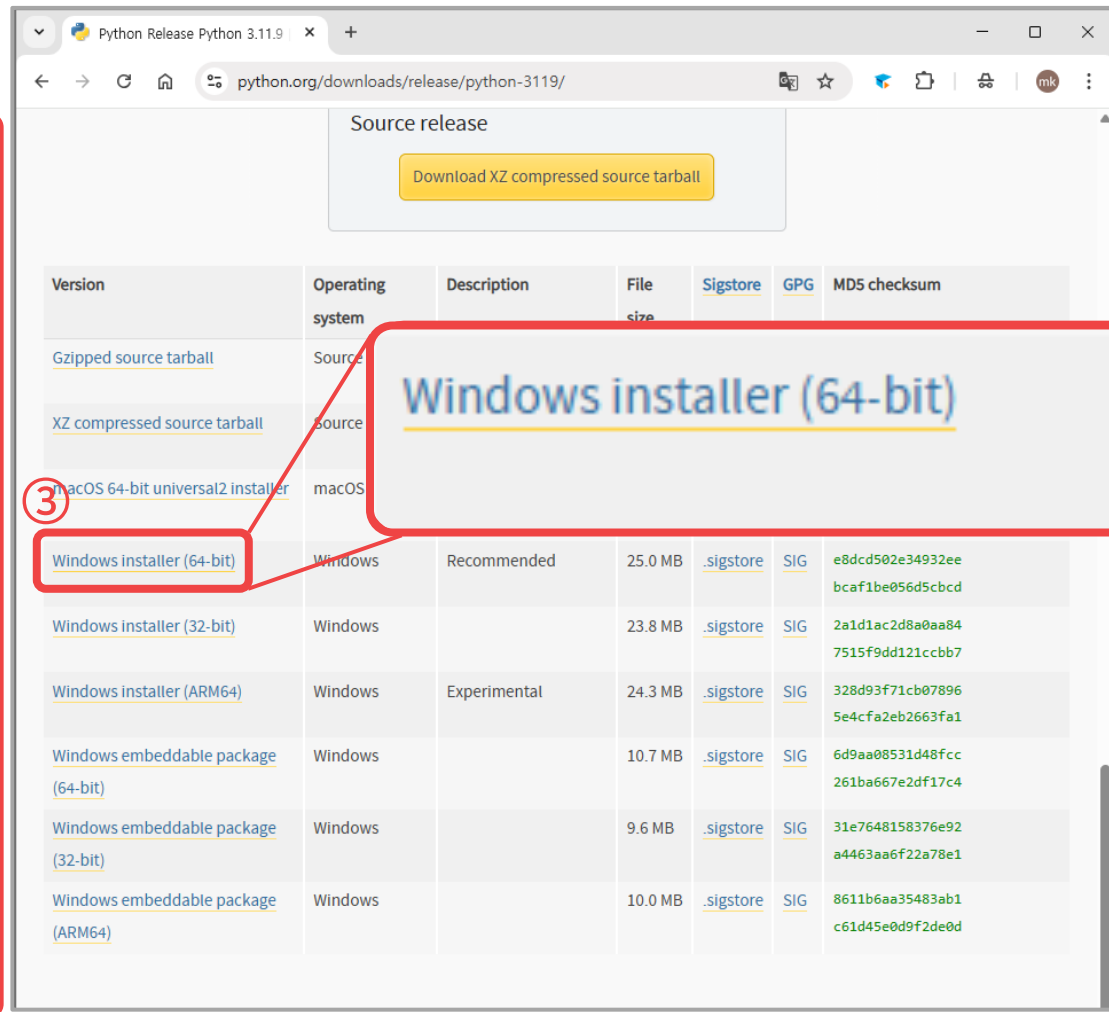
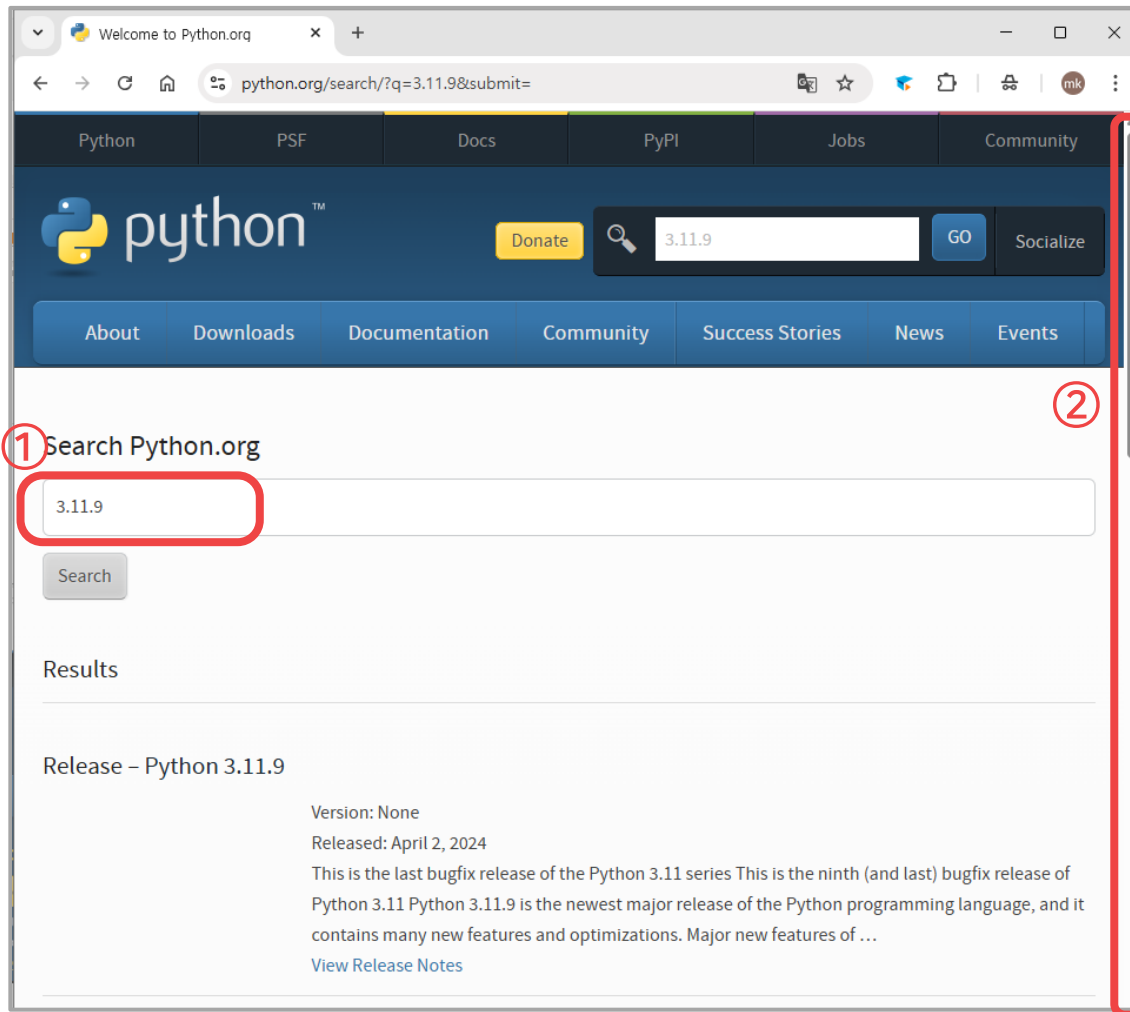
- The browser address bar shows a Google search for "파이썬 다운로드" (Python download).
- The search term "파이썬 다운로드" is circled in red and labeled with a red circle containing the number 1.
- The search results show a link to "Python.org" with the text "Download Python" circled in red and labeled with a red circle containing the number 2.

Right Screenshot (Python.org Website):

- The browser address bar shows the URL "python.org/downloads/".
- The website header includes the Python logo and navigation links: Python, PSF, Docs, PyPI, Jobs, and Community.
- The search bar on the website shows "3.11.9" entered, and the "GO" button is circled in red and labeled with a red circle containing the number 4.
- The main content area features a large banner with the text "Download the latest version for Windows" and a yellow button labeled "Download Python install manager".

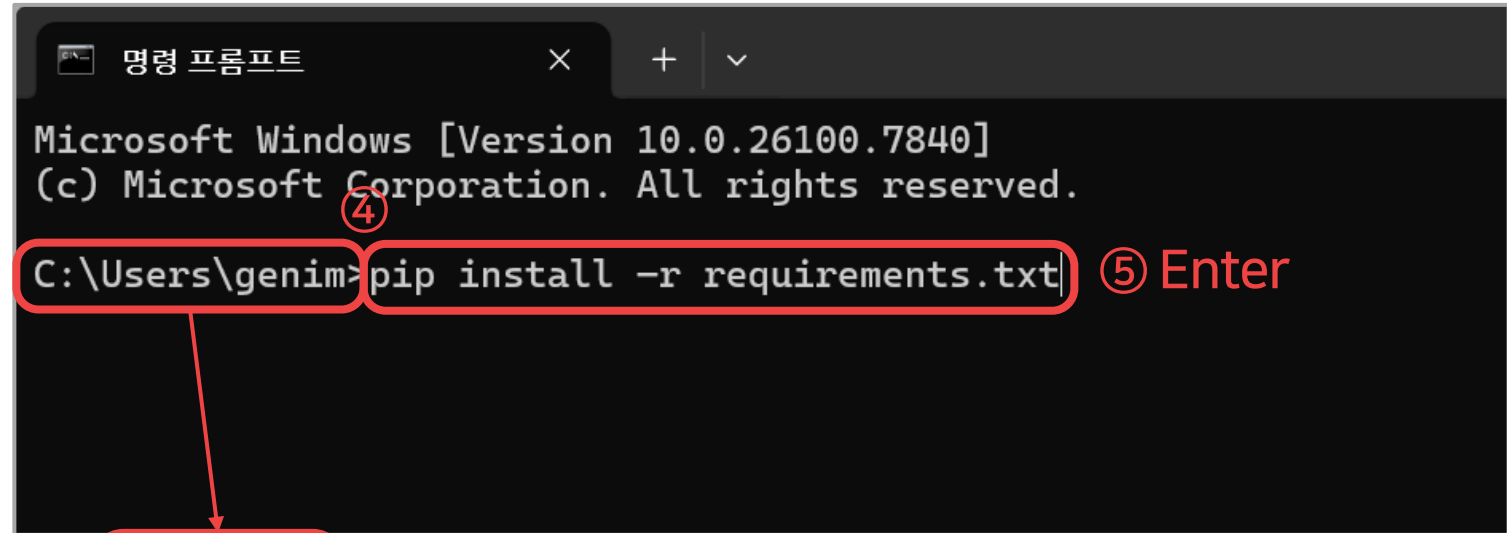
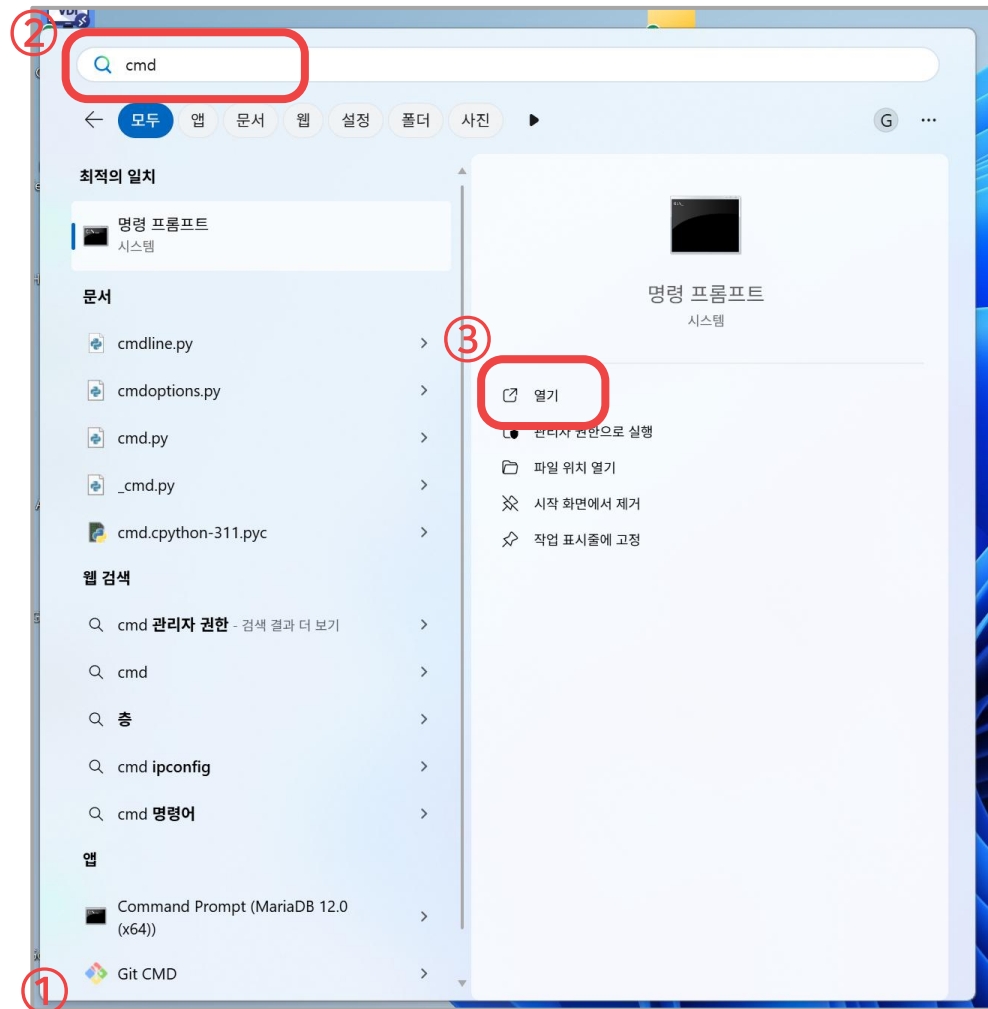
실습준비 _ ① 파이썬 설치

① Release - Python 3.11.9 클릭 → ② 스크롤 아래로 내리기 → ③ Windows Installer (64-bit) 클릭



실습준비 _ ② 패키지 설치

① 윈도우 클릭 → ② cmd 입력 → ③ 명령 프롬프트 선택 → ④ pip install -r requirements.txt 입력 → ⑤ Enter

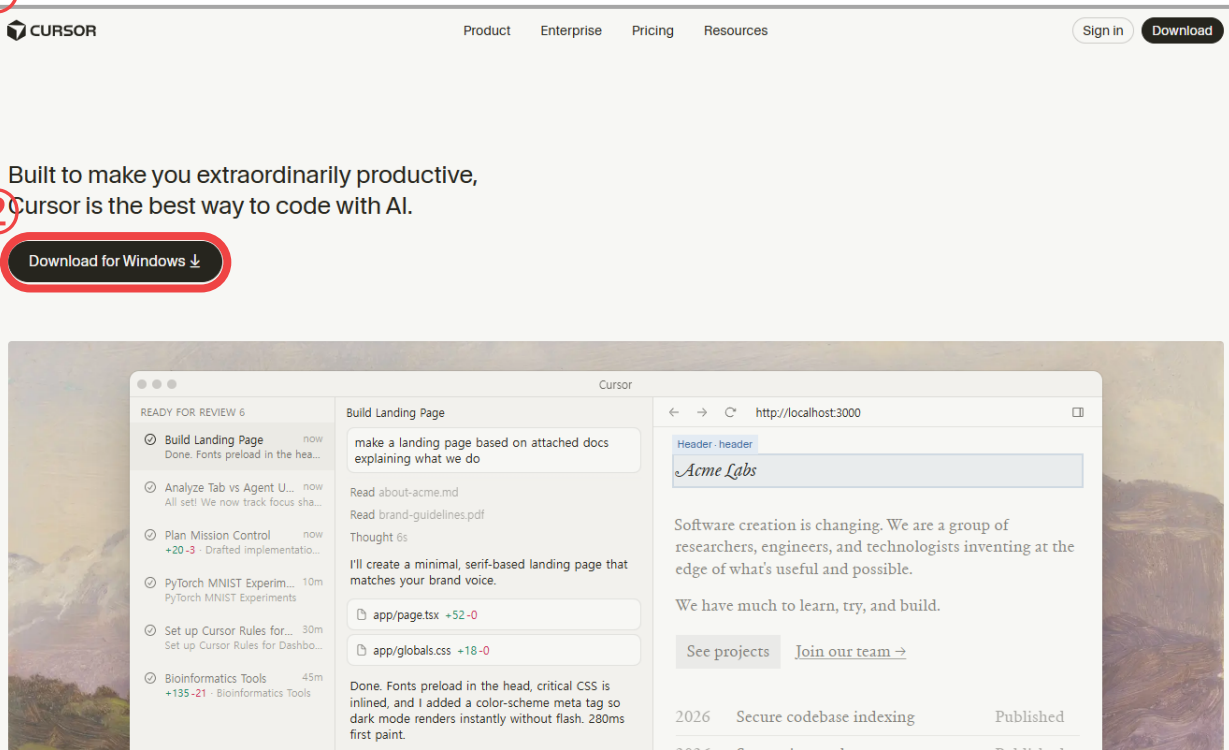


※ 해당 폴더에 requirements.txt 파일이 있어야 함

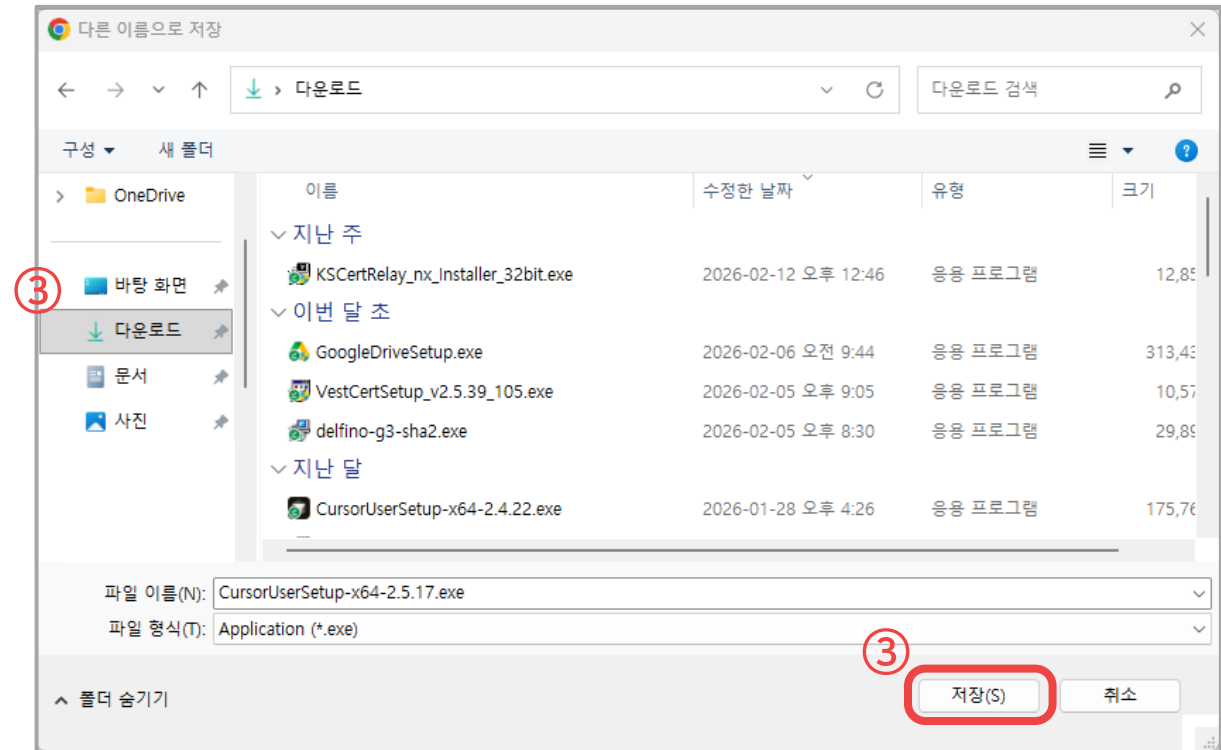
실습준비 _ ③ Cursor AI 설치

① cursor.com 접속 → ② Download for Windows 클릭 → ③ 다운로드 폴더 저장 → ④ 프로그램 설치

①



②



③

③

실습준비 _ ④ Git 설치

① 크롬에서 **Git 다운로드** 검색 → ② **Git - Install** 클릭 → ③ **Windows** 클릭 * 개인 PC 환경에 맞춰서 클릭

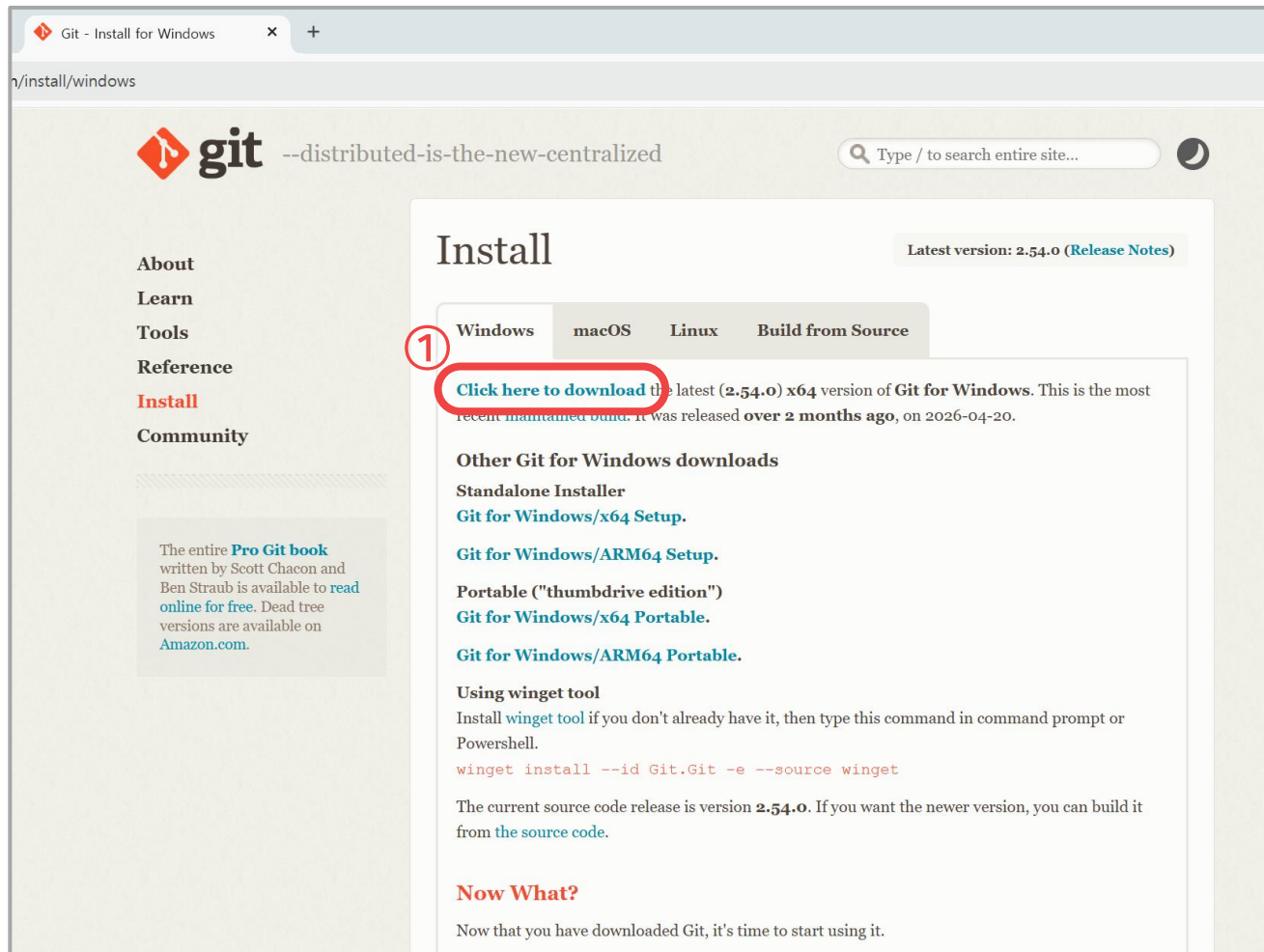
The image consists of two overlapping browser window screenshots illustrating the steps to download and install Git.

Left Screenshot (Step 1): A Google search page for "Git 다운로드". The search bar contains "Git 다운로드" (circled with a red 1). The first search result is for "Git" from "https://git-scm.com", with the link "Git 설치" (circled with a red 2). Below this, there is a link to "Git - Install" (https://git-scm.com/install) which is also circled with a red 2.

Right Screenshot (Step 2): The "Git - Install" page. The "Install" section is highlighted with a red 3. Under the "Install" heading, the "Windows" button is circled with a red 3. The page also shows the "Latest version: 2.54.0 (Release Notes)" and a prompt to "Choose your operating system above."

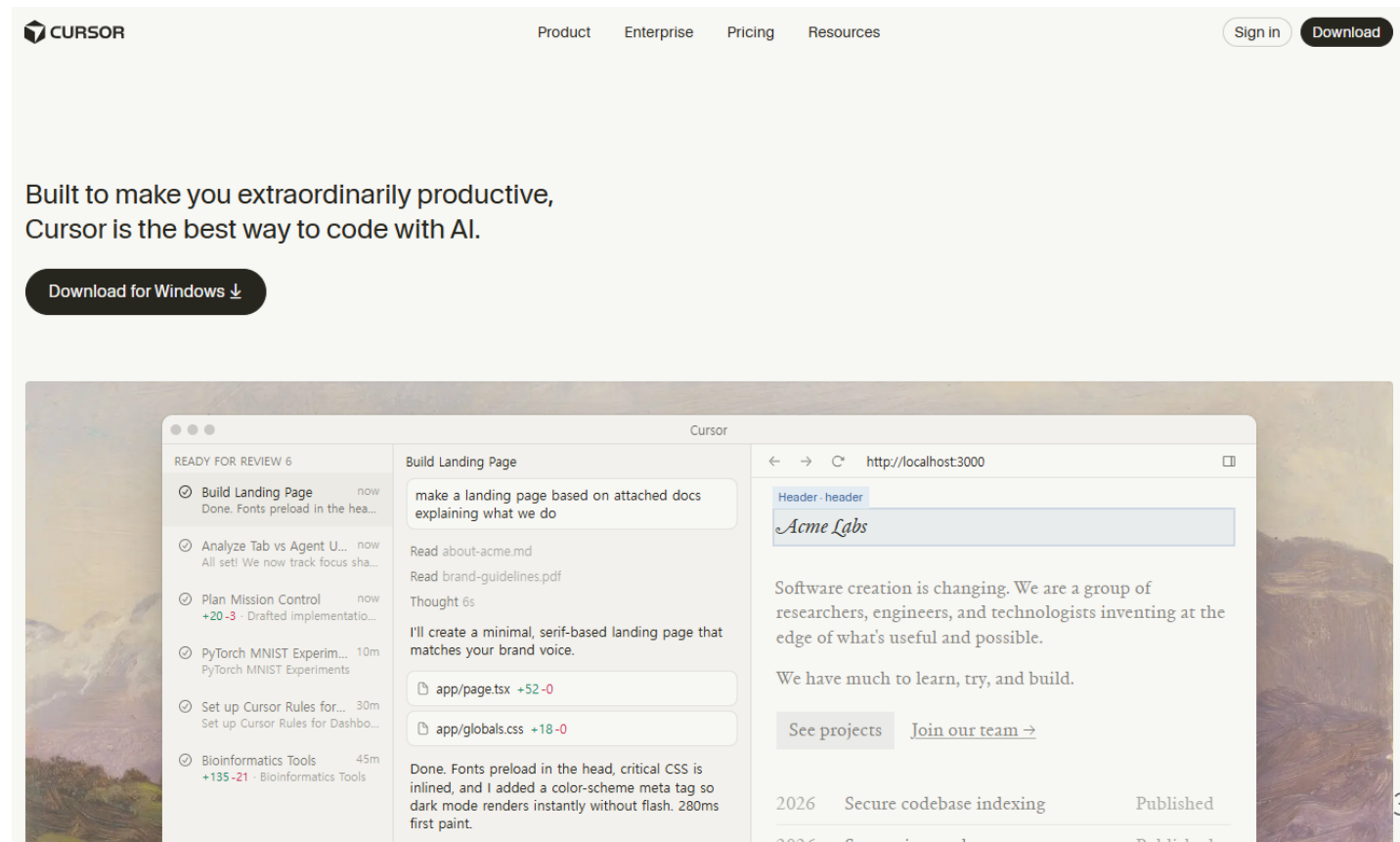
실습준비 _ ④ Git 설치

① Click here to download 클릭 → 다운로드 폴더에 저장 → 다운로드 폴더로 이동 → 설치파일 더블클릭



Cursor 개요

1. VS Code 기반의 'AI 네이티브' 에디터
2. 에디터 통합으로 프로젝트 전체 문맥(Context) 이해
3. 자연어 명령으로 멀티 파일 생성 및 수정
4. 개발자 의도를 예측하는 실시간 코드 완성
5. 최신 LLM 모델 선택 및 데이터 보안 지원



Cursor 주요 기능

1. 탭(Tab) : 사용자의 패턴을 학습하는 스마트 코드 자동 완성 기능

여러 줄 편집, 놓친 오류 수정, 다음 작업 예측 가능

2. 에이전트(Agent): 복잡한 코드 변경을 위한 AI 페어 프로그래머

정확한 컨텍스트 제어와 자동 수정으로 대규모 편집 가능

3. Cmd-K: 빠른 인라인 코드 편집 및 생성 기능

작업 흐름을 방해하지 않고 정확한 변경 가능

4. 채팅(Chat): 코드베이스를 이해하는 컨텍스트 인식 AI 어시스턴트

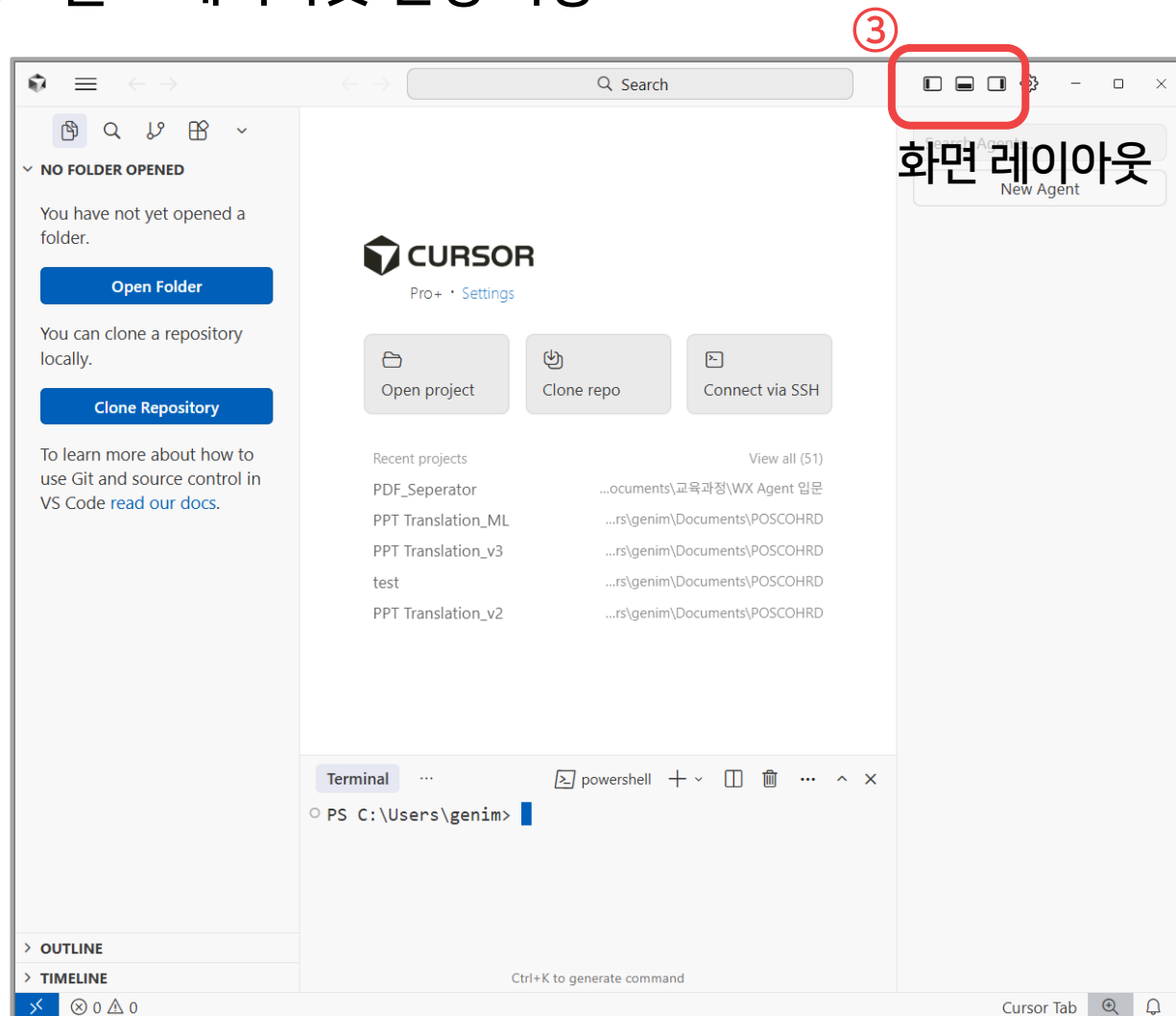
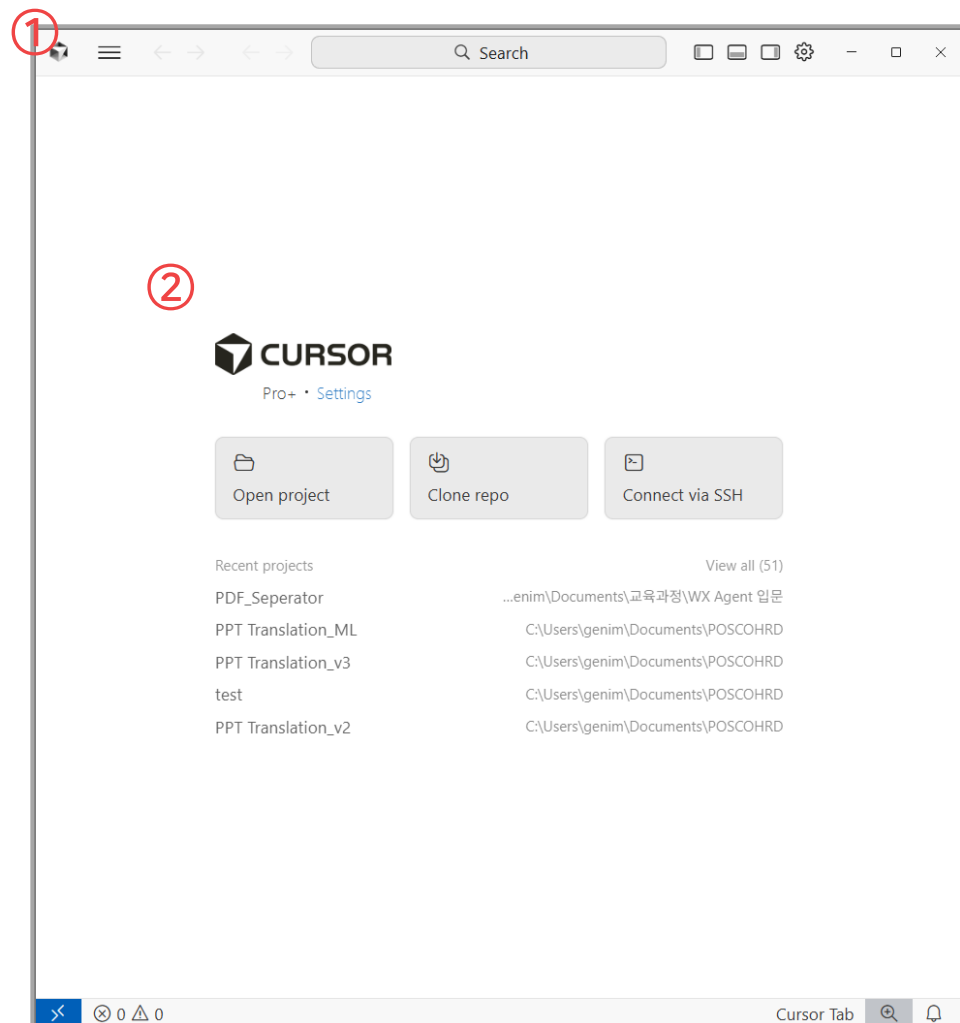
답변을 얻고 코드 변경사항을 에디터에 직접 적용

Cursor 주요 단축키

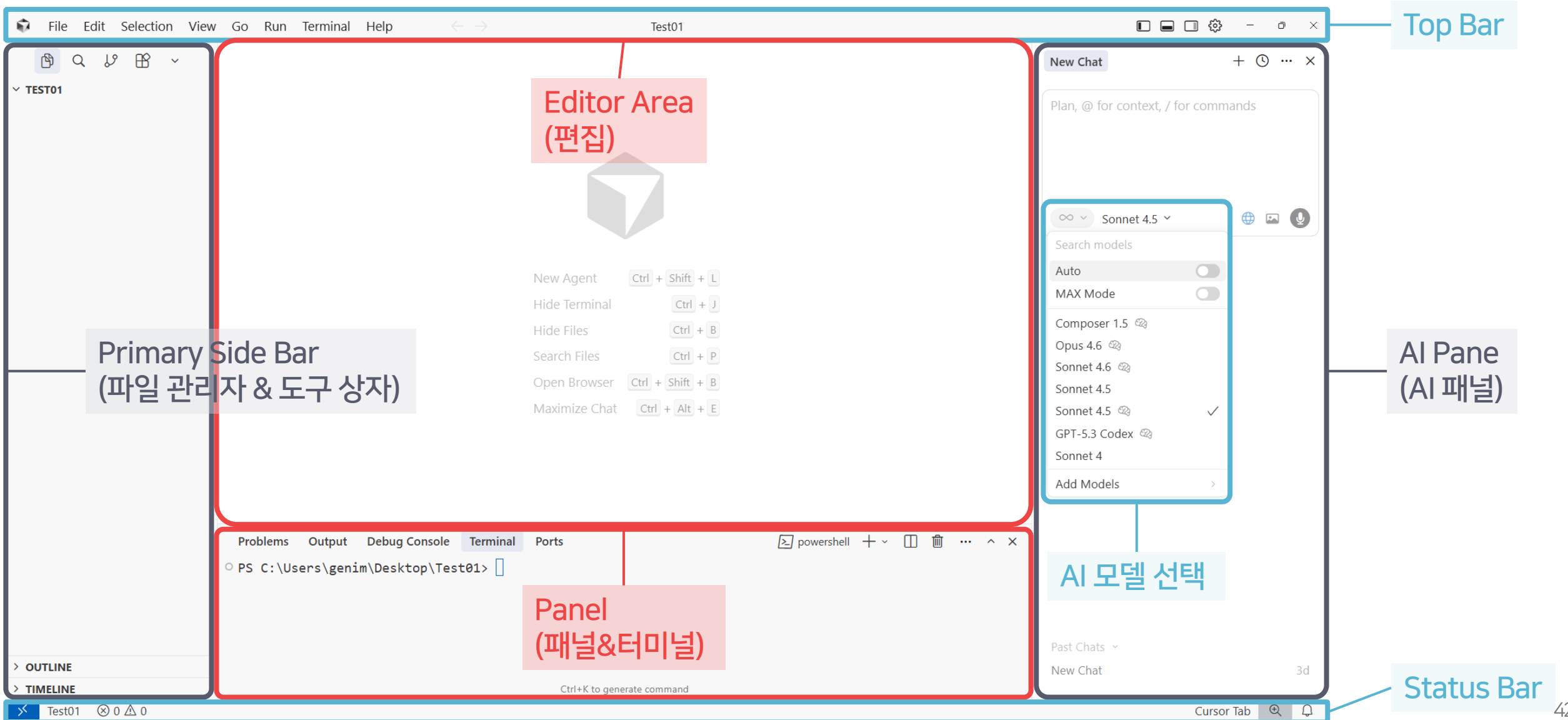
분류	기능	Windows	macOS	설명
AI	인라인 편집	Ctrl + K	Cmd(⌘) + K	현재 라인이나 선택한 범위에서 즉석으로 AI 편집 및 코드 생성
	통합 AI 인터페이스	Ctrl + I	Cmd(⌘) + I	사이드바 채팅창을 통해 코드 질문이나 로직 설명을 요청
	AI Chat	Ctrl + L	Cmd(⌘) + L	질문, 편집, 에이전트 모드가 포함된 Composer를 열어 프로젝트 전체 대상으로 작업
	코드 자동완성 수락	Tab	Tab	AI 기반의 코드 자동완성을 활성화하고 제안된 내용을 수락
설정	Cursor 설정 메뉴 열기	Ctrl + Shift + J	Cmd(⌘) + Shift(⌘) + J	사용할 AI 모델 선택 등 Cursor 전용 설정 메뉴 열기
	명령 팔레트/에디터 설정	Ctrl + Shift + P	Cmd(⌘) + Shift(⌘) + P	스타일 설정 인터페이스를 열거나 모든 명령어를 검색 및 실행
탐색	파일 검색	Ctrl + P	Cmd(⌘) + P	프로젝트 내의 파일을 이름으로 빠르게 찾아 열기
	코드 정의로 이동	F12	F12	선택한 함수나 변수가 정의된 실제 위치로 즉시 이동
	전체 프로젝트 텍스트 검색	Ctrl + Shift + F	Cmd(⌘) + Shift(⌘) + F	프로젝트 전체 파일 내에서 특정 텍스트를 검색
편집	코드 포매팅	Shift + Alt + F	Shift + Opt(⌥) + F	코드를 규칙에 맞게 자동으로 정렬
	주석 처리 / 해제	Ctrl + /	Cmd(⌘) + /	현재 줄이나 선택 영역을 주석으로 처리하거나 해제
	멀티 커서	Ctrl + D	Cmd(⌘) + D	동일한 단어를 찾아 다중 선택하여 한 번에 편집
화면	사이드바 제어	Ctrl + B	Cmd(⌘) + B	왼쪽 사이드바 탐색창을 보이거나 숨기기
	하단 패널 제어	Ctrl + J	Cmd(⌘) + J	터미널이나 출력 창이 있는 하단 패널을 열거나 닫기

Cursor 실행

①Cursor 실행 → ②메인화면 확인 → ③토글로 레이아웃 변경 가능

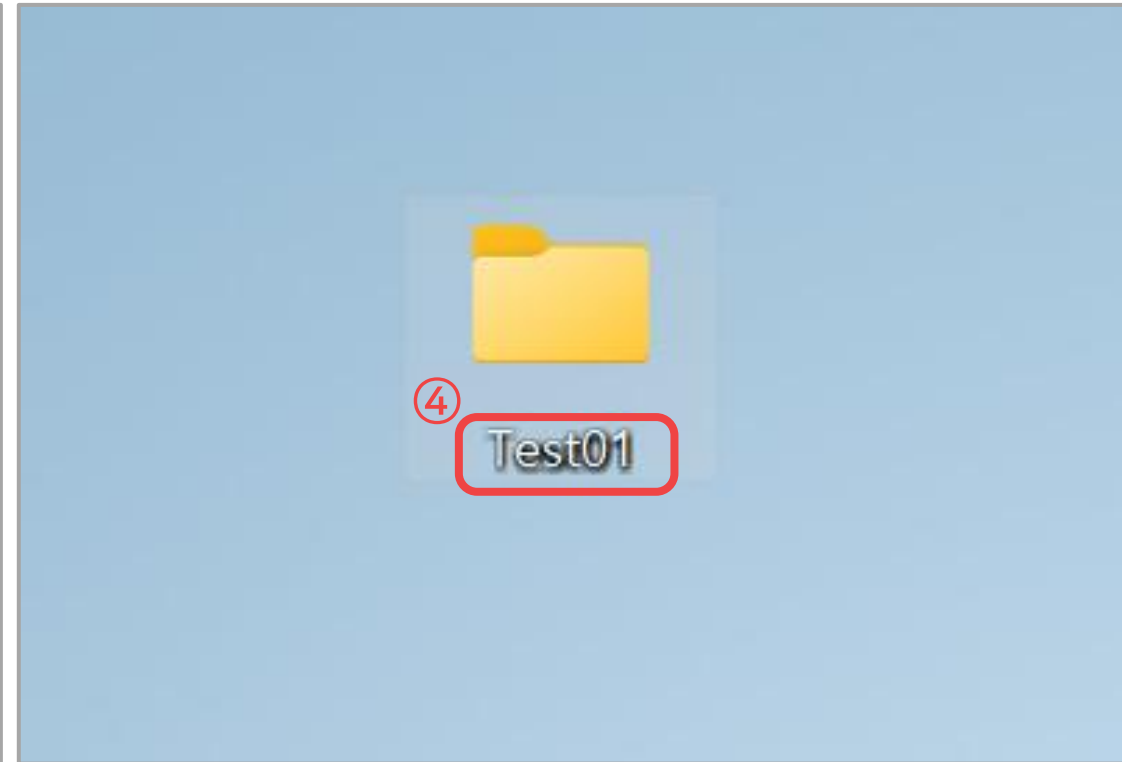
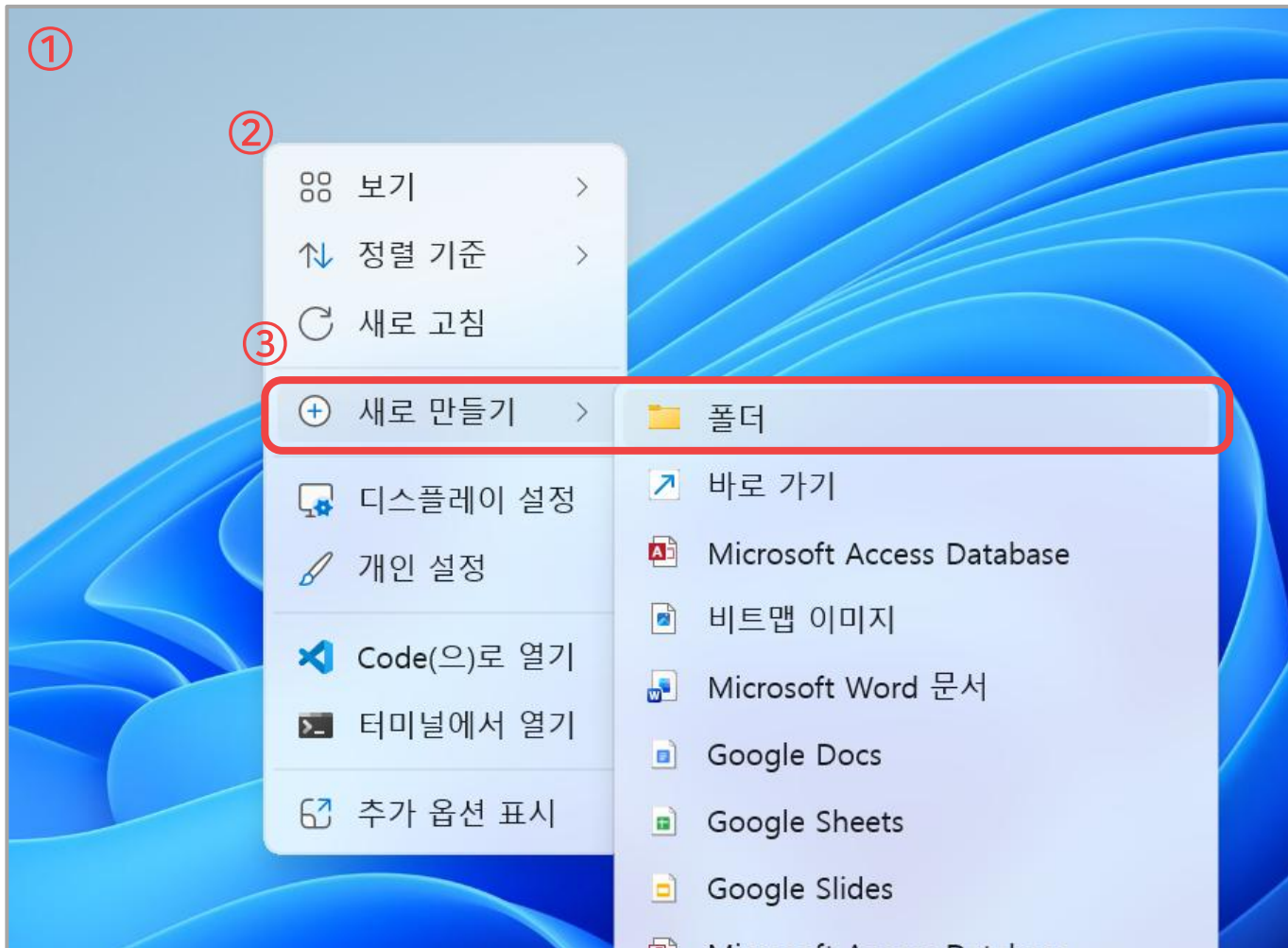


Cursor 실행 _ 화면 구성



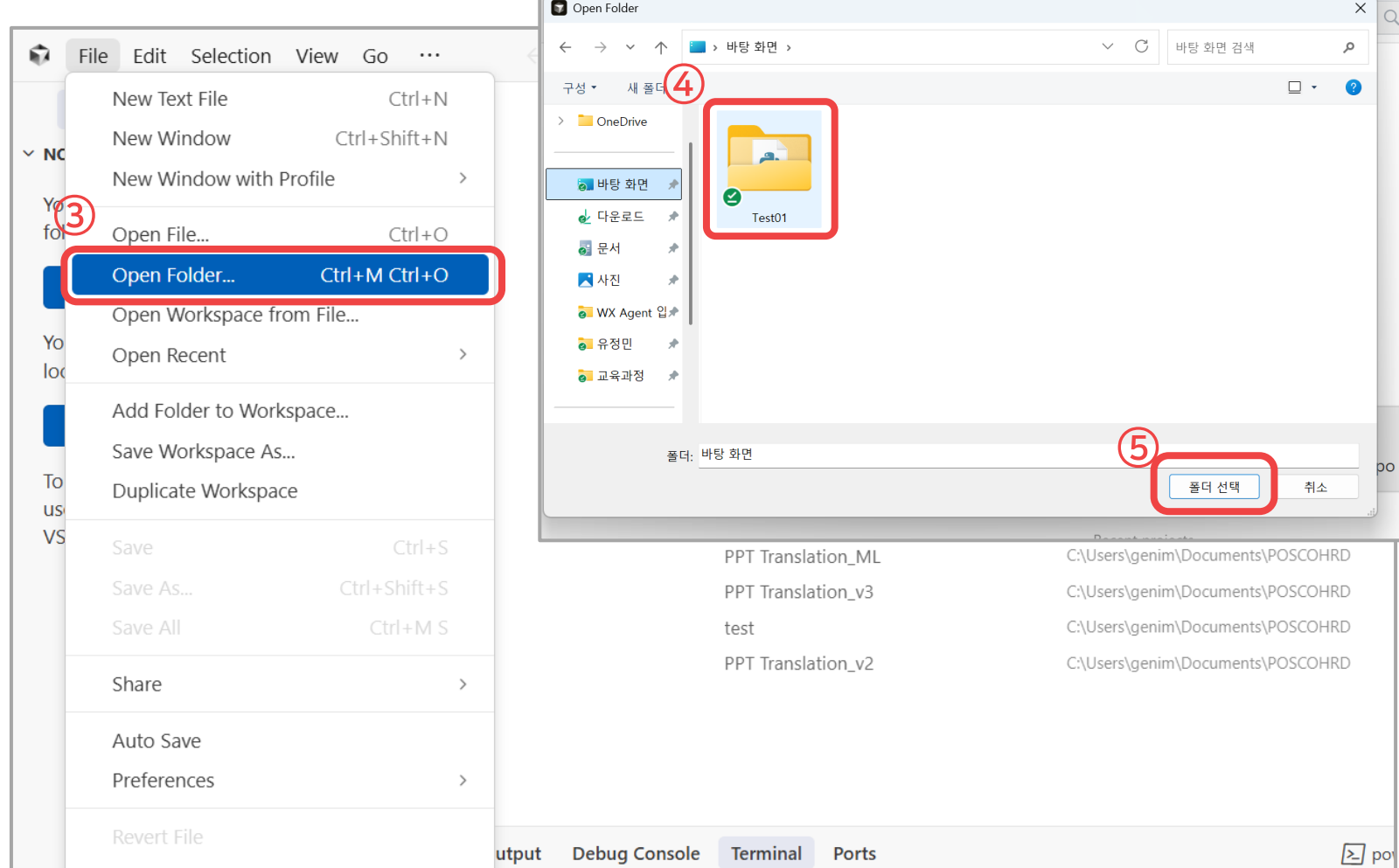
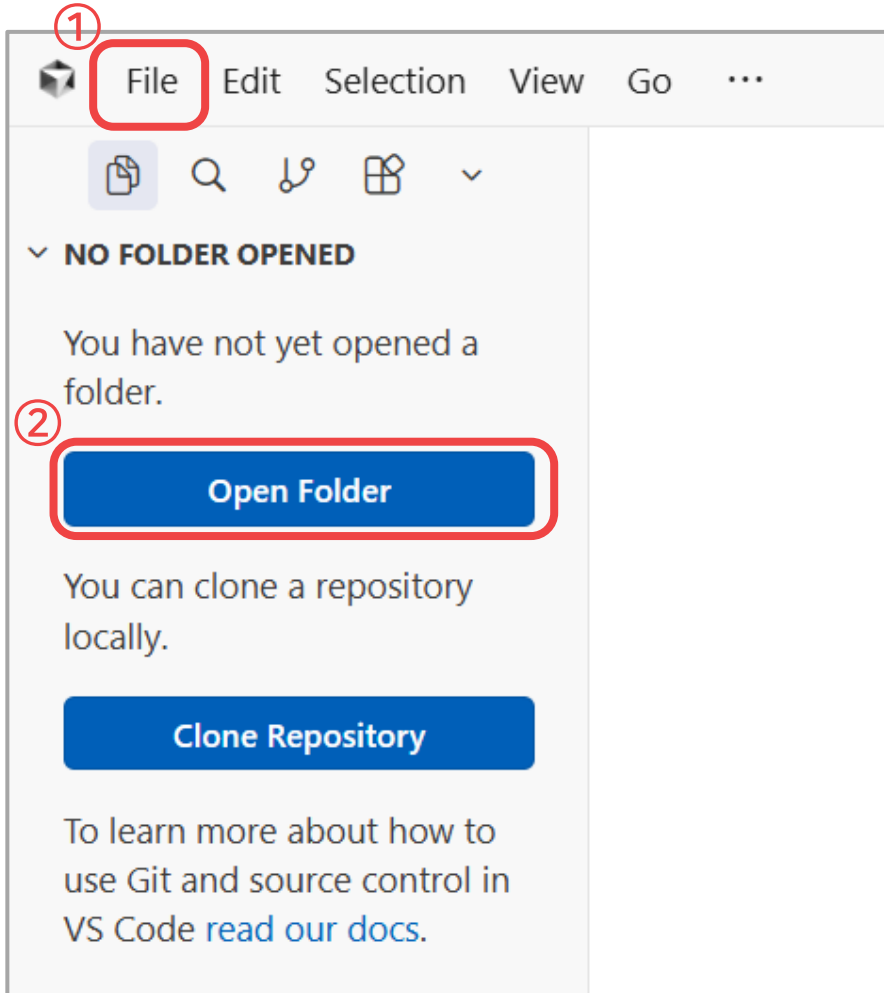
Cursor 시작 _ 바탕화면 실습 폴더 만들기

①바탕화면 접속 → ②마우스 오른쪽 클릭 → ③새로만들기 + 폴더 클릭 → ④파일명(Test01) 작성 → ⑤Enter



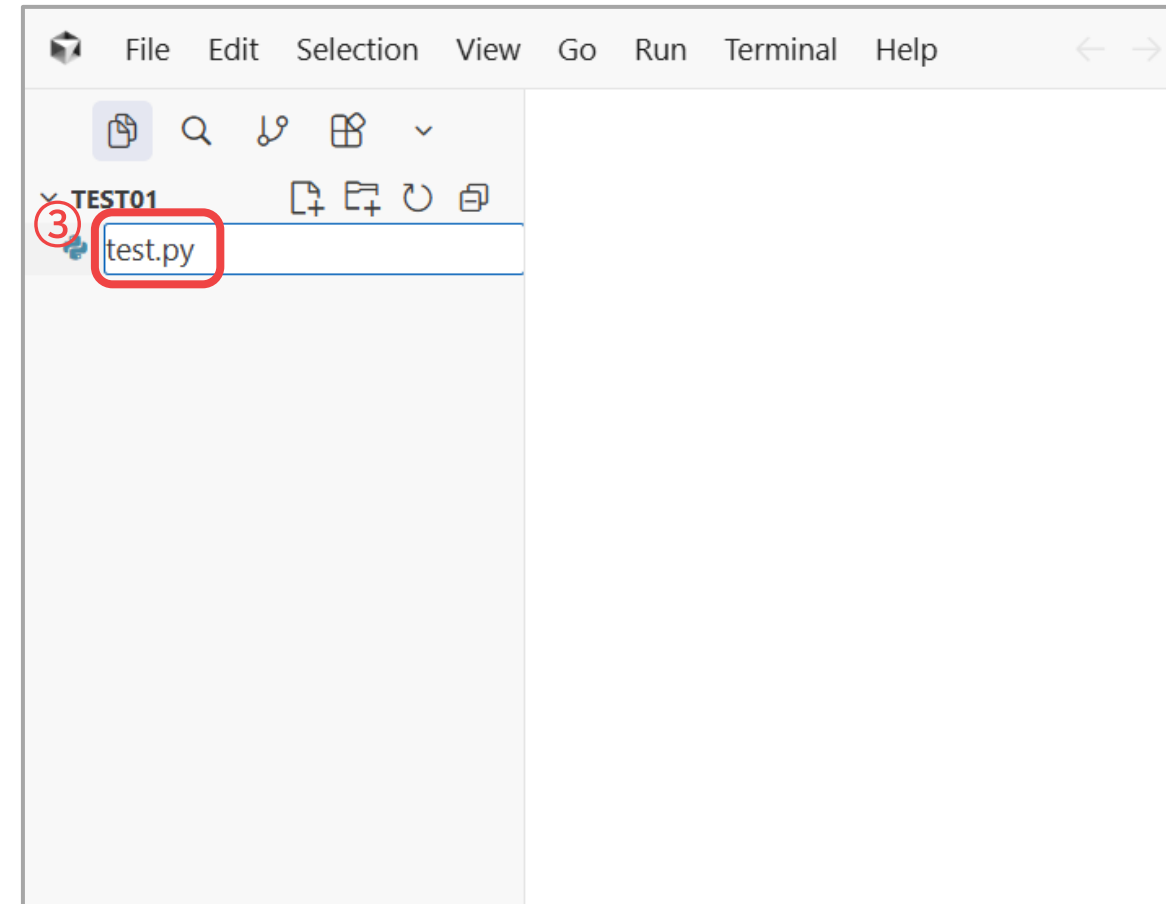
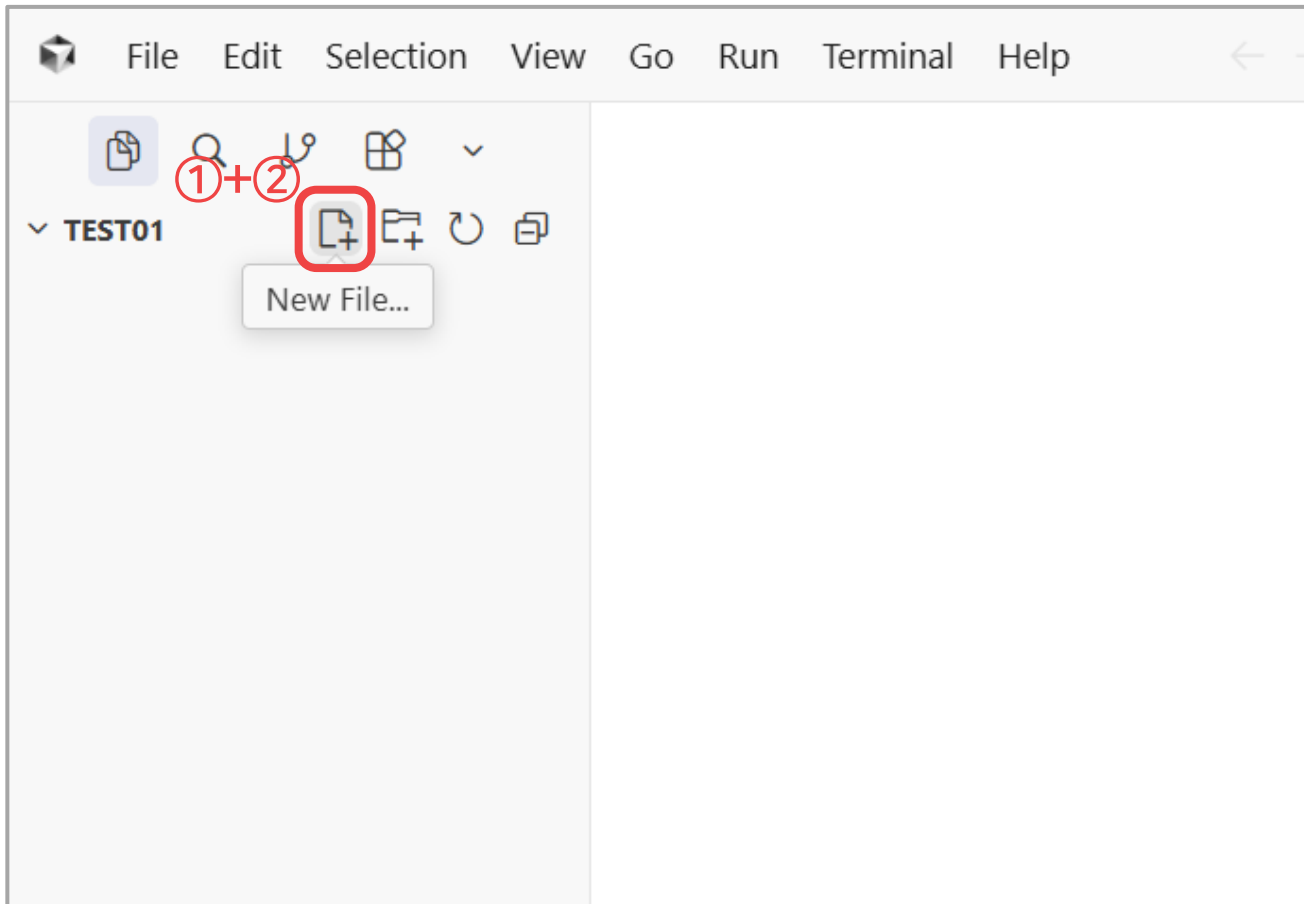
Cursor 실행 _ 폴더 열기

①File 클릭 → ②Open Folder 클릭 → ③ Open Folder... 클릭 → ④바탕화면-test01선택→ ⑤폴더 선택 클릭



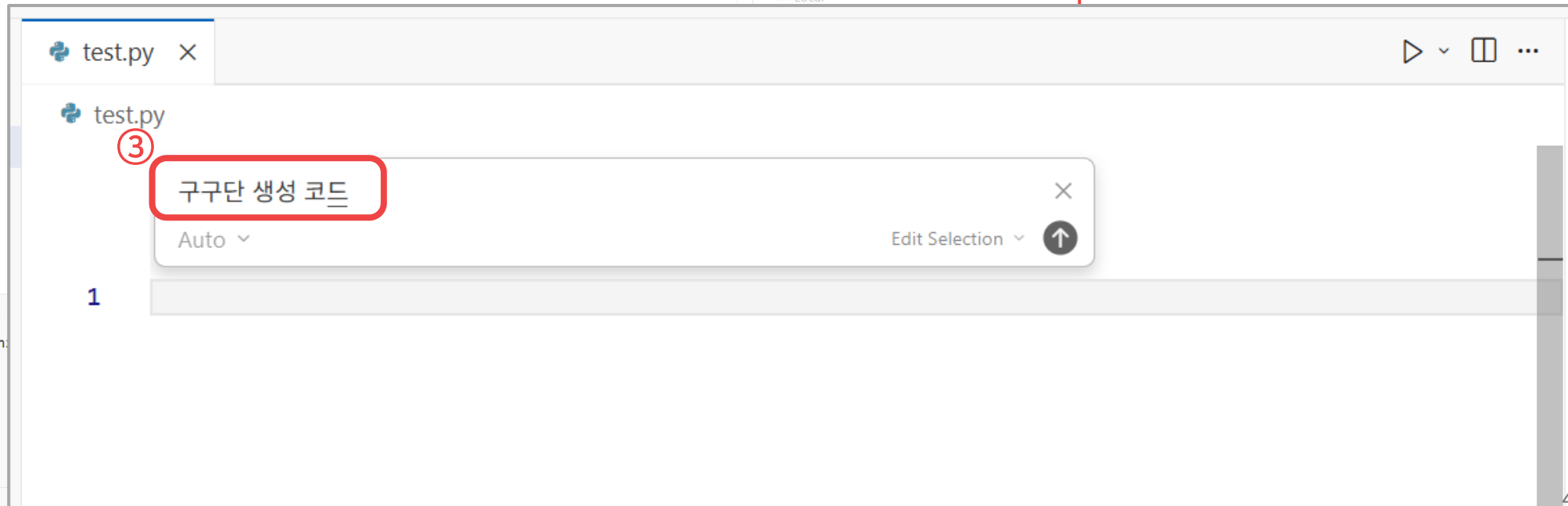
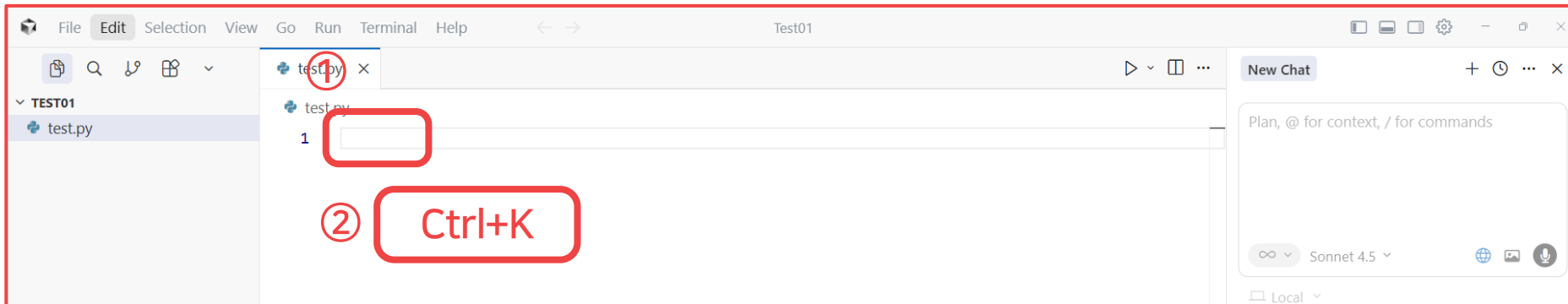
Cursor 실습 _ 실습파일 생성

①폴더명 옆에 **마우스 커서** 이동 → ②**문서+ icon** 클릭 → ③파일명 **test.py** 생성



Cursor 실습 _ Ctrl+K

①편집창 클릭 → ②Ctrl+K(인라인 편집 or 코드 생성 실행)→ ③구구단 생성 코드 입력 → ④Enter



Cursor 실습 _ Ctrl+K

Undo or Keep 클릭



Cursor 실습 _ AI Pane 활용

① 구구단 출력하는 코드 만들어줘 입력 → ② Enter → ③ Undo or Keep 클릭

The screenshot illustrates the workflow for using the AI Pane in the Cursor IDE to generate code. It shows three overlapping windows:

- Top Left Window:** A 'New Chat' dialog with the placeholder text 'Plan, @ for context, / for commands' and the model 'Sonnet 4.5' selected.
- Top Right Window:** A 'New Chat' dialog where the prompt '구구단 출력하는 코드 만들어줘' (Generate code for printing multiplication table) is entered. A red circle with the number '1' is next to the prompt, and a red box highlights the prompt and the 'Enter' key icon.
- Bottom Window:** The main IDE window showing a Python file 'test.py'. The code generated by the AI is:

```
1 for i in range(2, 10):
2     print(f"\n[{i}단]")
3     for j in range(1, 10):
4         print(f"{i} x {j} = {i*j}")
```

A red box highlights the code, and a red arrow points from the 'Keep 선택시' (When selecting Keep) label to the 'Keep Ctrl+Shift+Y' button in the IDE's toolbar.

On the right side, the AI Pane shows the generated code and a thought process:

구구단 출력 코드

Now · +2 · Sonnet 4.5

구구단 출력하는 코드 만들어줘

```
for i in range(2, 10):
    print(f"\n[{i}단]")
    for j in range(1, 10):
        print(f"{i} x {j} = {i*j}")
```

Thought for 1s

구구단 출력 코드를 완성했습니다!

이 코드는 2단부터 9단까지 구구단을 출력합니다. 각 단의 제목을 표시하고, 곱셈 결과를 보기 좋게 출력합니다.

실행하려면 터미널에서 `python test.py`를 입력하시면 됩니다.

M2. 바이트코딩 활용 앱/웹 개발

4. 바이트코딩 기반 앱 개발

기획과 설계 프롬프트 작성

바이트 코딩 프롬프트는 Why와 What을 명확하게 하는 것, How는 LLM이 수행



【필요 역량】

도메인 지식 + Process분석 역량 + IT 지식



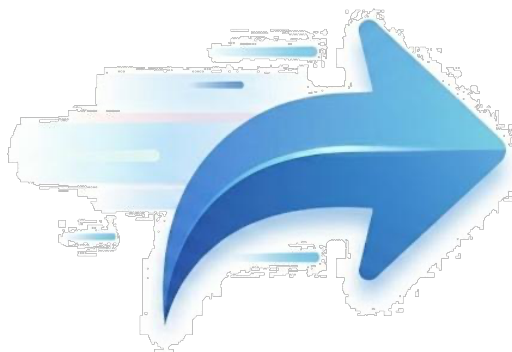
기획과 설계 프롬프트 작성

웹/앱 개발 프로세스: 웹/앱의 Why, What 생각 정리에서 프롬프트 설계까지



Why & What 생각정리

- 웹 서비스의 목적은 무엇인가?
- 보여줄 콘텐츠나 기능은 무엇인가?
- 메뉴는 어떻게 구성할 것인가?
- 메뉴별 콘텐츠 구성은 어떻게 할까?
- 데이터베이스 항목은?
- 개발 환경은 어떻게 할 것인가?



프롬프트 설계

- 웹 서비스 목적 정의

- 개발 환경 및 패키지

개발환경

- 데이터 베이스 및 백엔드 프로세스 구성

- 메뉴별 콘텐츠, 백엔드 역할

노드설명

- 웹 페이지의 컬러, 폰트 구성

UI설명

- 초기 화면 구성, 메뉴 위치 및 기능

웹/앱/웹앱/하이브리드 앱

구분	특징	활용	설치여부
웹 (web)	- 네이버, 블로그, 뉴스 사이트 - URL로 접근하고, 콘텐츠를 읽고 탐색하는 "페이지" 중심	- 네이버, 블로그, 뉴스 사이트 - URL로 접근하고, 콘텐츠를 읽고 탐색하는 "페이지" 중심	✗ (URL만 있으면 됨)
앱 (App, Native App)	기기 안에 들어온 프로그램	- 카카오톡, 인스타그램, 게임 앱 - 설치 후 OS에서 직접 실행하고, 알림·카메라·GPS 등 기기 기능 활용	◯ (앱스토어/플레이스토어)
웹앱 (Web App)	설치 안 해도 앱처럼 쓰는 서비스	- Gmail, Notion, Google Docs, EP - 로그인 후 작성·저장·수정하며, 새로고침 없이 화면이 계속 변함	✗ (기본은 브라우저) - 로그인 필요 ◯
하이브리드 앱 (Hybrid App)	웹앱을 앱 껍데기 안에 넣은 것	- 쿠팡 일부 화면 등 - 앱스토어에 올라가지만 내부 일부 화면은 HTML/CSS/JS 웹앱으로 구성	

■ 구분 기준: “설치 여부 → 실행 위치 → 기능과 데이터 처리 → 기기 기능 활용” 순서로 판단

1. 설치해야 하면? → 앱

2. 설치 안 하는데 기능이 많고
로그인/데이터 처리? → 웹앱

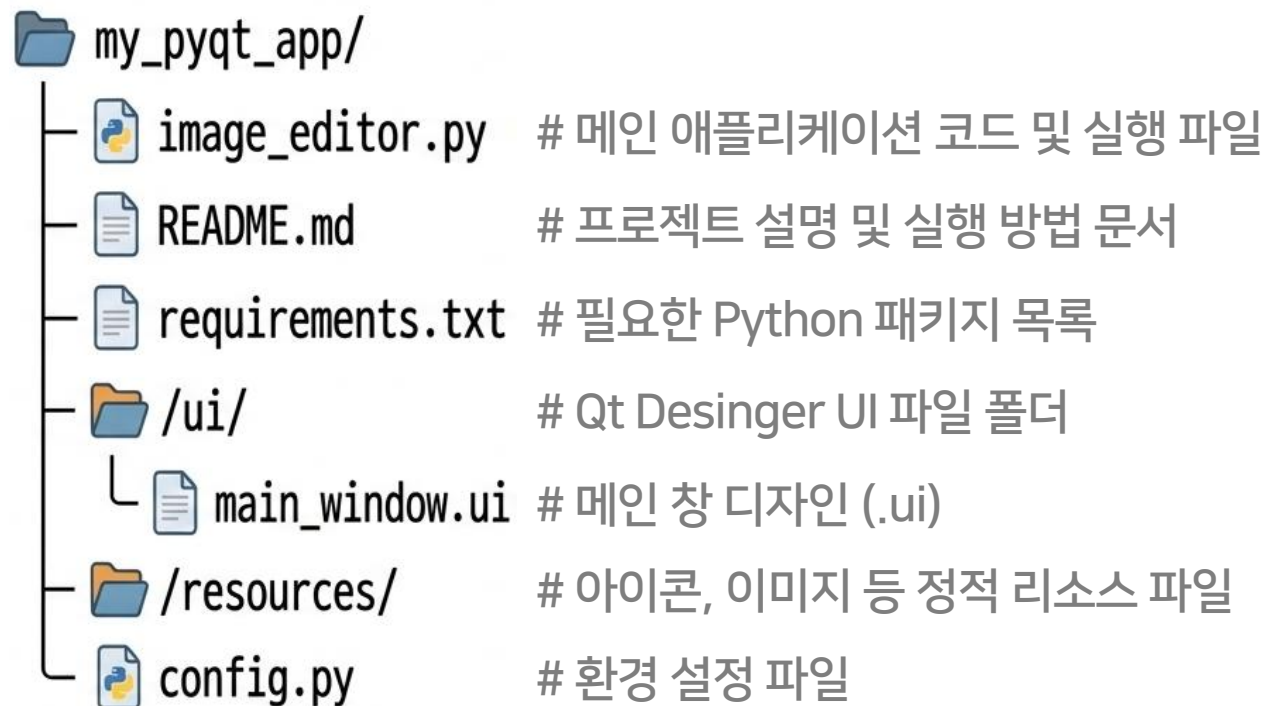
3. 그냥 정보 보기 위주? → 웹

앱 개발을 위한 GUI 이해(PyQt5)

Qt5 어플리케이션 프레임워크에 대한 파이썬 버전, 플랫폼에 관계없이 다양한 기능을 포함하는 개발툴

특징	
주요 특징	내용
크로스 플랫폼	Windows, macOS, Linux 등 다양한 운영체제에서 동일한 코드 실행 가능
다양한 위젯	버튼, 입력창, 표, 트리 등 어플리케이션 구성에 필요한 UI 요소 기본 포함
시그널 & 슬롯	이벤트 발생(시그널)시 특정 함수(슬롯)가 호출 되도록 연결하는 PyQt의 핵심 메커니즘
Qt Designer 지원	GUI 디자인 도구로 U를 시각적으로 디자인하고, .ui 파일로 저장
강력한 멀티미디어	오디오, 비디오, 2D/3D 그래픽 기능까지 포함하여 다양한 기능의 어플리케이션 개발이 가능

구조



앱 개발 실습 절차

① 프롬프트 만들기

- ChatGPT or Gemini에서 Cursor AI에 요청할 프롬프트 만들기

② 실습 폴더 생성

- 실습 폴더 아래 새 폴더(PDF2) 만들기

③ Cursor 실행

- Cursor 실행, 실습 폴더 열기, 파이썬 패키지 설치

④ New Chat 클릭

- 오른쪽 사이드바에서 New Chat 클릭, 프롬프트 입력

⑤ 진행사항 체크

- 모델 선택, 진행 여부(폴더 생성, 패키지 설치 등) 승인(Run) 클릭

Cursor

⑥ 앱 테스트

- 실습파일 업로드 후 앱 테스트, 이상 여부 확인

⑦ 오류 보완

- 오류 발생 화면 캡처, 오류발생 프롬프트 작성, 재실행

Cursor

⑧ 실행파일 생성

- 에러 해결 후, 실행파일로 만들기

앱 개발 실습1. PDF 분할/병합

▶ 실습목표: PDF 분할/병합 앱 만들기

The screenshot shows a desktop application window titled "PDF 분할 및 병합 도구". It has two tabs: "PDF 분할" (selected) and "PDF 병합". The interface is divided into three main sections:

- 1. PDF 파일 선택**: A section with the text "선택된 파일이 없습니다" and a blue button labeled "파일 선택".
- 2. 페이지 미리보기**: A large, empty rectangular area for previewing the PDF pages.
- 3. 분할 옵션 선택**: A section with three radio button options:
 - ☒ 범위 입력 (예: 1-5, 10-15): Includes a text input field with the example "예: 1-5, 10-15".
 - ☐ 특정 페이지 입력 (예: 1, 3, 5): Includes a text input field with the example "예: 1, 3, 5, 7".
 - ☐ 고정 페이지 분할 (균등 분할): Includes a label "페이지 수:" followed by a spinner box set to "2" and the text "페이지씩 분할".

At the bottom of the window is a green button labeled "분할 실행".

1. 기능 : PDF 분할/병합
2. 메뉴 : 분할(범위 입력, 페이지 입력, 고정페이지 분할)
병합(파일 선택, 병합버튼)
3. 데이터베이스 : 없음
4. 개발환경 : PyQt5, PyPDF2

앱 개발 실습1. PDF 분할/병합

① 프롬프트 만들기 : 초안_기본적인 내용 작성해서 LLM에 요청



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
PDF 분할/병합하는 앱을 만들거야

1. 개발환경

- PDF 업로드는 'pypdf'(또는 PyPDF2) 패키지 사용

2. 노드설명

- pdf 업로드 기능
- 분할 기능 : 범위 입력, 페이지 입력, 고정페이지 분할
- 병합 기능 : 파일 선택, 병합버튼

3. UI설명

- PyQt5 인터페이스, 심플하고 직관적인 기본 수준
- pdf 썸네일 미리보기 UI
- 분할 : 범위 입력창, 페이지 입력창, 고정페이지 입력창
- 병합 : 파일 선택 버튼, 병합 버튼

앱 개발 실습1. PDF 분할/병합

① 프롬프트 만들기 : 완성_LLM이 만들어준 프롬프트

Prompt

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- GUI: PyQt5 (심플하고 직관적인 기본 수준의 UI 구성, 탭 방식 활용 권장)
- PDF 처리: 'pypdf' (또는 PyPDF2)
- PDF 미리보기(썸네일) 렌더링: 'pypdf'로는 썸네일 이미지 추출이 어려우므로 'PyMuPDF(fitz)' 또는 'pdf2image' 중 가볍고 구현이 쉬운 것을 선택해서 적용해 줘.

핵심 기능 요구사항

1. 분할 (Split) 기능

- PDF 파일 업로드: 단일 PDF 파일을 선택하여 업로드할 수 있어야 함.
- 썸네일 미리보기: 업로드된 PDF의 각 페이지를 썸네일 형태로 미리 볼 수 있는 UI를 제공해.
- 분할 옵션 선택:
 - 1) 범위 입력 (예: 1-5)
 - 2) 특정 페이지 입력 (예: 1, 3, 5)
 - 3) 고정 페이지 분할 (예: 2페이지씩 균등 분할)
- 중요 분할 조건: 사용자가 지정한 분할 범위에 포함되지 않은 '나머지 페이지'들도 버려지지 않고 따로 분할되어 저장되어야 해.
- 저장 위치: 원본 PDF가 있는 경로에 새로운 폴더를 생성하여 저장해.
 - * 새 폴더명: '원제목_{분할된 총 파일 개수}' (예: document_3)
- 분할된 파일명 형식: '원제목_{분할된 페이지 정보}.pdf' (예: document_1-5.pdf, document_6-10.pdf 등)

2. 병합 (Merge) 기능

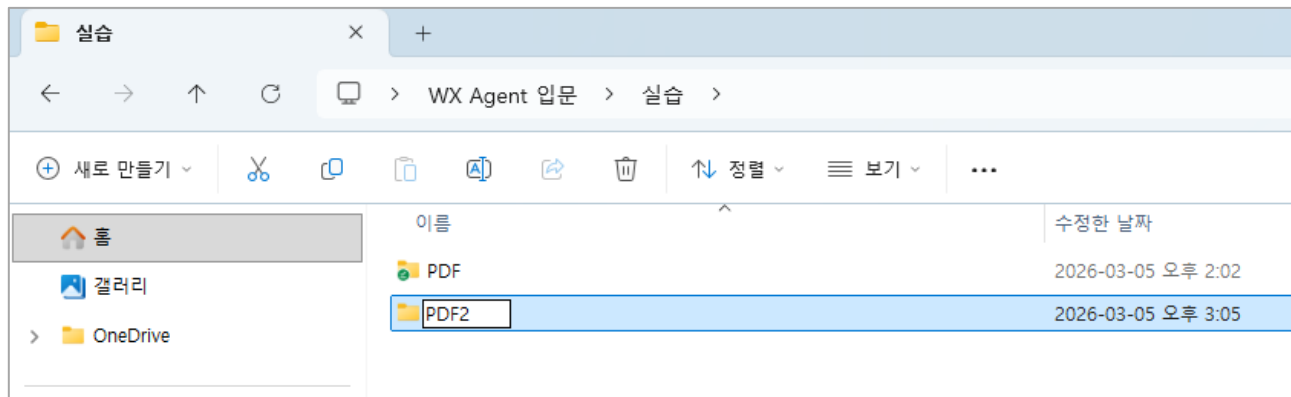
- PDF 다중 업로드: 여러 개의 PDF 파일을 한꺼번에 선택하고 업로드할 수 있어야 해 (목록 형태로 표시).
- 병합 실행: 업로드된 파일들을 하나의 PDF로 병합해.
- 병합 완료 미리보기: 병합이 완료되면, 완성된 PDF의 첫 페이지나 전체 썸네일을 미리보기로 보여주는 기능을 제공해.

추가 지시사항

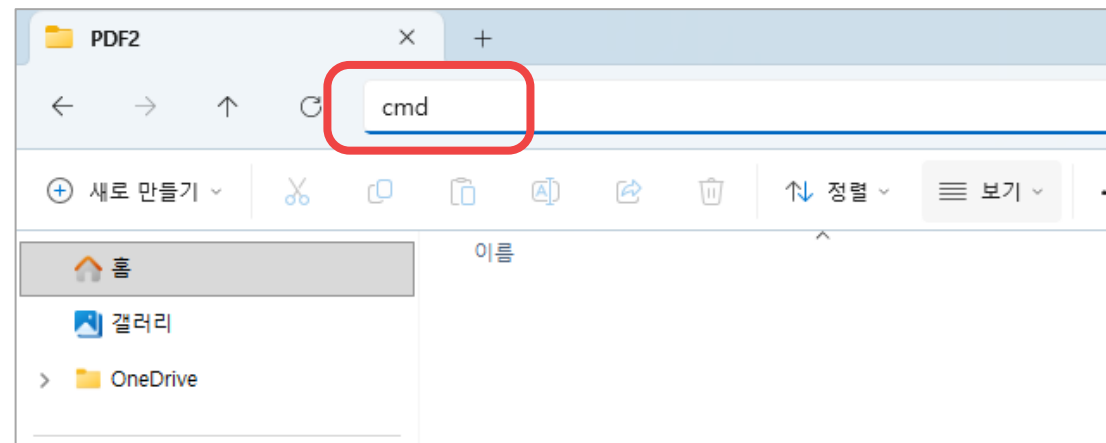
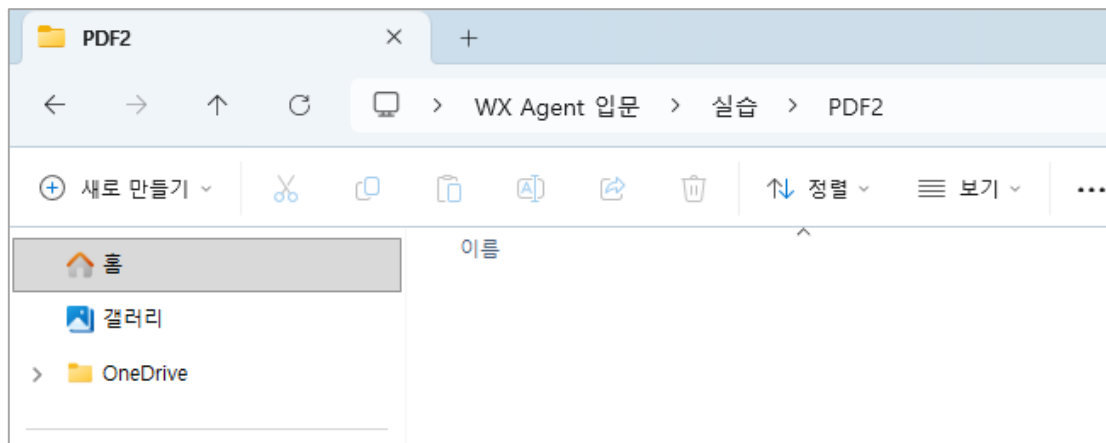
- 코드는 모듈화하여 가독성 있게 작성하고, 각 주요 클래스나 함수에 한국어로 주석을 달아줘.
- requirements.txt에 필요한 패키지 목록을 함께 출력해 줘.
- 에러 처리(예: 잘못된 페이지 범위 입력, 권한 부족 등)를 꼼꼼하게 적용해서 앱이 튕기지 않도록 해.

앱 개발 실습1. PDF 분할/병합

② 실습 폴더 생성 : 바탕화면 - WX Agent 폴더 - 실습 - 「PDF2」 폴더 생성

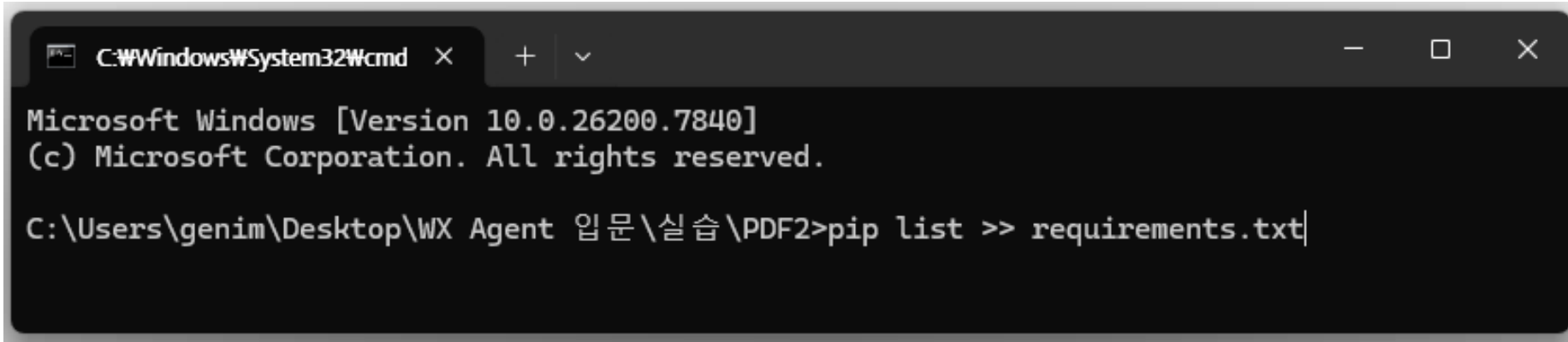


③ 개발 환경 설정 : 「PDF2」폴더 이동 → 주소창에 cmd 입력 → Enter



앱 개발 실습1. PDF 분할/병합

③ 개발 환경 설정 : `pip list >> requirements.txt`



```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\genim\Desktop\WX Agent 입문\실습\PDF2>pip list >> requirements.txt
```

③ 개발 환경 설정 : Cursor AI 실행 → File-Open Folder-바탕화면-WX Agent 입문-실습-PDF2 폴더 클릭



앱 개발 실습1. PDF 분할/병합

④ New Chat 클릭 : 프롬프트 입력 → 실행

Prompt

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- GUI: PyQt5 (심플하고 직관적인 기본 수준의 UI 구성, 탭 방식 활용 권장)
- PDF 처리: `pypdf` (또는 PyPDF2)
- PDF 미리보기(썸네일) 렌더링: `pypdf`로는 썸네일 이미지 추출이 어려우므로 `PyMuPDF(fitz)` 또는 `pdf2image` 중 가볍고 구현이 쉬운 것을 선택해서 적용해 줘.

핵심 기능 요구사항

1. 분할 (Split) 기능

- PDF 파일 업로드: 단일 PDF 파일을 선택하여 업로드할 수 있어야 함.
- 썸네일 미리보기: 업로드된 PDF의 각 페이지를 썸네일 형태로 미리 볼 수 있는 UI를 제공해.
- 분할 옵션 선택:
 - 1) 범위 입력 (예: 1-5)
 - 2) 특정 페이지 입력 (예: 1, 3, 5)
 - 3) 고정 페이지 분할 (예: 2페이지씩 균등 분할)
- **중요 분할 조건**: 사용자가 지정한 분할 범위에 포함되지 않은 '나머지 페이지'들도 버려지지 않고 따로 분할되어 저장되어야 해.
- 저장 위치: 원본 PDF가 있는 경로에 새로운 폴더를 생성하여 저장해.
- * 새 폴더명: `원제목\_{분할된 총 파일 갯수}` (예: document_3)
- 분할된 파일명 형식: `원제목\_{분할된 페이지 정보}.pdf` (예: document_1-5.pdf, document_6-10.pdf 등)

2. 병합 (Merge) 기능

- PDF 다중 업로드: 여러 개의 PDF 파일을 한꺼번에 선택하고 업로드할 수 있어야 해 (목록 형태로 표시).
- 병합 실행: 업로드된 파일들을 하나의 PDF로 병합해.
- 병합 완료 미리보기: 병합이 완료되면, 완성된 PDF의 첫 페이지나 전체 썸네일을 미리보기로 보여주는 기능을 제공해.

추가 지시사항

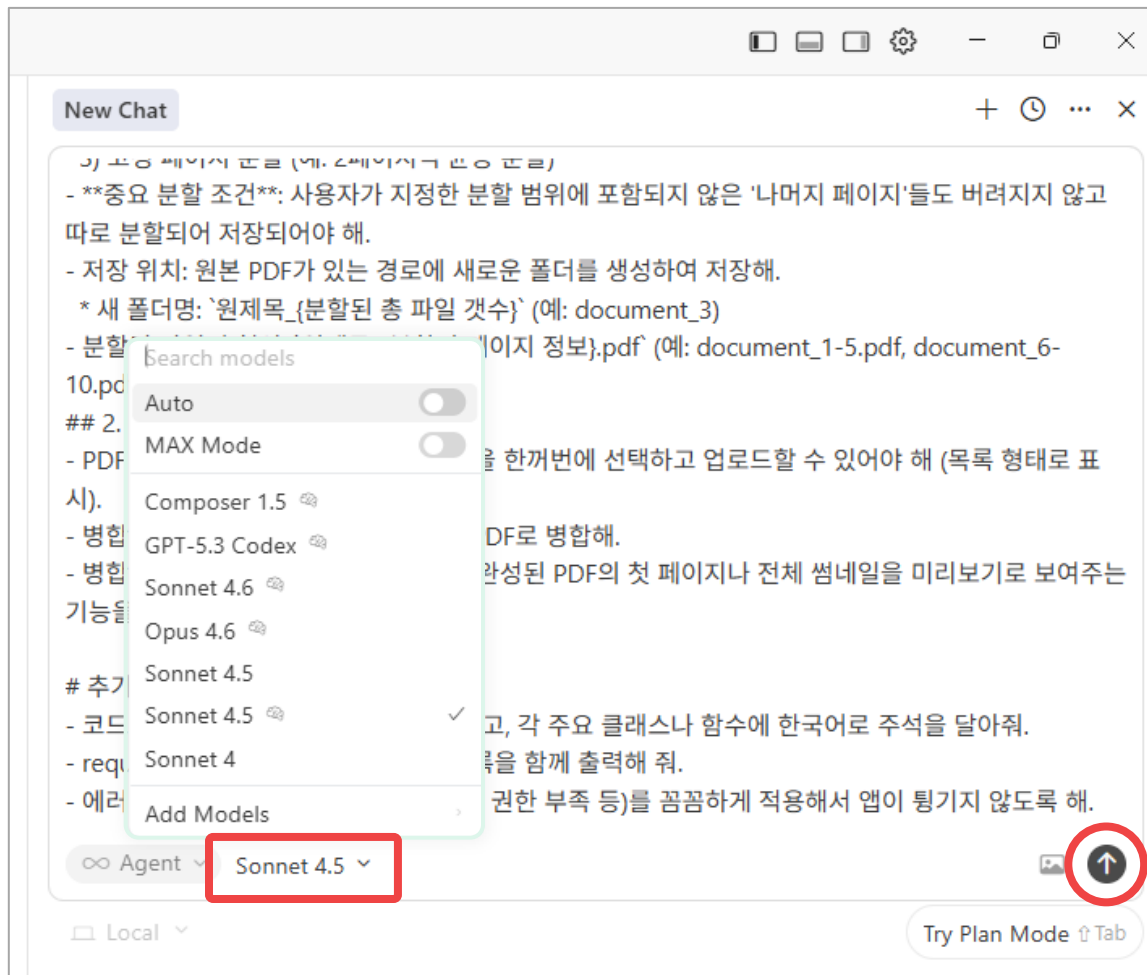
- 코드는 모듈화하여 가독성 있게 작성하고, 각 주요 클래스나 함수에 한국어로 주석을 달아줘.
- requirements.txt에 필요한 패키지 목록을 함께 출력해 줘.
- 에러 처리(예: 잘못된 페이지 범위 입력, 권한 부족 등)를 꼼꼼하게 적용해서 앱이 튼튼하게 작동하도록 해.

Cursor IDE Interface:

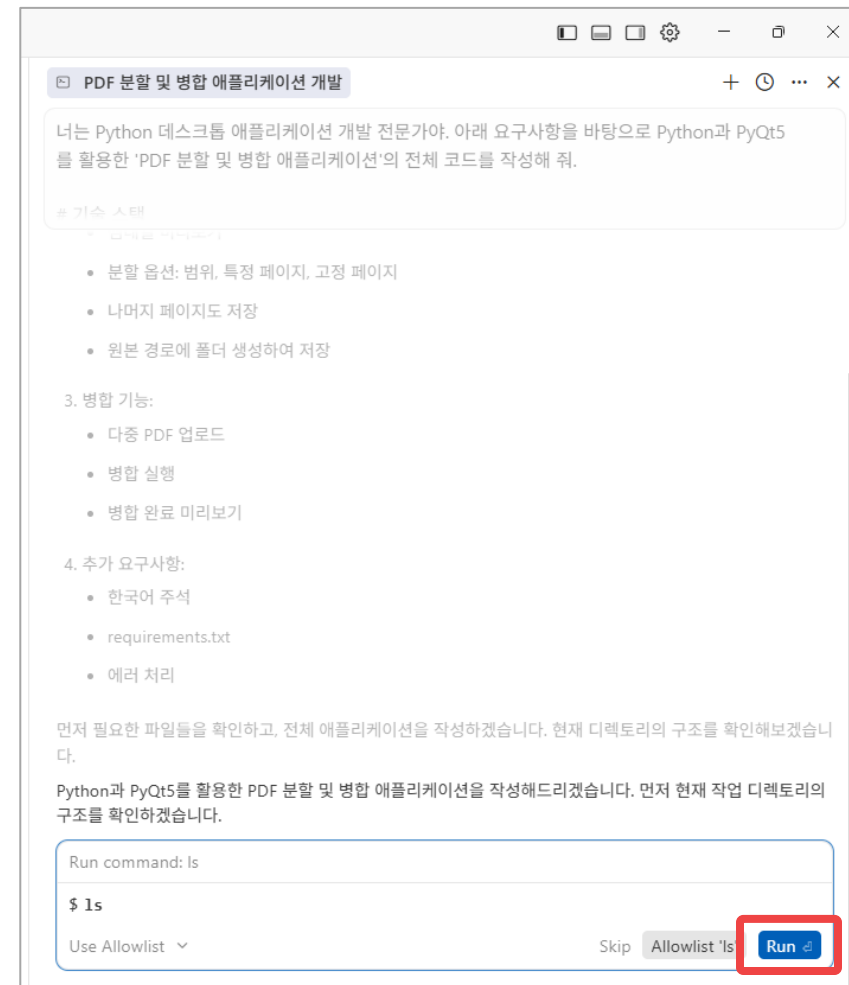
- File Edit Selection View Go Run Terminal Help
- PDF2
- PDF2 requirements.txt
- New Agent Ctrl + Shift + L
- Show Terminal Ctrl + J
- Hide Files Ctrl + B
- Search Files Ctrl + P
- Open Browser Ctrl + Shift + B
- Maximize Chat Ctrl + Alt + E
- Past Chats
- New Chat
- Cursor Tab

앱 개발 실습1. PDF 분할/병합

④ New Chat 클릭 : 모델 선택 → 실행 클릭



⑤ 진행사항 승인 체크 : Run 클릭



앱 개발 실습1. PDF 분할/병합

⑥ 앱테스트 : 터미널 창 열기 → python pdf_tool_app.py 입력 → Enter

PDF 분할 및 병합 애플리케이션 개발

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- 파일명: 원제목_{페이지범위}.pdf 형식

병합 (Merge) 기능

- 다중 PDF 파일 선택
- 파일 순서 조정 (위/아래 이동, 제거)
- 병합 완료 후 썸네일 미리보기 (최대 5페이지)
- 사용자 지정 경로에 저장

에러 처리

- 잘못된 페이지 범위/번호 입력 검증
- PDF 로드 실패 처리
- 파일 저장 권한 오류 처리
- 사용자 친화적인 오류 메시지

실행 방법

패키지가 이미 설치되어 있으므로 바로 실행 가능

```
python pdf_tool_app.py
```

애플리케이션은 탭 방식으로 구성되어 있어 분할과 병합 기능을 쉽게 전환할 수 있습니다. 모든 주요 함수에 한국어 주석이 포함되어 있으며, PyMuPDF(fitz)를 사용하여 가볍고 빠른 썸네일 렌더링을 지원합니다.

PDF2

File Edit Selection View Go Run Terminal Help

PDF2

pdf_tool_app.py
README.md
requirements.txt

New Agent Ctrl + Shift + L
Hide Terminal Ctrl + J
Hide Files Ctrl + B
Search Files Ctrl + P
Open Browser Ctrl + Shift + B
Maximize Chat Ctrl + Alt + E

PDF 분할 및 병합 애플리케이션 개발

너는 Python 데스크톱 애플리케이션 개발 전문가야. 아래 요구사항을 바탕으로 Python과 PyQt5를 활용한 'PDF 분할 및 병합 애플리케이션'의 전체 코드를 작성해 줘.

기술 스택

- 파일명: 원제목_{페이지범위}.pdf 형식

병합 (Merge) 기능

- 다중 PDF 파일 선택
- 파일 순서 조정 (위/아래 이동, 제거)
- 병합 완료 후 썸네일 미리보기 (최대 5페이지)
- 사용자 지정 경로에 저장

에러 처리

- 잘못된 페이지 범위/번호 입력 검증
- PDF 로드 실패 처리
- 파일 저장 권한 오류 처리
- 사용자 친화적인 오류 메시지

실행 방법

패키지가 이미 설치되어 있으므로 바로 실행 가능

```
python pdf_tool_app.py
```

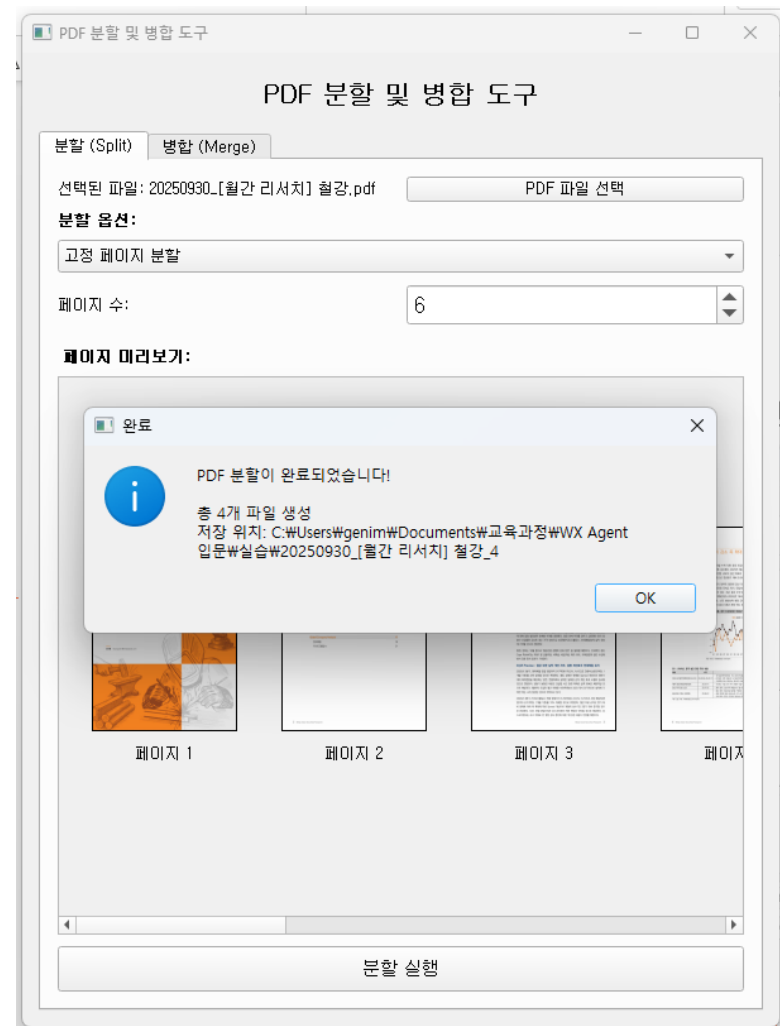
애플리케이션은 탭 방식으로 구성되어 있어 분할과 병합 기능을 쉽게 전환할 수 있습니다. 모든 주요 함수에 한국어 주석이 포함되어 있으며, PyMuPDF(fitz)를 사용하여 가볍고 빠른 썸네일 렌더링을 지원합니다.

Problems Output Debug Console **Terminal** Ports

PS C:\Users\genim\Desktop\WX Agent 입문\실습\PDF2> python pdf_tool_app.py

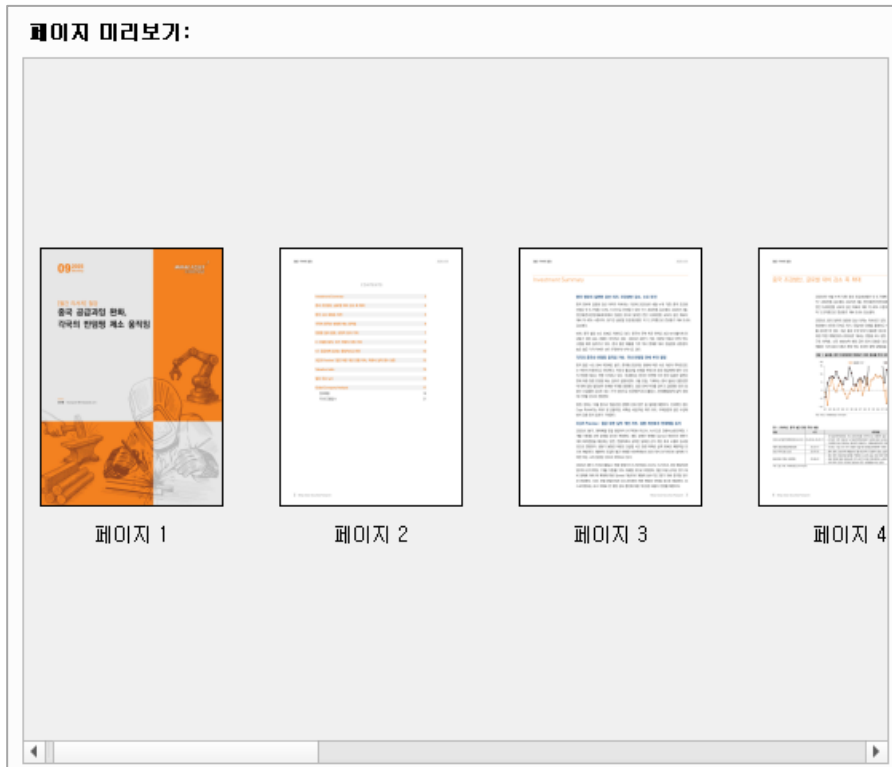
앱 개발 실습1. PDF 분할/병합

⑥ 앱테스트 : '20250930_[월간 리서치] 철강'파일 선택 → 분할 옵션 선택 → 분할 실행 클릭

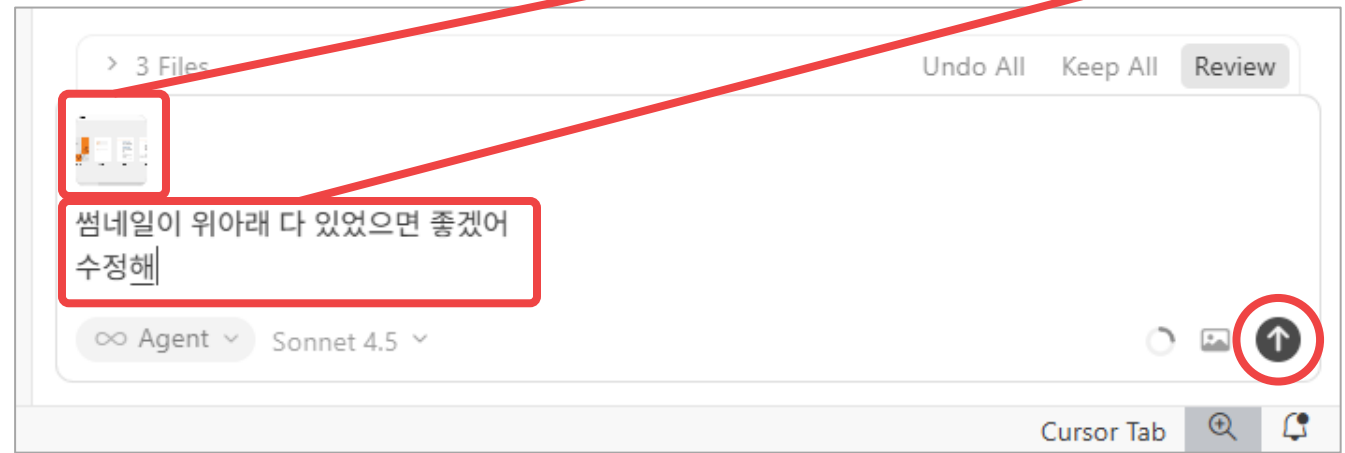


앱 개발 실습1. PDF 분할/병합

⑦ 오류 보완 : 오류 발생한 화면 또는 수정하고 싶은 화면 캡처(window + shift + s) → 앱 종료



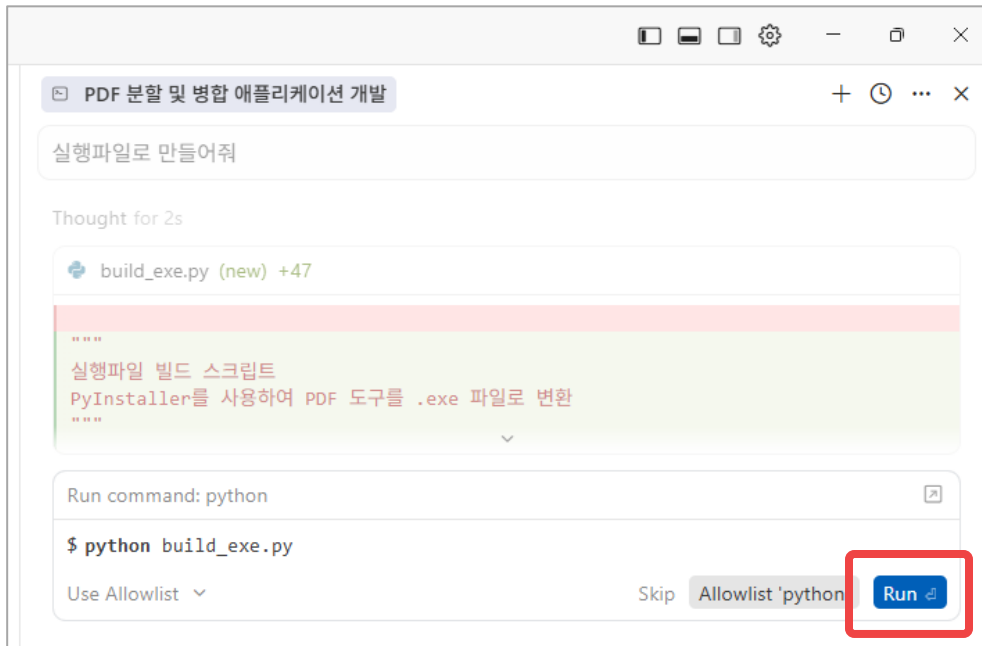
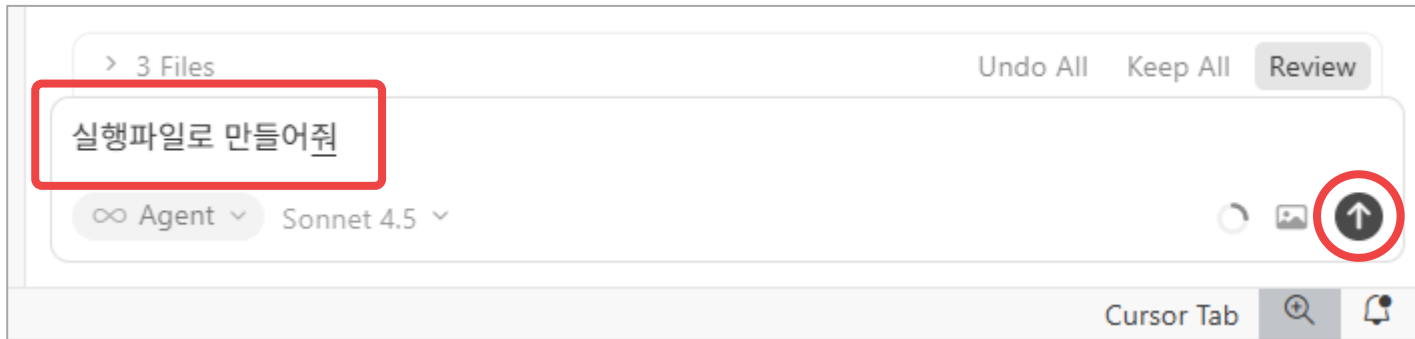
→ cursor AI 대화창에 캡처 이미지 붙여 넣고 보완 사항 입력



※ 수정 완료 후 앱 다시 실행(⑥앱테스트 슬라이드 참고)

앱 개발 실습1. PDF 분할/병합

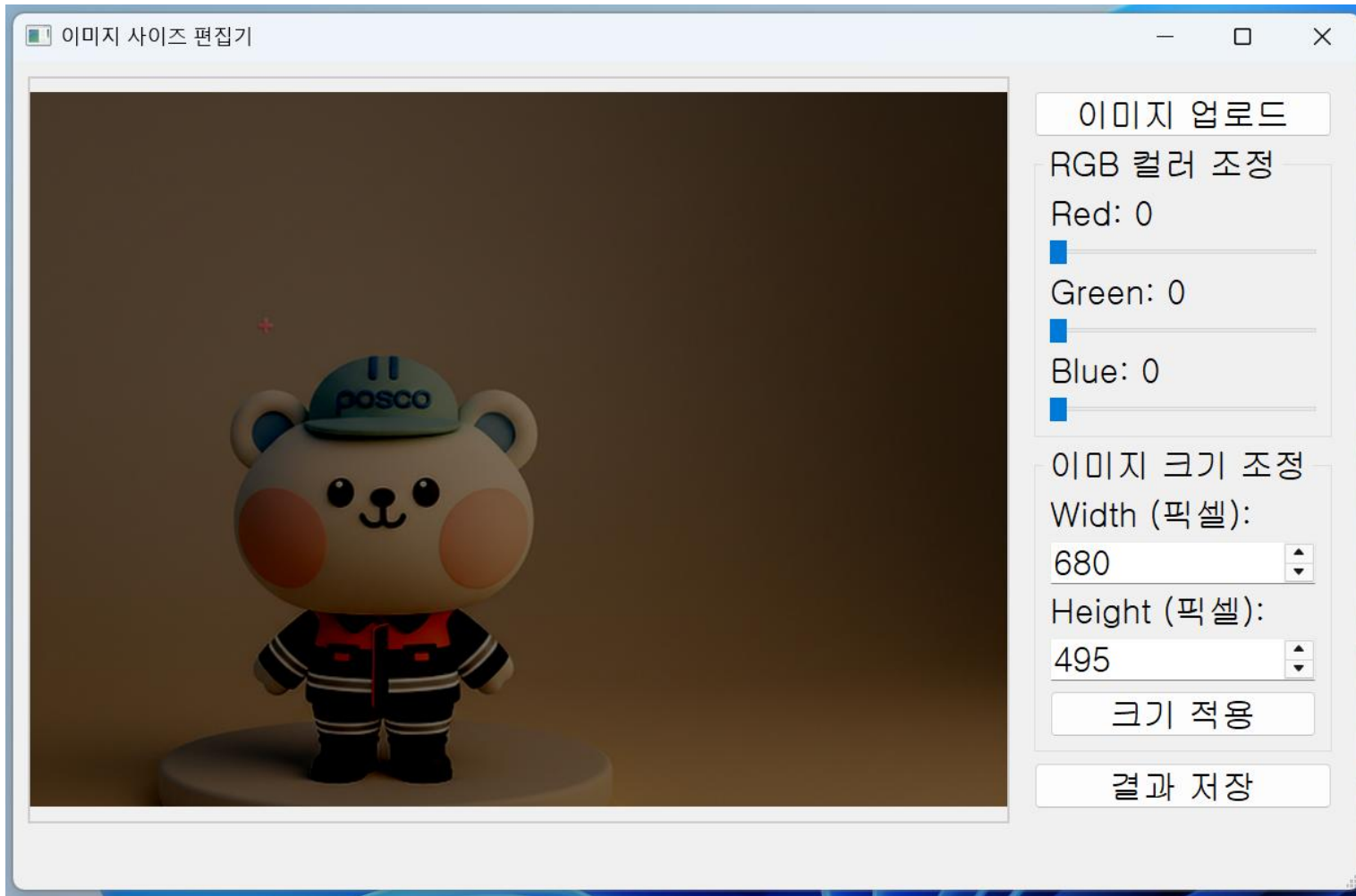
⑧ 실행파일 생성 : cursor AI 대화창에 '실행파일로 만들어줘' 입력 후 실행 → Run 클릭



⑨ dist 폴더에 실행파일(*.exe) 확인

앱 개발 실습2. 이미지 사이즈 및 컬러 조정하기

▶ 실습목표: 이미지 사이즈 및 컬러 조정하기



1. 기능 : 이미지업로드, 색상 조정, 사이즈 조정
2. 메뉴 : 이미지 업로드, RGB Bar, 저장
3. 데이터베이스 : 필요없음
4. 개발환경 : PyQt5

앱 개발 실습2. 이미지 사이즈 및 컬러 조정하기

▶ 프롬프트 초안



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
이미지 사이즈 및 컬러 조정하는 앱을 만들거야

1. 개발환경

- PyQt5 활용

2. 노드설명

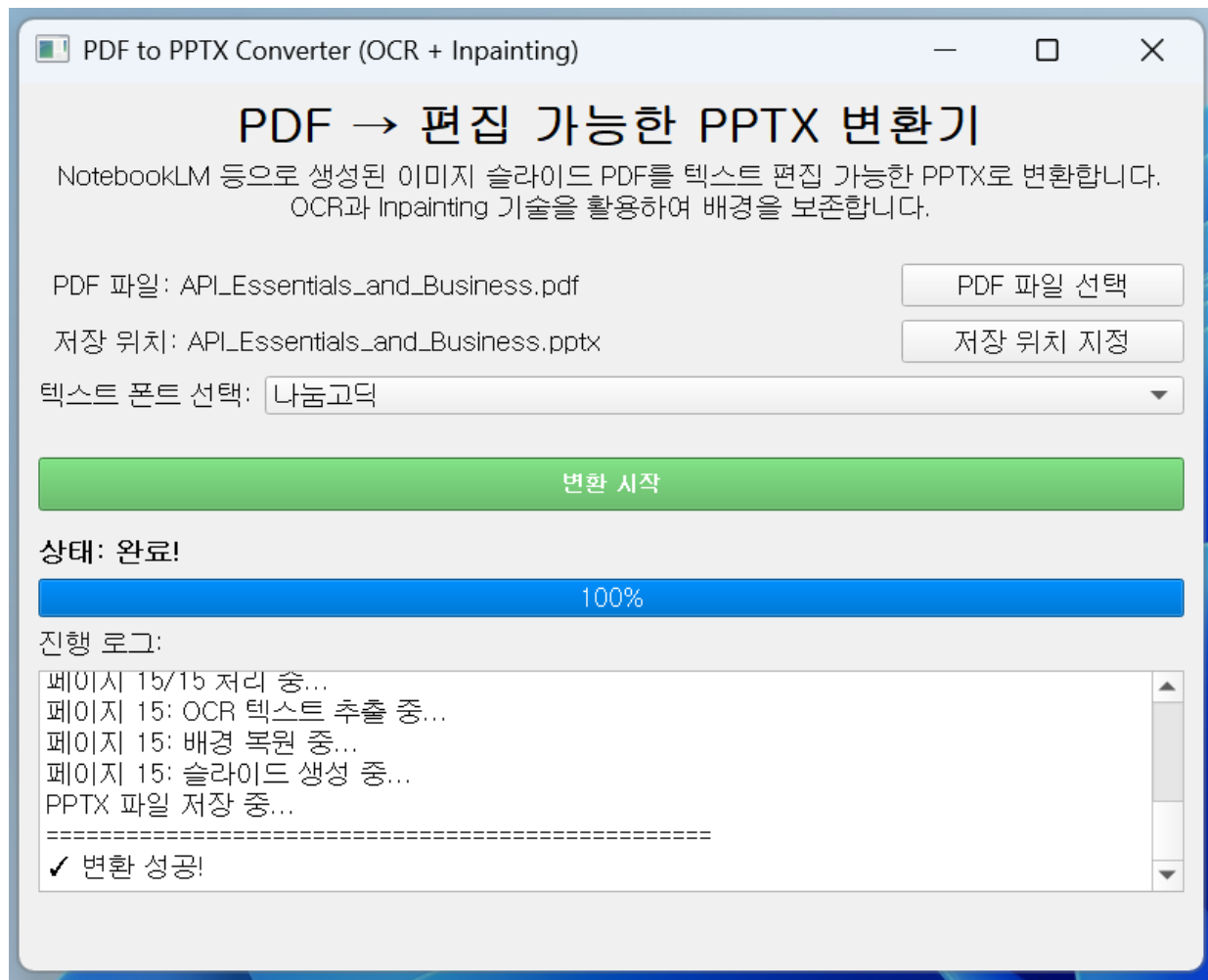
- 이미지 업로드 (다중 업로드 지원)
- 하나의 이미지를 수정하면 동일한 기준으로 다른 이미지에도 일괄 적용
- 컬러 조정: RGB 바를 이용해 실시간으로 이미지 컬러 조정
- 크기 조정: x사이즈 입력 시 비율에 맞춰 y사이즈 자동 조정
- 흑백 변환 기능
- 결과 저장: 수정된 결과를 jpg, png, gif 등으로 저장

3. UI설명

- 다중 이미지 업로드 UI
- RGB 슬라이더 바 (실시간 컬러 조정)
- x 픽셀 입력창 / y사이즈 자동 계산 표시
- 흑백 변환 버튼- 저장 형식 선택 (jpg, png, gif 등)

앱 개발 실습3. 노트북lm 슬라이드 PPTX로 변환

▶ 실습목표: 노트북lm 슬라이드 PPTX로 변환



1. 기능 : PDF 파일 등록, PPTX 파일 저장
2. 메뉴 : 파일 업로드, 변환버튼, 진행률, 저장
3. 데이터베이스 : 없음
4. 개발환경 : PyQt5, EasyOCR

앱 개발 실습3. 노트북lm 슬라이드 PPTX로 변환

▶ 프롬프트 초안



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.

구글 노트북 LM으로 작성한 이미지 슬라이드를 편집가능한 PPTX로 변환하는 앱을 만들어줘

1. 개발환경

- 파이썬 PyQt5 패키지를 이용
- 나에게 없는 패키지는 설치- OCR 패키지: EasyOCR
- 원본 16:9, 출력 PPTX도 16:9
- PPT_Width = 10 inches, PPT_Height = 5.625 inches (16:9 기준)
- 좌표 정밀도: 이미지 해상도(px)와 PPT 물리적 크기(inch) 비율 계산(Ratio) 적용

2. 노드설명

- PDF 파일 읽기
- 저장 파일명 및 경로 지정
- PPTX 파일명은 업로드한 PDF 파일명과 동일하게 자동 설정
- 변환 실행 시 슬라이드를 하나씩 읽어서 OCR로 텍스트 추출 (한글, 영어 등)
- 기존 텍스트가 있던 위치의 이미지 글자를 배경과 동일하게 처리 (텍스트 삭제)
- 추출한 텍스트를 원래 위치 및 크기 그대로 삽입
- 이미지 등 다른 레이아웃은 변경하지 않음

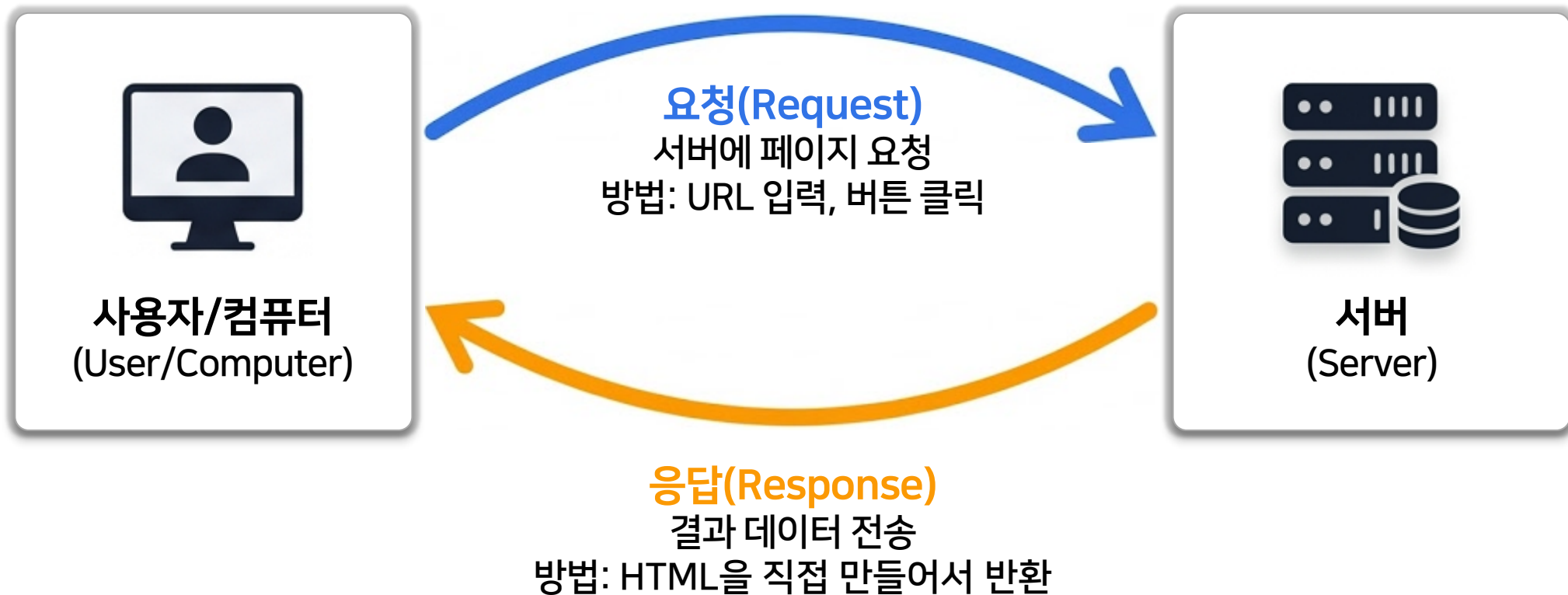
3. UI설명

- PDF 파일 불러오기 메뉴
- 저장 경로 및 파일명 지정 메뉴
- 기본 폰트 선택: 나눔고딕, 맑은 고딕, 나눔스퀘어 Bold
- 변환 실행 버튼

M2. 바이트코딩 활용 앱/웹 개발

5. 바이트코딩 기반 웹 개발

웹 통신방식: 요청과 응답



웹은 일방적인 방송이 아닌, 사용자의 행동(클릭)에 따라 서버가 반응하는 양방향 대화

웹의 구조

프론트엔드



클라이언트(Client / Browser)

사용자의 눈에 보이고
직접 상호작용하는 영역

UI & Interaction

백엔드



서버(Server / System)

눈에 보이지 않는 곳에서
데이터를 처리하고 저장하는 영역

Logic & Data

프론트엔드 구성 3대 요소

프론트엔드: 사용자가 웹사이트나 애플리케이션에서 직접 보고 상호작용하는 앞단 화면(GUI)

HTML

뼈대
(Skeleton)



구조와 배치
'여기는 제목, 여기는 버튼'

CSS

살과 옷
(Skin & Clothing)



디자인과 스타일
'버튼은 파란색, 모서리는 둥글게'

JavaScript

근육과 신경
(Muscle & Nerves)



동작과 생명력
'버튼을 누르면 팝업창을 띄워라'

프론트엔드 구성 3대 요소

1

HTML

구조만



HTML = 뼈대



<HTML>

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>우리 여행 블로그</title>
</head>
<body>
  <h1>우리 여행 블로그</h1>
  <p>여행의 순간을 기록하고 공유하는 공간입니다.</p>
  
  <br>
  <a href="/travel">여행지 더 보기</a>
</body>
</html>
```

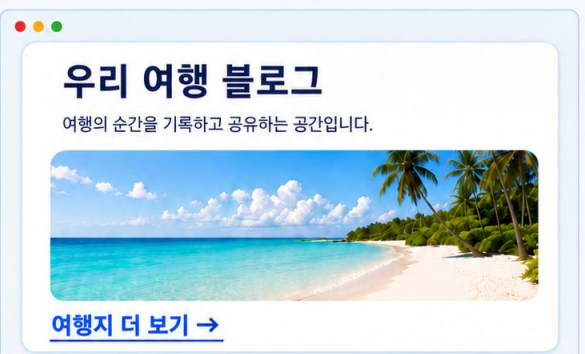
2

HTML + CSS

구조 + 디자인



CSS = 꾸밈



```
<script>
  const menuBtn = document.getElementById("menuBtn");
  const menu = document.getElementById("menu");

  menuBtn.addEventListener("click", () => {
    menu.style.display = "block";
  });

  const moreBtn = document.getElementById("moreBtn");
  const popup = document.getElementById("popup");

  moreBtn.addEventListener("click", () => {
    popup.style.display = "block";
  });
</script>
```

<JavaScript>

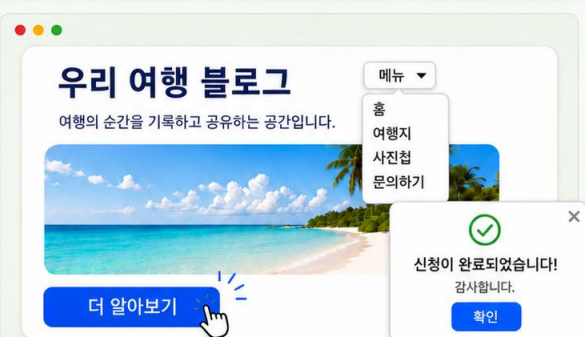
3

HTML + CSS + JavaScript

구조 + 디자인 + 동작



JavaScript = 동작



```
<style>
  body {
    font-family: sans-serif;
    padding: 40px;
    background: #f4f8ff;
  }
  .card {
    background: white;
    padding: 30px;
    border-radius: 20px;
    width: 700px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
  h1 {
    color: #0d2a7a;
    font-size: 48px;
  }
  p {
    color: #444;
    margin-bottom: 20px;
  }
  img {
    width: 100%;
    border-radius: 20px;
  }
  a {
    display: inline-block;
    margin-top: 20px;
    color: #0066ff;
    font-size: 28px;
    font-weight: bold;
    text-decoration: none;
  }
</style>
```

<CSS>

개발자 도구(DevTools)

- 웹페이지 내부 구조 확인 가능함
- HTML, CSS, JavaScript 분석 가능함
- 서버 통신 확인 가능함
- 웹 서비스 문제 분석 시 필수 도구임
- 실행방법: F12 클릭 / Ctrl + Shift + I / 마우스 우클릭 → 검사 클릭

구분	설명	실습
Elements	▪ HTML 구조 확인 ▪ CSS 스타일 확인 ▪ 화면 요소 수정	▪ 요소 선택 버튼 클릭 → 네이버 검색창 선택 → HTML 확인 (태그 종류 확인, id 확인, class 확인, style 확인)
Console	▪ JavaScript 실행 ▪ 오류 확인 가능 ▪ 변수 확인	▪ <code>document.querySelector("input").value</code> → 검색창 입력값 확인
Network	▪ 서버 통신 확인 ▪ API 호출 확인 ▪ 이미지 로딩 확인	▪ Network 탭 선택 → F5 새로고침 → 요청 목록 확인
Application	▪ 쿠키 등 확인	▪ 로그인 정보 확인, 저장 데이터 확인

HTML(HyperText Markup Language)

'무슨 내용을(WHAT)' 보여줄 것인가? 를 정의

- 웹 콘텐츠의 구조와 의미를 문단, 제목, 표, 삽입 이미지, 동영상 등의 태그로 정의하고 부여
- HTML Tag는 제목, 단락, 목록 등과 같이 웹문서 본문의 구조적 의미를 정의함
- Markup Language : 태그, 구분자 등을 이용해 문서나 데이터의 구조를 표기하는 언어



HTML Tag(태그)

<태그> 데이터 </태그>

- **웹페이지의 뼈대**가 되는 **HTML**에서는 모든 내용을 태그(Tag)로 작성하고 구조화
 - 웹 콘텐츠의 구조와 의미를 문단, 제목, 표, 이미지, 동영상 등에 해당하는 태그로 정의하고 부여
 - 태그는 사전에 약속된 암호와 같음. 태그를 사용하여 데이터를 나열하면 웹 브라우저가 해석 후 시각화
- 태그는 시작과 끝이 있음 : 시작 <태그명>, 끝 </태그명>
 - <html> 내용 </html>, <head> 내용 </head>, <p> ~ </p>

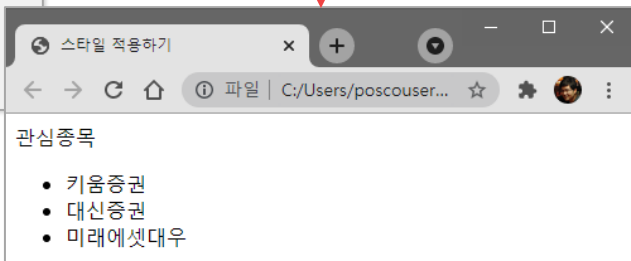
관심종목

키움증권

대신증권

미래에셋대우

렌더링



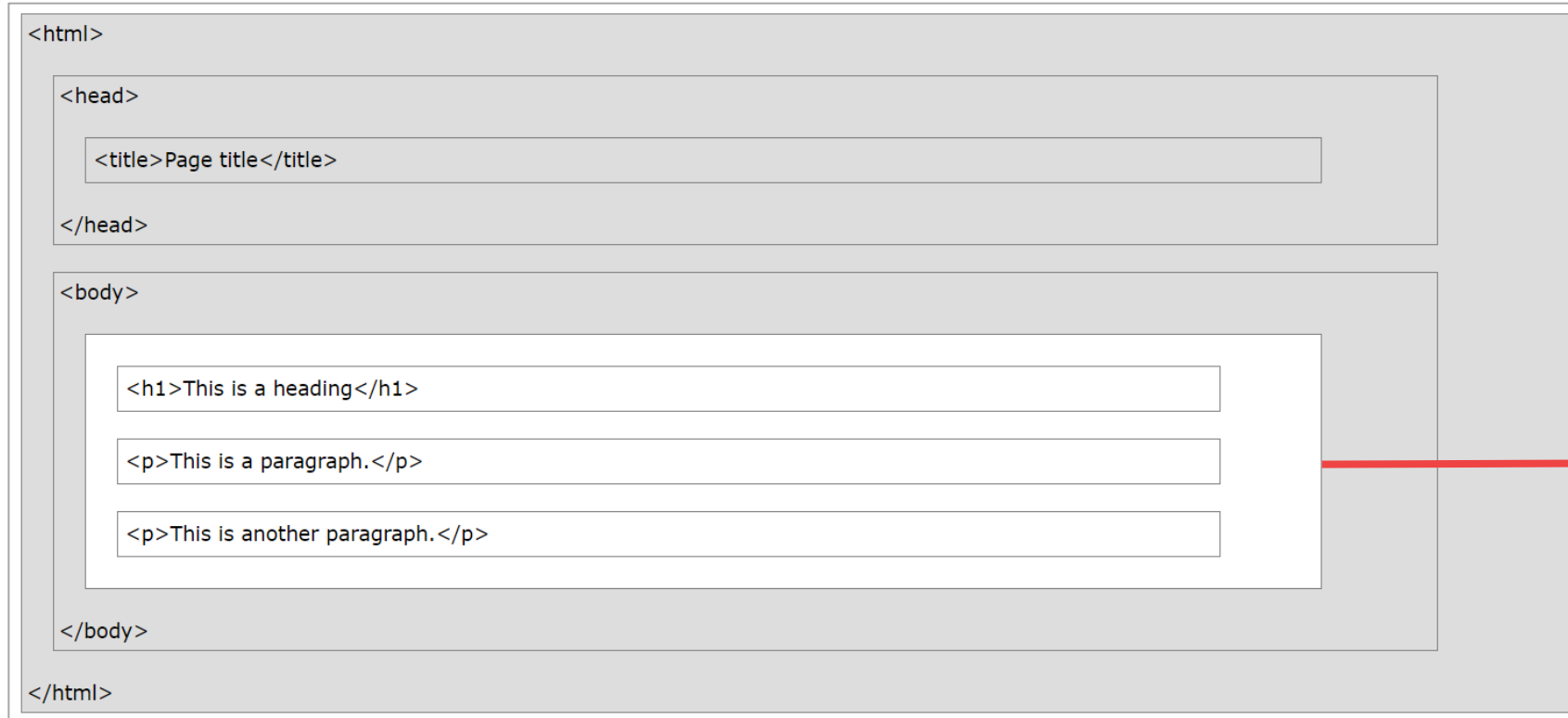
렌더링

```
<p> == $0
<b>하이퍼텍스트 마크업 언어</b>
"("
  <span lang="en">
    "HyperText Markup Language, "
    <b>HTML</b>
  </span>
  ", "
  <span style="font-size: smaller;">
    <a href="/wiki/%EB%A7%B8%ED%99%94%EC%96%B4" title="문화어">문화어</a>
    ";"
  </span>
  " 초본문표식달기언어, 하이퍼본문표식달기언어)는 "
  <a href="/wiki/%EC%98%B9_%ED%8E%98%EC%9D%B4%EC%A7%80" title="웹 페이지">웹 페이지</a>
  "를 위한 지배적인 "
  <a href="/wiki/%EB%A7%B8%ED%81%AC%EC%97%85_%EC%96%B8%EC%96%B4" title="마크업 언어">마크업 언어</a>
  "다. 또한, HTML은 제목, 단락, 목록 등과 같은 본문을 위한 구조적 의미를 나타내는 것뿐만 아니라 링크, 인용과 그 밖의 항목으로
  <a href="/wiki/%EA%B5%AC%EC%A1%B0%EC%A0%81_%EB%A7%B8%EC%84%9C" title="구조적 문서">구조적 문서</a>
  "를 만들 수 있는 방법을 제공한다."
```

하이퍼텍스트 마크업 언어(HyperText Markup Language, **HTML**, **문화어**: 초본문표식달기언어, 하이퍼본문표식달기언어)는 **웹 페이지**를 위한 지배적인 **마크업 언어**다. 또한, HTML은 제목, 단락, 목록 등과 같은 본문을 위한 구조적 의미를 나타내는 것뿐만 아니라 링크, 인용과 그 밖의 항목으로 **구조적 문서**를 만들 수 있는 방법을 제공한다.

HTML Tag의 기본 구조

- HTML 페이지는 <html> 태그 하위에 크게 <head>와 <body> 태그로 구분 됨
- <head> ~ </head> → 본문에 보여지지 않음 (메타데이터 등 문서의 속성을 정의)
- <body> ~ </body> → 본문에 보여지며 문서의 실제 내용이 하위 태그들로 작성되어 있음

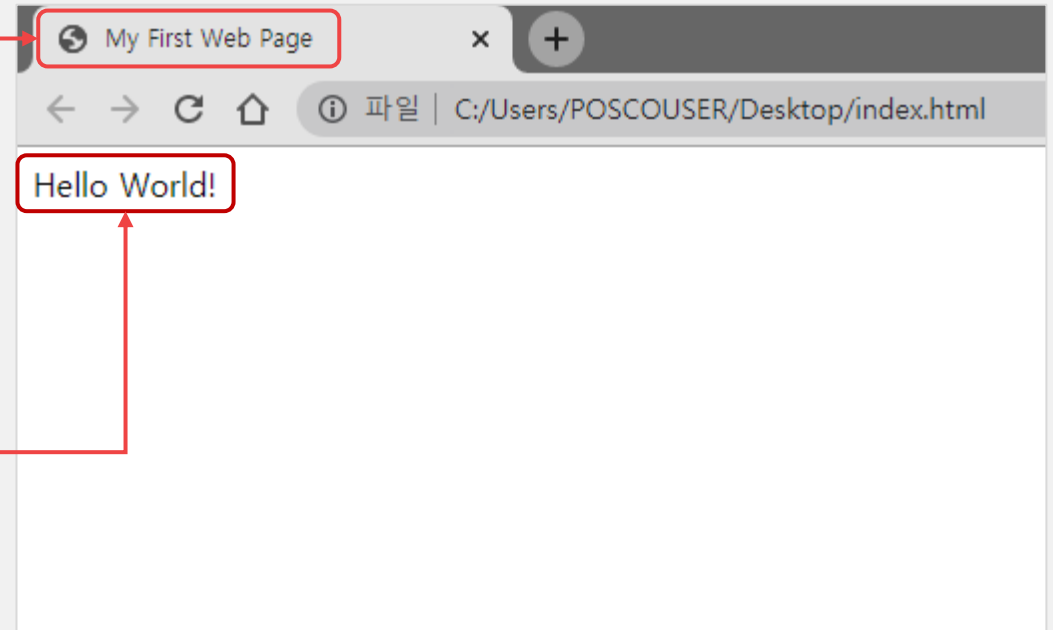


<body> 섹션(흰색 영역) 내의
내용만 웹 브라우저 본문에 표시

HTML Tag의 기본 구조

<!-- 기본 HTML 구조 이해하기 -->

```
<html>  
<head>  
  <title>My First Web Page</title>  
</head>  
  
<body>  
  Hello World!  
</body>  
</html>
```



#<head></head> 부분은 본문에 나타나지 않으며, <body></body> 부분만 본문 화면에 나타남

네이버 HTML 확인

Naver 접속 → F12 클릭

The screenshot shows the Naver homepage. At the top, there's a search bar with the text "검색어를 입력해 주세요." and a magnifying glass icon. Below the search bar are several navigation icons: 메일 (Email), 카페 (Cafe), 블로그 (Blog), 스토어 (Store), 뉴스 (News), 증권 (Securities), 부동산 (Real Estate), 지도 (Map), 웹툰 (Webtoon), and 라운지 (Lounge). The main content area includes a large advertisement for Japanese language learning, a section for news and media (뉴스스탠드), a section for shopping (쇼핑), and a section for featured products (주목할 만한 상품은?).

The screenshot shows the Chrome DevTools Elements panel. The HTML structure is displayed, starting with the DOCTYPE declaration and the html element. The body element contains several divs, including one with the id "u_skip" and another with the id "wrap". The "wrap" div contains a script element and an iframe element. The script element has the type "text/plain" and the id "ad-timeboard-response". The iframe element has the id "rfs-iframe" and the src attribute set to "https://search.naver.com/remote_frame".

CSS(Cascading Style Sheet)

내용을 '어떻게(HOW)' 보여줄 것인가? 를 정의

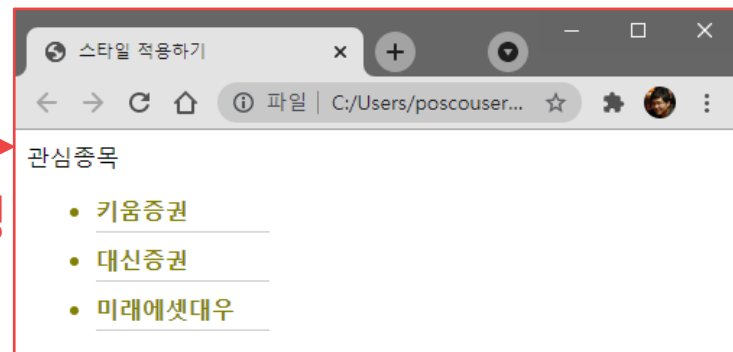
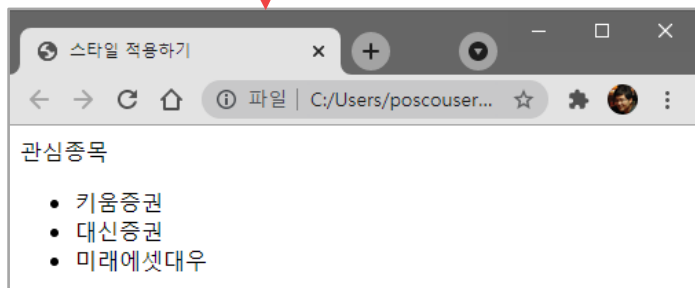
- HTML 등의 마크업 언어로 작성된 문서가 웹사이트에 표현되는 '서식'을 정해주는 언어
- HTML 문서 內 <style>태그 안에 HTML 각 요소의 배경색, 폰트 등의 내용을 정의하는 형태로 작성

```
관심종목<br>
<ul>
  <li>키움증권</li>
  <li>대신증권</li>
  <li>미래에셋대우</li>
</ul>
```



```
<style>
li {
  width: 120px;
  margin: 8px;
  padding-bottom: 4px;
  font-weight: bold;
  color: olive;
  border-bottom: 1px solid #ccc;
}
</style>
```

렌더링



네이버에 CSS를 제거하면...

NAVER

검색

검색어를 입력해 주세요.

Q 검색

입력도구 자동완성/최근검색어펼치기

- 메일
- 카페
- 블로그
- 스토어
- 뉴스
- 증권
- 부동산
- 지도
- 웹툰
-  치지직추천
- 바로가기 펼침

확장 영역

페이 바로가기

알림

장바구니알림5

닥터피펫 퓨어팟



아직도 변기청소 하는 분들 주목! 변기 청소 1년 동안 안 해요

타임 특가

- 뉴스탠드언론사편집
- 엔터
- 쇼핑투데이
- 스포츠LIVE

- [쇼핑투데이](#)
- [스포츠LIVE](#)
- [경제](#)

분야별 뉴스

국민일보

속보

김정은 “우리 다치게 하면 韓완전봉괴 배제 못해”

머니투데이

속보

北, 어제 김일성광장서 9차 당대회 기념 열병식 개최

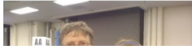
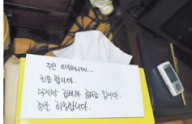
뉴스홈



-  [머니투데이](#) 구독

[르포]"이게 진짜 아이폰" 갤럭시S26, 새 기능 발표하자...환호·탄성 쏟아졌다

02월 26일 07:01

-  [서울경제](#) 구독

“마지막 집세입니다, 죄송합니다”...세 모녀는 70만원 남기고 그렇게 떠났다 [오늘의 그날]

02월 26일 07:30



이전 페이지

언론사 더보기1페이지전체/21

다음 페이지

쇼핑 / 맨즈 / 브랜드샵 / 추천한딜 / MY추천 / 쇼핑라이브

1/14 < >

새로고침하면...

☰

pay

🔍

검색어를 입력해 주세요.

🔍

📧

☕

📄

📺

📰

📈

🏠

📍

📖

🔥

⋮

메일

카페

블로그

스토어

뉴스

증권

부동산

지도

웹툰

라운지

NUZAM

비싸게 매트리스 갈지 말고
 탄탄한 토퍼 장만하세요!

네이버를 더 안전하고 편리하게 이용하세요

NAVER 로그인

아이디 찾기 | 비밀번호 찾기 | 회원가입

뉴스스탠드 · 언론사편집 / 엔터 / 스포츠 LIVE / 경제 / 쇼핑투데이

전체언론사 ▾ | 연합뉴스 · 속보 김정은 "한국 현 정권 유화적 태도, 기만극이자 졸작" | 뉴스스탠드 | 뉴스홈

경향신문	MBC	KBS NEWS	이데일리	매일경제	ZDNET Korea
뉴시스	동양일보	스포츠조선	에럴드경제	프레시안	사사IN
디지털타임스	OSEN	한국경제TV	엑스포뉴스	DAILY NK	스포츠동아
에너지경제	TBC	뉴스핍	독서신문	Byline Network	EBS

<

언론사 더보기 1/4

>

☰ ☐

쇼핑 / 맨즈 / 브랜드샵 / 추천핫딜 / MY추천 / 쇼핑라이브

1/2 < >

쿠팡 · G마켓 · 옥션

세프매찬 >

LF스퀘어를 >

3·1절 그 후, 아직도 침묵 속에

독립운동가 후손들에게 도움이 필요합니다.

대한적십자사

더 알아보기 >

나의 첫 번째 쇼핑자능, 쇼핑에이전트
 NCS 스토어

맘카페 추천 제품 비교할게요

주목할 만한 상품은?

실크처럼 부드럽게~ 기장... | 오래도록 좋은 코디 친구! ... | 55~88, 사이즈 선택! 세련된...

AD

JavaScript(JS)란?

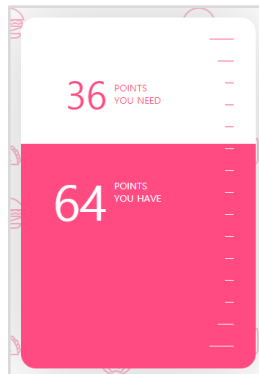
HTML 문서 안에 삽입되어 실행되는, 프로그래밍 가능한 인터프리트(Interpret) 언어

- 아주 먼 과거(20C)에는 JS없는 웹페이지도 많았으나, 이제 JS없이 웹페이지를 만든다는 것은 상상도 못할 일
- 따라서 모든 웹브라우저가 필수적으로 JS 코드 해석기(Interpreter)를 내장하여 JS 실행을 지원하고 있음



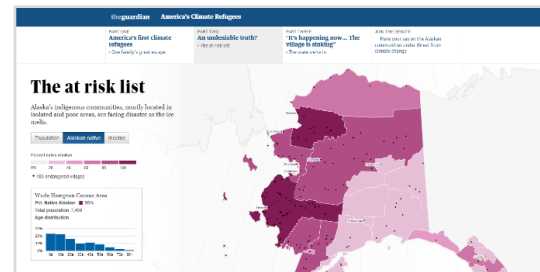
JS로 할 수 있는 것들

애니메이션*(anime.js)

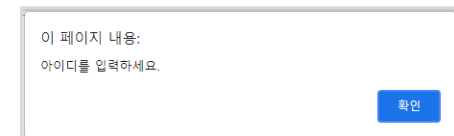


<https://codepen.io/lmgonzalves/pen/NyzKdr>

그래프 그리기(데이터 시각화)



경고창 출력



웹게임



<https://santatracker.google.com/intl/ko/>

*애니메이션은 CSS에서 지원하는 기능이나, 사용의 편리함 때문에 JS 라이브러리를 활용하는 것이 일반적

백엔드: 주방과 창고 비유

서버 (Server Application)



복잡한 연산을 처리하고
주문을 받는 주방

Python Node.js Java

데이터 요청 및 저장

데이터베이스 (DB: Data Base)



회원 정보와 게시물을
영구적으로 보관하는 창고

MySQL PostgreSQL

API(Application Programming Interface)

프로그램이 다른 프로그램의 기능이나 데이터를 사용할 수 있게 해 주는 연결 통로

1. 사용자가 음식점 목록을 봄
2. 앱이 서버에 음식점 목록 요청함
3. 서버가 음식점 데이터를 응답함
4. 사용자가 주문 버튼을 누름
5. 앱이 주문 정보를 서버에 전송함
6. 서버가 결제, 주문 접수, 매장 전달을 처리함



1. 앱은 API를 호출함
2. 서버는 데이터를 처리하고 결과를 돌려줌

웹페이지와 API의 차이점

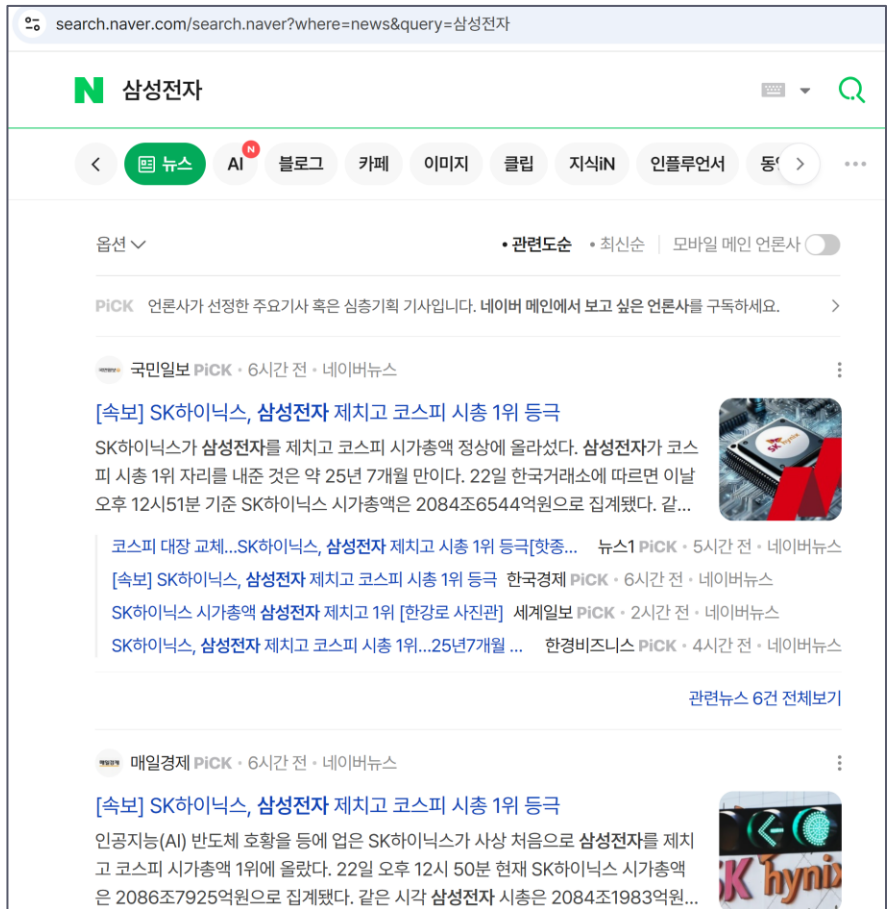
- 웹페이지(앱)는 사람이 보기 위한 결과
- API는 프로그램이 처리하기 위한 결과임

구분	웹페이지	API
대상	▪ 사람	▪ 프로그램
목적	▪ 화면 제공	▪ 데이터제공 또는 처리
응답 형태	▪ HTML, 이미지, 화면	▪ JSON 같은 데이터 형식
사용 방식	▪ 브라우저로 접속	▪ 코드나 도구로 호출
예시	▪ 뉴스 기사 페이지	▪ 뉴스 목록 데이터

웹페이지와 API 비교

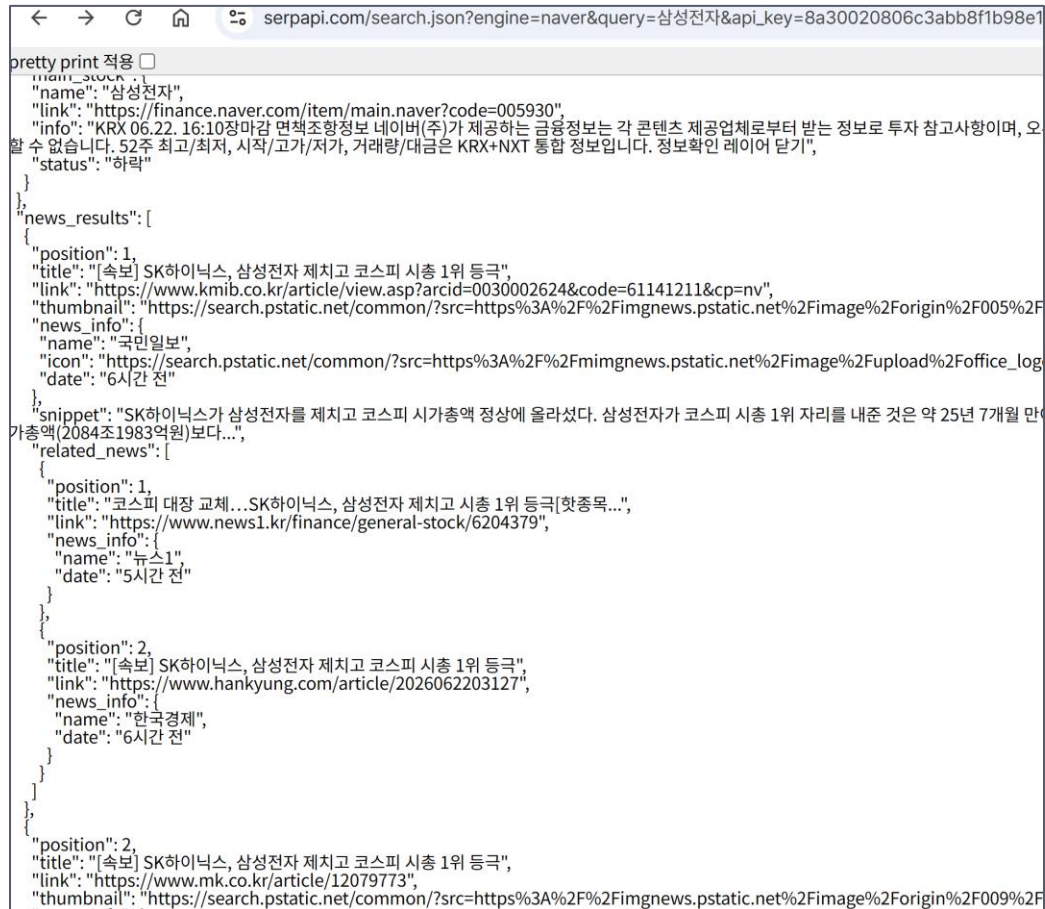
■ 웹페이지 요청

`https://search.naver.com/search.naver?where=news&query=삼성전자`



■ API 요청

`https://serpapi.com/search.json?engine=naver&query=삼성전자&api_key=키`



JSON: 데이터 형식

- 데이터를 저장하고 교환하기 위한 텍스트 형식
- 웹 API, 설정 파일, 로그, 데이터 저장에 많이 씀
- 사용규칙: 키(Key)와 문자열은 쌍따옴표, 값(Value) 사이에는 콤마, true/false/null은 소문자, 주석 지원 안 함

JSON 구조

{ Key : Value }

- 파이썬의 딕셔너리와 유사한 구조
- 직관적이고 가공이 편리한 텍스트 형식
- 화면에 필요한 데이터만 전송

JSON 예시

```
1 {  
2     "id": "user123",  
3     "status": "success",  
4     "message": "로그인 성공"  
5 }
```

* 예시 주소: <https://api.github.com/users/torvalds>

* 비교 주소: <https://github.com/torvalds>

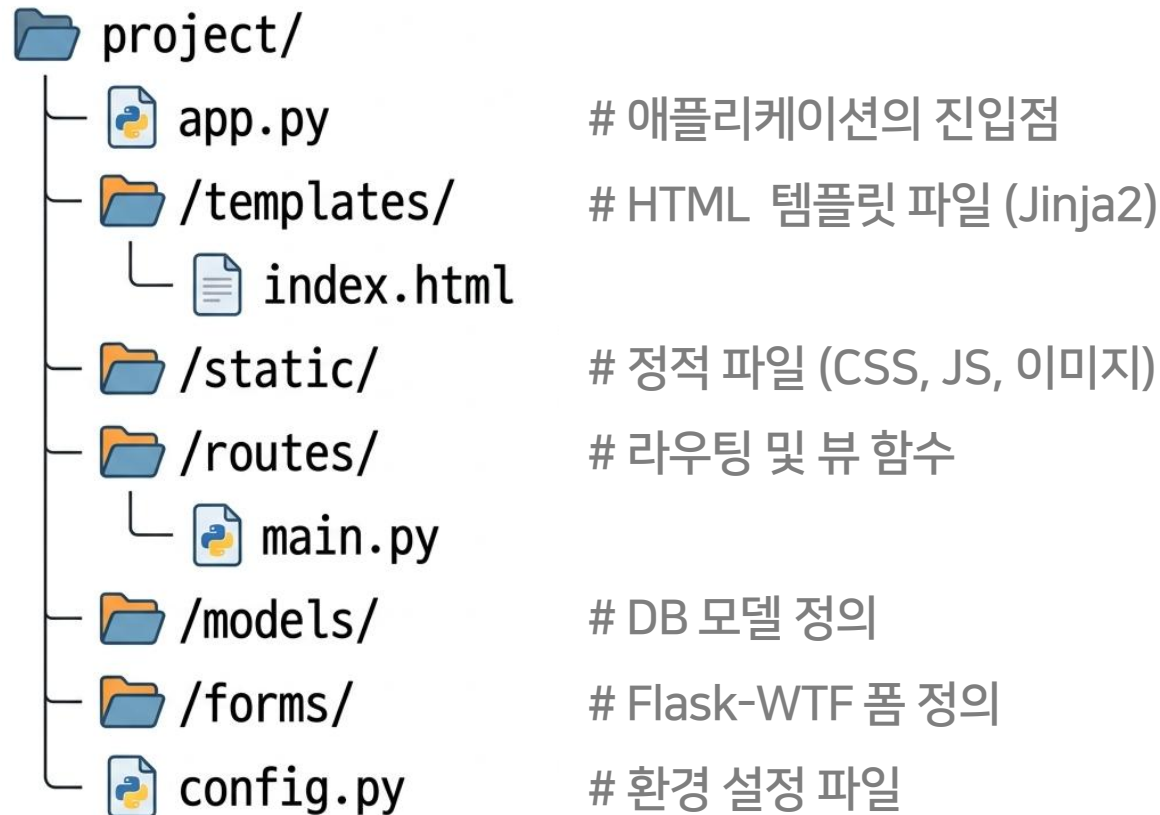
웹 개발을 위한 웹 프레임워크(Flask)

파이썬(Python) 기반 경량 웹 프레임워크(Micro Web Framework)

특징

주요 특징	내용
경량성 & 유연성	기본 기능만 포함되어 가볍고 빠르며, 플러그인 및 확장 패키지로 기능 추가 가능
모듈화된 구조	Blueprint를 사용하여 코드 구조를 모듈화하고 대규모 프로젝트에 유리
템플릿 렌더링	Jinja2 템플릿 엔진을 통해 HTML과 데이터를 쉽게 연결
디버깅 및 테스트	내장 디버거와 유닛 테스트 도구로 개발 편의성 향상
풍부한 확장 기능	Flask-Login, SQLAlchemy, Migrate 등 다양한 확장 모듈 제공

구조



실행방법

- 패키지 설치: 터미널(또는 CMD)에서 `pip install flask`를 입력해 설치
- 파일 저장: 아래 코드를 `app.py`라는 이름으로 저장
- 서버 실행: 터미널에서 `python app.py`를 입력 후 <ENTER> → Running on <http://127.0.0.1:5000> 문구 확인
- 사용: 크롬 브라우저 주소줄에 <http://127.0.0.1:5000> 입력 후 <ENTER>

```
from flask import Flask

app = Flask(__name__)    # 내 프로그램을 Flask 앱으로 등록

@app.route('/')          # 사용자가 홈페이지 주소로 들어오면
def hello():
    return "안녕하세요! 여러분의 첫 번째 웹 서버입니다."

if __name__ == '__main__':
    app.run()            # 서버 실행!
```

웹 개발 실습 절차

① 프롬프트 만들기

- ChatGPT or Gemini에서 Cursor AI에 요청할 프롬프트 만들기

② 실습 폴더 생성

- 실습 폴더 아래 새 폴더(PDF2) 만들기

③ Cursor 실행

- Cursor 실행, 실습 폴더 열기, 파이썬 패키지 설치

④ New Chat 클릭

- 오른쪽 사이드바에서 New Chat 클릭, 프롬프트 입력

⑤ 진행사항 체크

- 모델 선택, 진행 여부(폴더 생성, 패키지 설치 등) 승인(Run) 클릭

Cursor

⑥ 앱 테스트

- 실습파일 업로드 후 앱 테스트, 이상 여부 확인

⑦ 오류 보완

- 오류 발생 화면 캡처, 오류발생 프롬프트 작성, 재실행

Cursor

웹 개발 실습1. PDF 번역

▶ 실습목표: PDF 문서의 텍스트 추출 및 다국어 번역

PDF 번역기

1. PDF 파일 업로드

파일 선택

20250930_월간 리서치] 철강_part_2.pdf

2. 번역 방향 선택

영어 → 한국어

한국어 → 영어

번역 시작

번역 진행률

100%

추출된 원본 텍스트

[월간 리서치] 철강

Mirae Asset Securities Research 3 2025.9.30

Investment Summary

중국 정부의 일관된 감산 의지. 조강생산 감소, 수요 부진

중국 정부의 일관된 감산 의지가 지속되는 가운데 2025년 (1~8월 누계 기준) 중국 조강생

산량은 약 6.7억톤 (-2.8%, YoY) 으로 전년동기 대비 약 1,950 만톤 감소했다. 2025 년 3월,

전인대 (전국인민대표회의) 에서 언급된 것으로 알려진 연간 5,000만톤 규모의 감산 목표치

대비 약 40% 수준이다. 동기간 글로벌 조강생산량은 약 12.3억톤으로 전년동기 대비 0.6%

감소했다.

반면, 중국 철강 수요 회복은 지연되고 있다. 중국의 주택 착공 면적은 2021년 8월이래 전

년동기 대비 감소 흐름이 이어지고 있다. 2025 년 상반기 기준, 미분양 부동산 면적 역시

고점을 재차 높여가고 있다. 중국 철강 제품별 가격 역시 판매류 대비 건설업과 관련성이

높은 철근 가격 약세가 보다 뚜렷하게 나타나고 있다.

각국의 중국산 반덤핑 움직임 지속. 국내 반덤핑 관세 부과 결정

번역된 텍스트

[Monthly Research] Steel

Mirae Asset Securities Research 3 September 2025

Investment Summary

Consistent reduction efforts by the Chinese government. Decrease in steel production, weak demand

Amidst the Chinese government's consistent reduction efforts, as of 2025 (cumulative basis from January to August), China's steel

production volume decreased by approximately 67 million tons (-2.8%, YoY) compared to the previous year, amounting to a reduction of

about 19.5 million tons. In March 2025, the annual reduction target of 50 million tons, known to have been mentioned at the National

People's Congress, accounts for approximately 40% of the target. During the same period, global steel production decreased by 0.6% to

approximately 1.23 billion tons compared to the previous year.

On the other hand, the recovery of steel demand in China is being delayed. The area of housing construction started in China has been

following a decreasing trend compared to the previous year since August 2021. In the first half of 2025, the area of unsold real

estate has also reached a new high. Prices of Chinese steel products, particularly reinforcing bars that are highly related to the

construction industry, are showing a more distinct decline compared to raw materials.

Continued anti-dumping movements against Chinese products. Decision on imposing domestic anti-dumping tariffs

번역 파일 다운로드

1. 기능 : PDF 파일 업로드, 한/영 및 영/한 번역, 번역 진행률 표시, 원본 및 번역 텍스트 대조, 결과물 다운로드
2. 메뉴 : PDF 파일 업로드, 번역 방향 선택, 번역시작, 추출된 원본 텍스트 확인, 번역된 텍스트 확인
3. 데이터베이스 : 필요없음
4. 개발환경 : 파이썬(Python), 웹 프레임워크(Flask), PDF 추출 라이브러리, LLM API(OpenAI)

웹 개발 실습1. PDF 번역

① 프롬프트 만들기 : 초안_기본적인 내용 작성해서 LLM에 요청



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
PDF파일을 업로드하면 번역해주는 앱을 만들고 싶어

1. 개발환경

- Flask 패키지 사용
- pypdf (또는 pypdf2) 사용
- 번역: OpenAI API 사용
- OpenAI API key는 시스템 환경 변수에 저장

2. 노드설명

- PDF 파일 업로드 시 텍스트 추출
- 영어→한국어, 한국어→영어 번역 방향 선택에 따라 OpenAI API 호출
- 번역 진도율 실시간 전달
- 번역 완료 후 업로드한 PDF 파일명과 동일한 이름으로 텍스트 파일 저장
- 추출한 텍스트와 번역 결과 미리보기 반환

3. UI설명

- PDF 업로드 메뉴
- 번역 방향 선택 (영어→한국어 / 한국어→영어)
- 번역 진도율 표시 UI
- 원문 텍스트 미리보기 영역
- 번역 결과 미리보기 영역
- 저장 버튼

웹 개발 실습1. PDF 번역

① 프롬프트 만들기 : 완성_LLM이 만들어준 프롬프트

Prompt

너는 파이썬 및 Flask 웹 개발 전문가야. 아래의 요구사항을 바탕으로 PDF 파일의 텍스트를 추출해 번역하고, 결과를 텍스트 파일로 저장할 수 있는 웹 애플리케이션 코드를 작성해 줘.

기술 스택

- Backend: Python, Flask
- PDF 텍스트 추출: pypdf (또는 PyPDF2)
- 번역 API: OpenAI API (openai 패키지)

UI/UX 및 화면 레이아웃 요구사항 (ChatGPT 스타일의 2단 분할 화면)
화면을 좌/우 두 개의 영역으로 분할하여 데스크톱에서 보기 좋게 구성해.

1. 왼쪽 패널 (컨트롤 및 진행 상태):

- PDF 파일 업로드 폼
- 번역 방향 설정 (영어 -> 한국어, 한국어 -> 영어 선택 라디오 버튼 또는 드롭다운)
- '번역 시작' 버튼
- 번역 진행률(%)을 실시간으로 보여주는 프로그레스 바 또는 텍스트

2. 오른쪽 패널 (미리보기 및 다운로드):

- 'PDF에서 추출한 텍스트'를 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
- '번역된 내용'을 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)
- 번역 완료 시 활성화되는 '다운로드' 버튼

3. 스타일링:

- 배경이 어두운 색이면 글자 색깔을 흰색으로, 배경이 밝은 색이면 글자 색깔을 검은색으로 자동 적용되도록 CSS를 작성해.
- 좌우 패널이 깔끔하게 나뉘도록 Flexbox나 Grid를 활용해 직관적으로 디자인해.

핵심 기능 요구사항

1. PDF 텍스트 추출 및 번역 연동:

- pypdf를 이용해 업로드된 PDF 파일에서 '텍스트'만 추출해.
- OpenAI API를 사용하여 추출된 텍스트를 선택한 언어로 번역해.
- API Key는 절대 하드코딩하지 말고, 시스템 환경 변수에 저장된 'OPENAI_API_KEY'를 'os.environ'으로 불러와서 사용해.

2. 번역 진행률 표시 및 실시간 렌더링 (SSE):

- 텍스트를 페이지 단위나 적절한 청크 단위로 나누어 번역을 진행해.
- 번역이 진행되는 동안 왼쪽 패널의 진도율이 업데이트되고, 동시에 오른쪽 패널의 '번역된 내용' 영역에 텍스트가 실시간으로 추가되도록 SSE(Server-Sent Events)나 AJAX 폴링을 구현해.

3. 텍스트 파일(.txt) 저장 및 다운로드:

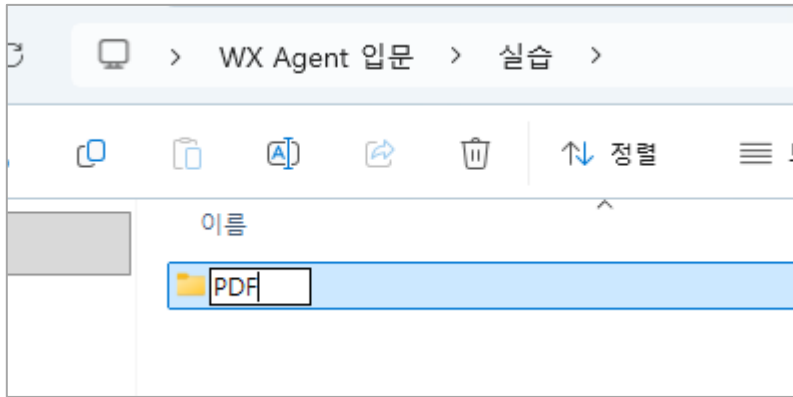
- 번역이 완료되면 오른쪽 패널의 다운로드 버튼을 통해 번역된 텍스트 파일(.txt)을 받을 수 있게 해.
- 다운로드되는 텍스트 파일의 이름은 업로드한 원본 PDF 파일의 이름과 동일하게 설정해 (예: 'report.pdf' 업로드 시 'report.txt'로 다운로드).

출력 지침

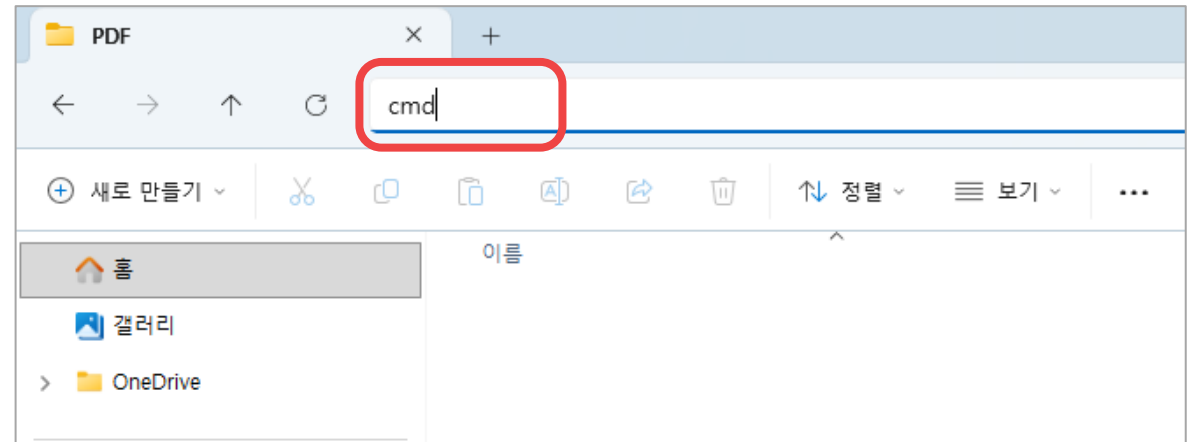
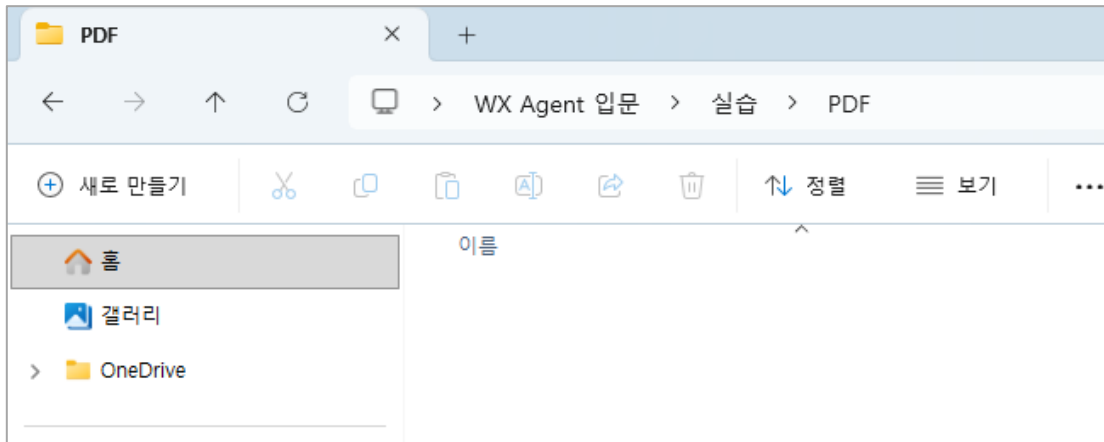
- 'app.py', 'templates/index.html' 코드와 필요한 폴더 구조를 명확히 나누어 작성해 줘.
- 코드를 실행하기 위해 설치해야 하는 'requirements.txt' 내용과 터미널 실행 명령어도 함께 안내해 줘.

웹 개발 실습1. PDF 번역

② 실습 폴더 생성 : 바탕화면 - WX Agent 폴더 - 실습 - 「PDF」 폴더 생성

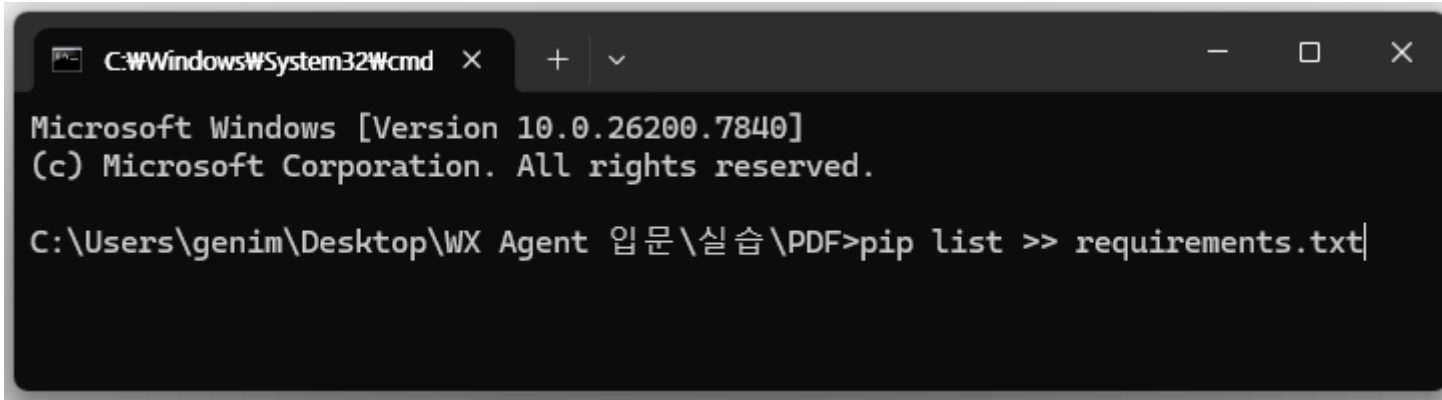


③ 개발 환경 설정 : 「PDF」폴더 이동 → 주소창에 cmd 입력 → Enter



웹 개발 실습1. PDF 번역

③ 개발 환경 설정 : `pip list >> requirements.txt`



```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\genim\Desktop\WX Agent 입문\실습\PDF>pip list >> requirements.txt
```

③ 개발 환경 설정 : Cursor AI 실행 → File-Open Folder-바탕화면-WX Agent 입문-실습-PDF 폴더 클릭



웹 개발 실습1. PDF 번역

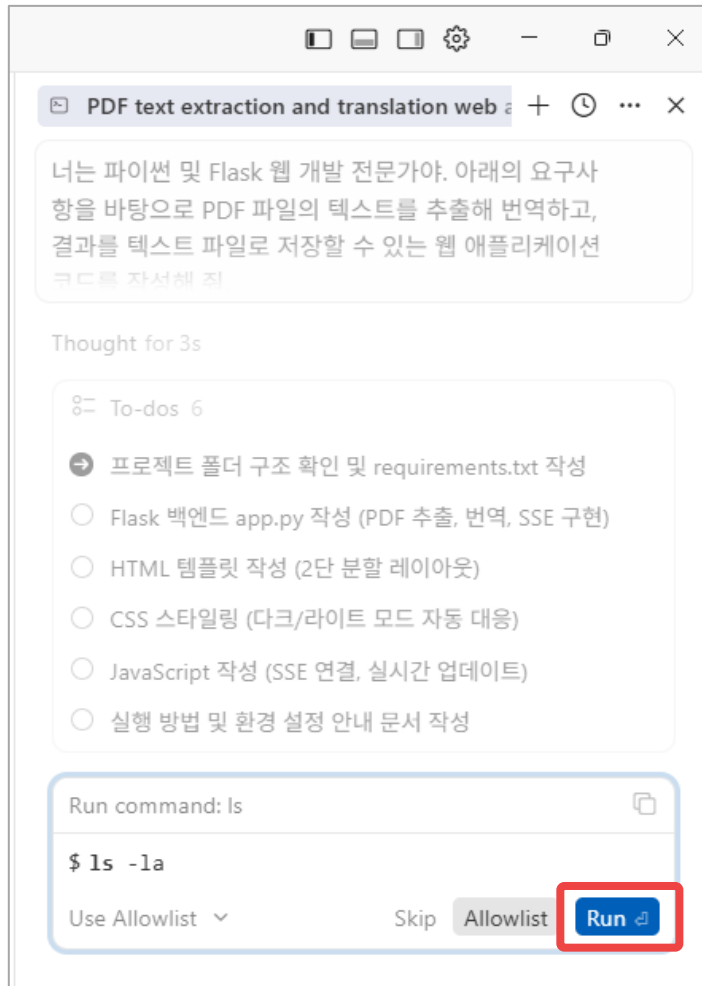
④ New Chat 클릭 : 프롬프트 입력 → 모델 선택 → 실행 클릭

The screenshot shows a web application interface for PDF translation. The interface is divided into several sections:

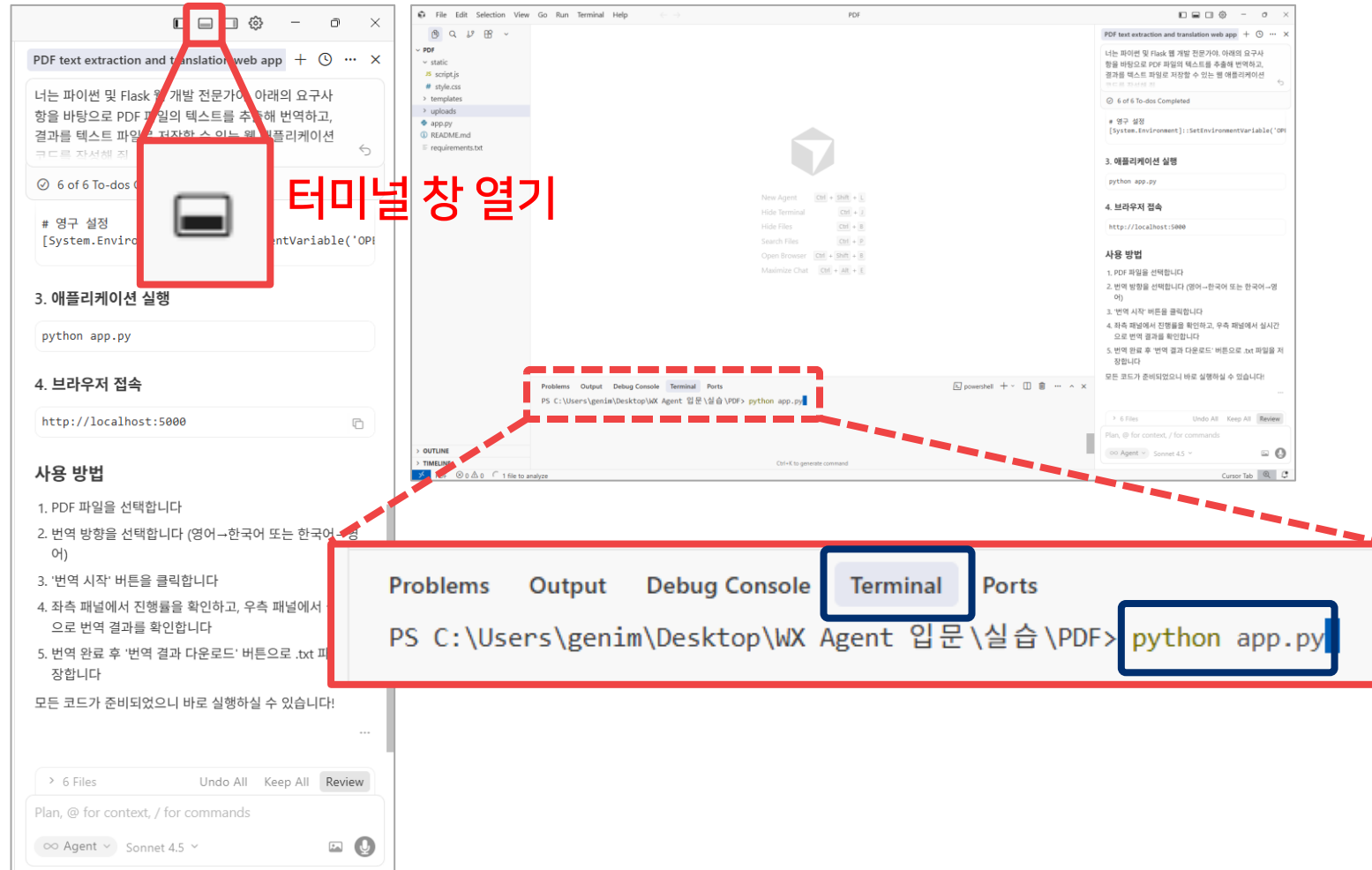
- Prompt:** A blue button labeled "Prompt" is at the top left. Below it, a text area contains the instruction: "너는 파이썬 및 Flask 웹 개발 전문가야. 아래의 요구사항을 바탕으로 PDF 파일의 텍스트를 추출해 번역하고, 결과를 텍스트 파일로 저장할 수 있는 웹 애플리케이션 코드를 작성해 줘."
- # 기술 스택:** A list of technologies: "- Backend: Python, Flask", "- PDF 텍스트 추출: pypdf (또는 PyPDF2)", "- 번역 API: OpenAI API (openai 패키지)".
- # UI/UX 및 화면 레이아웃 요구사항 (ChatGPT 스타일의 2단 분할 화면):** A list of requirements: "화면을 좌/우 두 개의 영역으로 분할하여 데스크톱에서 보기 좋게 구성해."
- 1. 왼쪽 패널 (컨트롤 및 진행 상태):** A list of requirements: "- PDF 파일 업로드 폼", "- 번역 방향 설정 (영어->한국어, 한국어->영어 선택 라디오 버튼 또는 드롭다운)", "- '번역 시작' 버튼", "- 번역 진행률(%)을 실시간으로 보여주는 프로그레스 바 또는 텍스트".
- 2. 오른쪽 패널 (미리보기 및 다운로드):** A list of requirements: "- 'PDF에서 추출한 텍스트'를 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)", "- '번역된 내용'을 보여주는 스크롤 가능한 미리보기 영역 (Textarea 또는 div)", "- 번역 완료 시 활성화되는 '다운로드' 버튼".
- 3. 스타일링:** A list of requirements: "- 배경이 어두운 색이면 글자 색깔을 흰색으로, 배경이 밝은 색이면 글자 색깔을 검은색으로 자동 적용되도록 CSS를 작성해.", "- 좌우 패널이 깔끔하게 나뉘도록 Flexbox나 Grid를 활용해 직관적으로 디자인해."
- # 핵심 기능 요구사항:** A list of requirements: "1. PDF 텍스트 추출 및 번역 연동: - pypdf를 이용해 업로드된 PDF 파일에서 '텍스트'만 추출해. - OpenAI API를 사용하여 추출된 텍스트를 선택한 언어로 번역해. - API Key는 절대 하드코딩하지 말고, 시스템 환경 변수에 저장된 'OPENAI_API_KEY'를 'os.environ'으로 불러와서 사용해. 2. 번역 진행률 표시 및 실시간 렌더링 (SSE): - 텍스트를 페이지 단위나 적절한 청크 단위로 나누어 번역을 진행해. - 번역이 진행되는 동안 왼쪽 패널의 진도율이 업데이트되고, 동시에 오른쪽 패널의 '번역된 내용' 영역에 텍스트가 실시간으로 추가되도록 SSE(Server-Sent Events)나 AJAX 폴링을 구현해. 3. 텍스트 파일(.txt) 저장 및 다운로드: - 번역이 완료되면 오른쪽 패널의 다운로드 버튼을 통해 번역된 텍스트 파일(.txt)을 받을 수 있게 해. - 다운로드되는 텍스트 파일의 이름은 업로드한 원본 PDF 파일의 이름과 동일하게 설정해 (예: 'report.pdf' 업로드 시 'report.txt' 로 다운로드)."
- # 출력 지침:** A list of requirements: "- 'app.py', 'templates/index.html' 코드와 필요한 폴더 구조를 명확히 나누어 작성해 줘. - 코드를 실행하기 위해 설치해야 하는 'requirements.txt' 내용과 터미널 실행 명령어도 함께 안내해 줘."
- New Chat:** A red box highlights the "New Chat" button in the top right corner. Below it, a text area contains the instruction: "Plan, @ for context, / for commands".
- Agent:** A dropdown menu labeled "Agent" is set to "Sonnet 4.5".
- Chat Window:** A red arrow points from the "New Chat" button to the chat window. The chat window contains the instruction: "Plan, @ for context, / for commands".
- Bottom Bar:** A bar at the bottom contains the text "PDF" and "0 0 0".

웹 개발 실습1. PDF 번역

⑤ 진행사항 승인 체크 : Run 클릭

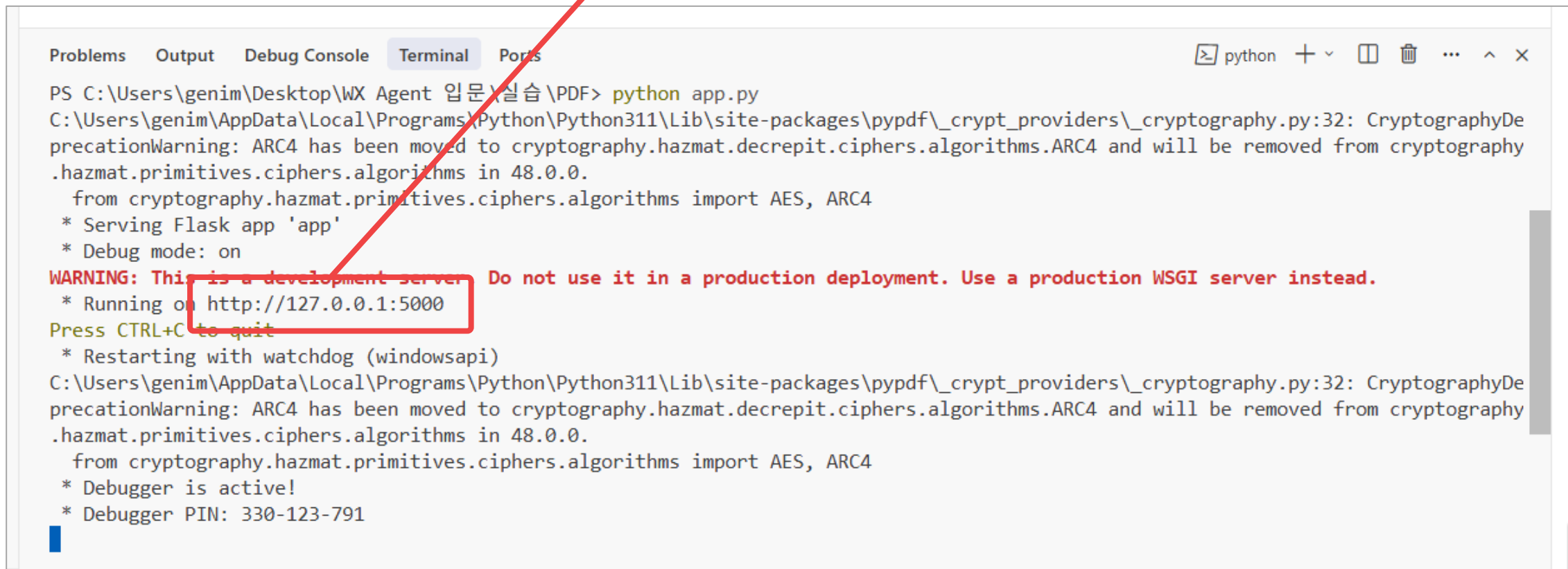


⑥ 웹 테스트 : 터미널 창 열기 → python app.py 입력 → Enter



웹 개발 실습1. PDF 번역

⑥ 웹 테스트 : ctrl 키 누른 상태에서 주소 클릭

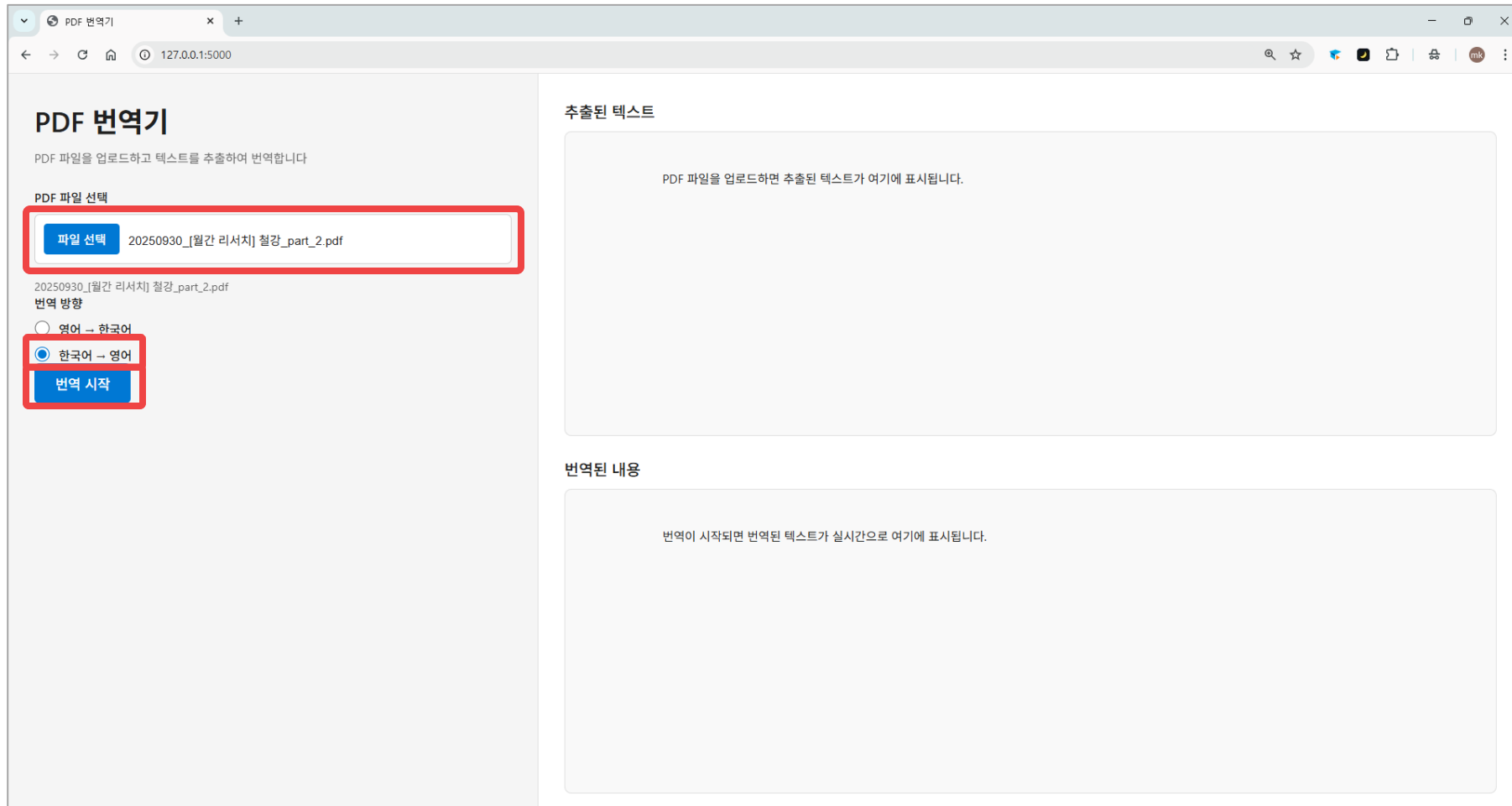


```
Problems Output Debug Console Terminal Ports
PS C:\Users\genim\Desktop\WX Agent 입문\실습\PDF> python app.py
C:\Users\genim\AppData\Local\Programs\Python\Python311\Lib\site-packages\pypdf\_crypt_providers\_cryptography.py:32: CryptographyDeprecationWarning: ARC4 has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  from cryptography.hazmat.primitives.ciphers.algorithms import AES, ARC4
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
C:\Users\genim\AppData\Local\Programs\Python\Python311\Lib\site-packages\pypdf\_crypt_providers\_cryptography.py:32: CryptographyDeprecationWarning: ARC4 has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  from cryptography.hazmat.primitives.ciphers.algorithms import AES, ARC4
* Debugger is active!
* Debugger PIN: 330-123-791
```

※ Cursor 안에서 웹페이지 실행될 경우, 크롬 창 열어서 크롬 주소창에 http://127.0.0.1:5000 입력 후 Enter

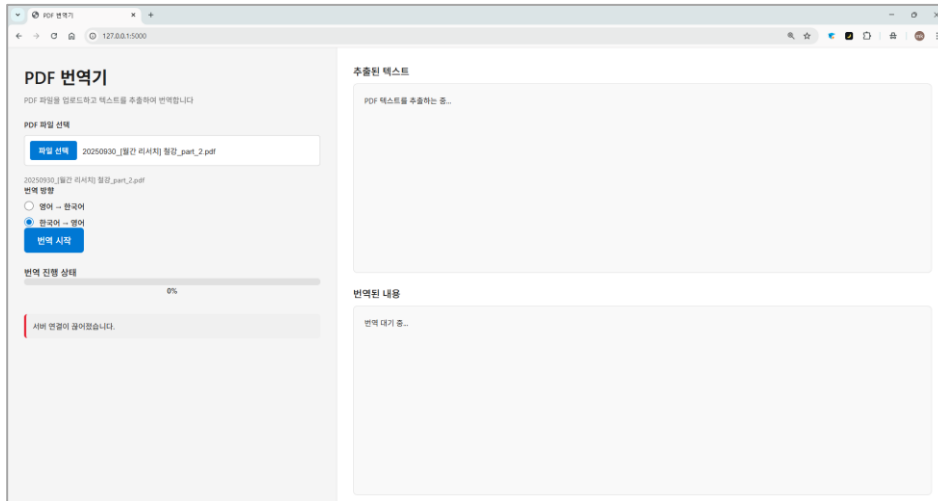
웹 개발 실습1. PDF 번역

⑥ 웹 테스트 : '20250930_[월간 리서치] 철강_part_2'파일 업로드 → 번역 방법 선택 → 번역 시작 클릭

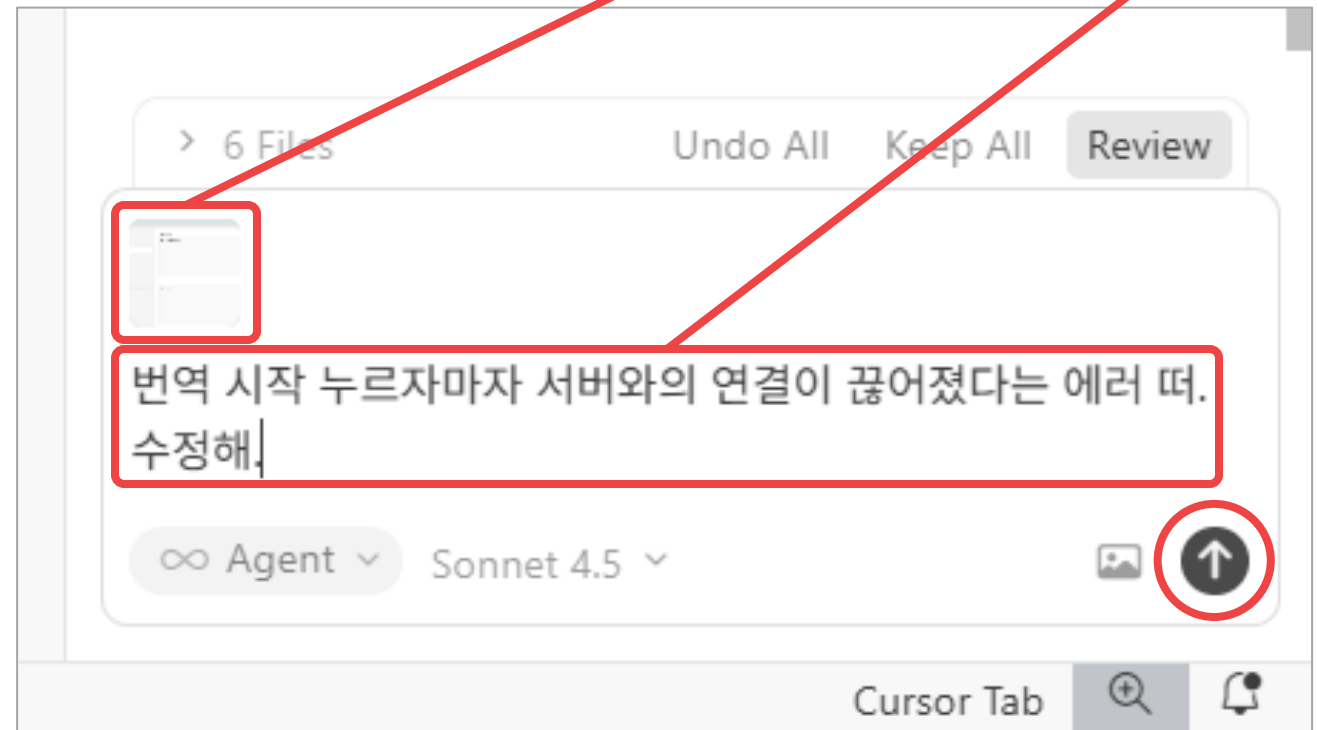


웹 개발 실습1. PDF 번역

⑦ 오류 보완 : 오류 발생한 화면 캡처(window + shift + s)



→ cursor AI 대화창에 캡처 이미지 붙여 넣고 오류 상황 입력



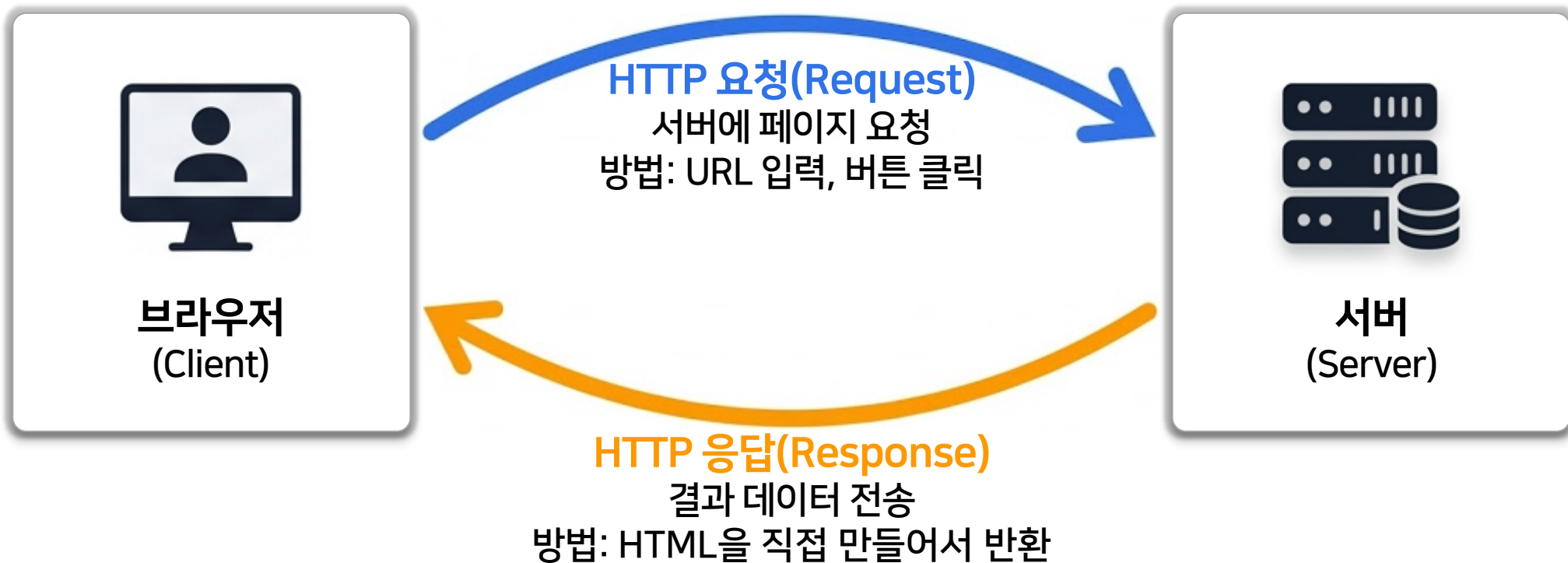
※ 수정 완료 후 웹 페이지 새로고침하여 수정 여부 확인

Flask 작동원리

- 대기 (Run): Flask 서버가 실행되면 사용자의 요청이 올 때까지 대기(리스닝)
- 요청 (Request): 사용자가 브라우저 주소줄에 주소를 입력해서 Flask 서버에 접속
- 라우팅 (Routing): `@app.route('/')` 부분이 안내 → 담당 함수(hello)를 실행
- 처리 및 응답 (Response): 함수 실행 → "안녕하세요!..."라는 문서(HTML)을 브라우저에 전송
- 브라우저 처리: 문서(HTML)를 브라우저가 렌더링(서식 등 적용)
 - 웹 서버가 보내주는 데이터는 보통 HTML이라는 문서 형식
 - 웹 브라우저는 서버가 보내준 코드와 데이터를 해석해서 그림, 버튼, 글자 등으로 바꿔주는 '전용 뷰어' 역할

HTTP란?

- 웹 브라우저와 서버가 요청과 응답을 주고받기 위한 기본 규칙
- 클라이언트가 요청하고 서버가 응답하는 웹의 기본 통신 규칙



특징

- 클라이언트-서버 구조
- 무상태성, Stateless
- 텍스트 기반 메시지 구조

왜 중요한가?

API 호출, 로그인, 파일 다운로드, 웹 페이지 표시 등 대부분의 웹 동작이 HTTP 위에서 이루어짐

대표 포트

- HTTP 기본 포트는 80번
- HTTPS는 TLS 보안 계층을 사용하며 기본 포트는 443번

IP 주소와 포트(Port)

■ IP 주소

- 인터넷에 연결된 모든 컴퓨터(서버)가 가지는 고유의 주소(예: 203.245.11.13)
- 특수 주소: 네트워크 트래픽이 외부로 나가지 않고 내부에서만 처리되도록 하는 "루프백(Loopback)" 기능을 수행
127.0.0.1은 "나 자신(내 컴퓨터)"을 가리키는 예약된 주소
localhost: 현재 사용 중인 컴퓨터 자신을 가리키는 예약된 도메인 이름(호스트명)

■ 포트(Port)

- 컴퓨터 하드웨어와 네트워크 통신에서 외부와 데이터를 주고받는 '관문' 또는 '종단점'을 의미
- 특정 애플리케이션으로 전달하는 0~65535번 사이의 논리적 번호
- 80 (HTTP): 암호화되지 않은 웹 트래픽, 443 (HTTPS): 보안 웹 트래픽
- 22 (SSH): 안전한 원격 접속, 21 (FTP): 파일 전송 프로토콜

포트 변경

- `http://naver.com` 같은 사이트들은 모두 뒤에 `:80`이 생략(80번은 HTTP) 가능
- 5000번 포트로 실행하면 주소창에 `127.0.0.1:5000`으로 접속
- `host='0.0.0.0'`: 내 컴퓨터에 연결된 모든 네트워크 장치(IP)를 통해 접속을 허용
 - 생략하면 내 컴퓨터에서만 접속

```
from flask import Flask

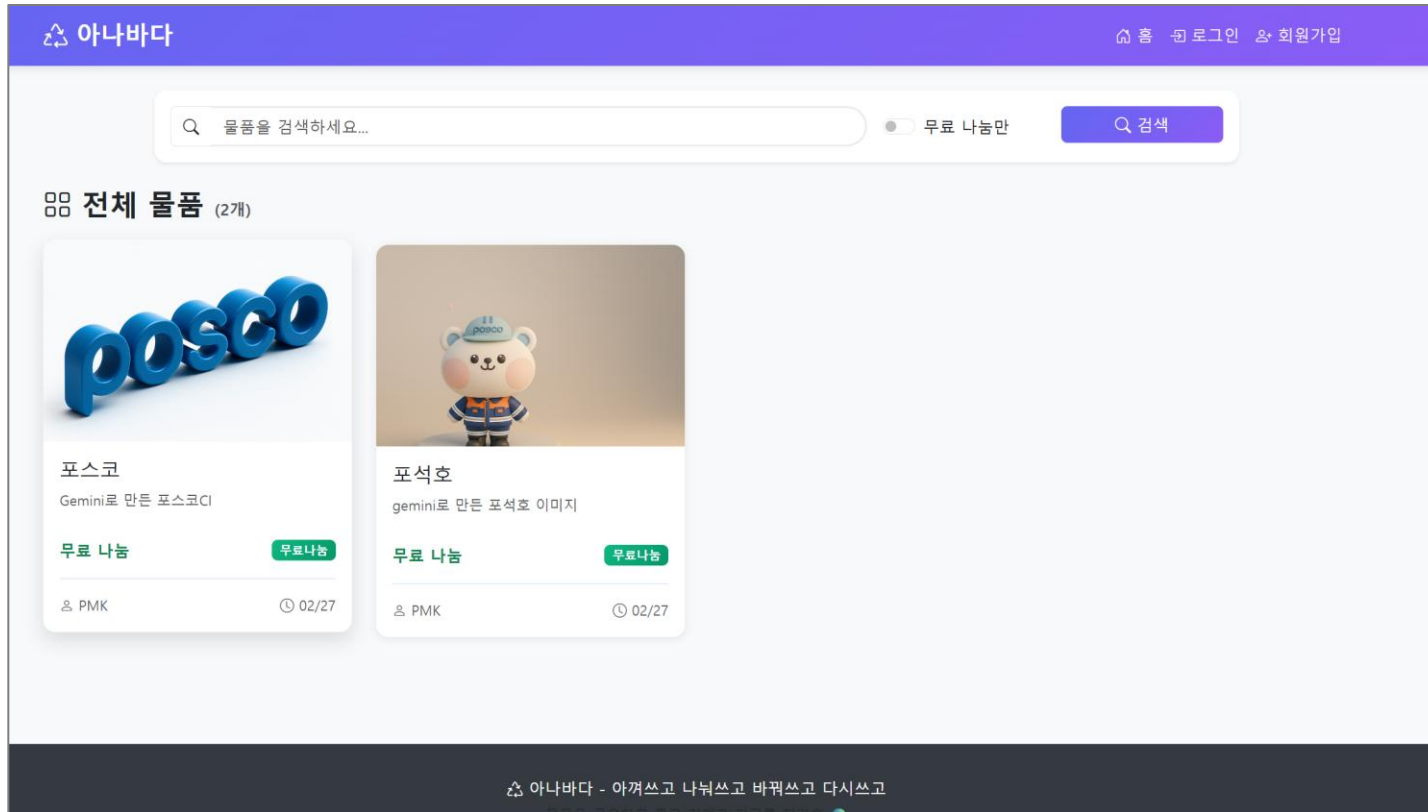
app = Flask(__name__)

@app.route('/')
def hello():
    return "80번 포트로 접속하셨습니다!"

if __name__ == '__main__':
    # port 설정을 추가하여 80번 문을 엽니다.
    app.run(host='0.0.0.0', port=80)
```

웹 개발 실습2. 물품 공유

▶ 실습목표: 물품 공유 웹 개발(flask사용, Cursor로 코드 생성)



1. 기능 : 물품 사진, 위치 등
2. 메뉴 : 등록, 수정, 삭제
3. 데이터베이스 : 등록자, 등록일, 수정일, 제품정보
4. 개발환경 : flask

웹 개발 실습2. 물품 공유

▶ 프롬프트 초안



아래 내용을 참고해서 Cursor에 넣을 프롬프트 만들어.
아나바다처럼 물품 공유 웹을 만들고 싶어

1. 개발환경

- Flask 패키지 사용
- 데이터베이스 구성: 등록자, 등록일, 수정일, 제품정보
- 물품 사진 파일 업로드 저장소 구성
- 사용자 인증 세션 관리

2. 노드설명

- 물품 사진 등록, 수정, 삭제
- 물품 설명 입력
- 가격 입력 (무료 옵션 포함)
- 물품 목록 조회 및 검색
- 사용자 회원가입 및 로그인/로그아웃
- 관심 물품 찜하기 기능
- 등록자 간 댓글 또는 문의 기능
- 거래 완료 상태 표시

3. UI설명

- 메인 페이지: 물품 목록 카드형 표시
- 물품 등록 폼 (사진 업로드, 설명, 가격/무료 선택)
- 물품 수정/삭제 버튼
- 검색창
- 회원가입 / 로그인 메뉴
- 찜하기 버튼
- 댓글/문의 입력 영역
- 거래 완료 배지 표시

웹 개발 실습2. 물품 공유

▶ 프롬프트 최종

Prompt

Flask와 Python을 사용하여 '아나바다(물품 공유 및 중고 거래)' 웹 애플리케이션을 개발해 줘. 아래의 요구사항과 데이터베이스 구조를 바탕으로 필요한 모든 코드(app.py, models.py, HTML 템플릿 등)를 작성해 주길 바라.

1. 기술 스택

- Backend: Python, Flask
- Database: SQLite 및 Flask-SQLAlchemy (ORM)
- Frontend: HTML, CSS (Bootstrap 5를 사용하여 깔끔하고 반응형으로 디자인), Jinja2 템플릿
- Image Handling: Flask-Uploads 또는 werkzeug.utils.secure_filename을 이용한 로컬 저장

2. 핵심 기능 요구사항

- 회원 관리: 사용자가 물품의 '등록자'가 되어야 하므로 간단한 회원가입 및 로그인/로그아웃 기능이 필요해 (Flask-Login 사용).
- 물품 CRUD: 사용자는 물품을 등록(사진 업로드 포함), 조회, 수정, 삭제할 수 있어야 해. 수정과 삭제는 해당 물품을 등록한 작성자만 가능해야 해.
- 물품 상세 정보: 제목, 사진(이미지), 상세 설명, 가격 정보가 들어가야 해.
- 가격 및 무료 나눔 처리: 가격을 입력받되, 가격이 '0'이거나 체크박스를 선택하면 '무료 나눔'으로 표시해.
- 물품 상태 관리 (추가 기능): 물품이 현재 '거래 중'인지 '나눔/거래 완료'인지 상태를 변경할 수 있는 버튼을 추가해.
- 검색 및 필터 (추가 기능): 메인 페이지에서 제목이나 내용으로 물품을 검색할 수 있고, '무료 나눔만 보기' 필터가 있으면 좋겠어.

3. 데이터베이스 모델 (Database Schema)

- User 모델:
 - id (Primary Key)
 - username (사용자 이름, Unique)
 - password_hash (비밀번호)
- Item 모델:
 - id (Primary Key)
 - title (물품 제목, String)
 - description (물품 설명, Text)
 - price (가격, Integer, 0이면 무료)
 - image_filename (업로드된 이미지 파일명, String)
 - status (상태: '거래가능', '거래완료' 등, String)
 - author_id (등록자, User 모델의 id를 참조하는 Foreign Key)
 - created_at (등록일, DateTime, 기본값 현재시간)
 - updated_at (수정일, DateTime, 수정 시 업데이트)

4. 디렉토리 구조 및 단계별 작업

- 프로젝트의 기본 디렉토리 구조를 먼저 제시해 줘.
- 그 다음, app.py 세팅, 데이터베이스 모델 구성, 라우팅, 그리고 프론트엔드 HTML(base.html, index.html, item_detail.html, item_form.html 등) 코드를 순서대로 제공해 줘.

웹 개발 실습 3. 예측 프로그램

▶ 프롬프트



커서로 예측 시스템을 만들건데, 아래 내용을 커서에 넣을 프롬프트로 작성해줘
파이썬 Flask를 활용한 예측 시스템을 개발하고 싶어

1. 개발환경

- Flask 패키지 사용
- 데이터 처리: pandas 패키지
- 그래프: matplotlib (한글 깨짐 방지 설정), Plotly, Dash
- 세션 관리: SQLite3 (단계별 결과 저장, 전 단계 결과를 다음 단계에서 사용)
- 파일 업로드: csv, excel 지원
- 전체 시스템 한글로 구현

2. 노드설명

- 파일 업로드부터 시뮬레이션까지 순차적으로 진행
- 각 단계의 결과는 SQLite3에 저장되며, 다음 단계에서 이전 결과를 불러와 사용
- 파일 업로드: csv, excel 업로드 후 pandas로 10개 행 출력 및 기초 통계량 출력
- 변수 선택: 종속변수 및 독립변수 선택, 이후 분석에 적용
- 결측치 제거: 결측치 행 탐색 및 제거, 총 행 수 / 제거 행 수 / 잔여 행 수 출력
- 이상치 탐색:
 - 변수별 ± 3 표준편차 범위 밖 데이터 탐색
 - 변수별 이상치 제거 여부 선택 및 제거
- 탐색적 분석:
 - 히스토그램: 변수 선택 후 히스토그램 + 밀도 곡선 출력
 - 산점도: 다중 변수 선택 후 산점도 출력
 - 상관관계(히트맵): 전체 변수 상관관계 히트맵 출력

- 회귀분석:

- 전진선택법 / 후진제거법 / 릿지 / 라쏘 선택
- 회귀모형 및 예측 정확도 출력, 최종 모델 선택
- 모델 검증: 잔차분석 및 적합도 검정 결과 출력
- 시뮬레이션: 선택된 회귀모형으로 시뮬레이션, 독립변수 슬라이더바 입력, 종속변수 값 및 막대그래프 출력
- 단계별 순차 진행 (전 단계 완료 후 다음 단계 활성화)
- 추가 기능 필요 시 사용자에게 확인 후 추가

3. UI설명

- 컬러: 연한 녹색, 연한 파란색, 회색- 화면 2분할: 좌측 20% / 우측 80%
- 우측 화면: 상단 10% (현재 단계 표시) / 하단 90% (서브메뉴 및 실행 결과)
- 좌측 메뉴 구성: 1.파일업로드 / 2.변수선택 / 3.결측치제거 / 4.이상치탐색 / 5.탐색적 분석 / 6.회귀분석 / 7.모델검증 / 8.시뮬레이션
- 좌측 메뉴: 현재 단계 진하게, 나머지 연하게 표시, 단계 완료 시 다음 메뉴 활성화
- 우측 상단: 현재 메뉴 단계명 표시
- 우측 하단: 서브메뉴 및 실행 결과 표시
- 탐색적 분석 서브메뉴: 히스토그램 / 산점도 / 상관관계(히트맵) 탭
- 회귀분석 서브메뉴: 전진선택법 / 후진제거법 / 릿지 / 라쏘 탭
- 시뮬레이션: 회귀식 출력, 슬라이더바 입력, 종속변수 값 및 막대그래프 한 화면 구성 (Plotly/Dash)

M2. 바이트코딩 활용 앱/웹 개발

6. 바이트코딩 스킬 활용

Spec-Driven Development (SDD)

DeepLearning.AI에서는 SSD를 "명확한 마크다운 스펙을 먼저 작성하고, AI코딩 에이저트가 그것을 구현하게 하는 방식"으로 정의

■ 3개 MD 파일 적용

구분	API
architecture.md	프로젝트의 전체 아키텍처와 기술적인 설계 기준을 정의하는 파일
prompts.md	AI가 프로젝트를 개발할 때 따라야 하는 작업 규칙과 프롬프트를 정의하는 파일
ui.md	UI/UX와 디자인에 대한 기준을 정의하는 파일

architecture.md

- 프로젝트 목적
- 상태 관리 방식
- 핵심 기능
- 인증 및 권한
- 기술 스택
- 비즈니스 로직
- 디렉터리 구조
- 개발 제약사항
- 데이터 모델
- API 설계

prompts.md

- AI 역할(Role)
- 테스트 작성 기준
- 개발 원칙
- 응답 형식
- 코드 스타일
- 금지사항
- 구현 우선순위
- 자주 사용하는 프롬프트 템플릿
- 리팩토링 기준
- 에러 처리 규칙

ui.md

- 디자인 컨셉
- 페이지별 UI 요구사항
- 컬러 시스템
- 인터랙션
- 타이포그래피
- 접근성 기준
- 컴포넌트 규칙
- 레이아웃
- 반응형 기준

SDD 방법론 적용 효과



AI 기억력 보완

- MD = AI의 장기 기억 장치. @참조 한 번으로 모든 맥락을 AI에게 전달. 설명 반복 끝!



일관성 유지

- 스타일·구조·규칙이 흔들리지 않음. 10번째 기능도 1번째와 같은 스타일로 생성됨



시간 절약

- 매번 설명 반복 없이 @참조 한 번으로 끝. 설명 시간 대신 기능 추가에 집중 가능



협업 가능

- 팀원이 MD 3개만 읽으면 프로젝트 파악 완료. 온보딩 시간 대폭 단축



내 자산이 됨

- MD는 코드보다 오래 쓰는 설계 문서. 다음 프로젝트에도 템플릿으로 재사용 가능

MD 파일 3개 작성에 드는 1시간이 이후 수십 시간을 절약!

md 파일 작성 요청(프롬프트)

■ STEP 1 — Cursor Chat (Ctrl+L) 열고 아래 프롬프트 입력



나는 "오늘의 할일" 앱을 만들 거야.

기능: 할일 추가, 완료 체크, 삭제, 카테고리(업무/개인/공부) 분류, 마감일 설정

- Backend: Python Flask

- Frontend: 순수 HTML + CSS + JavaScript

- Database: SQLite3

아래 3개 MD 파일 초안을 각각 작성해줘.

파일에 바로 붙여넣을 수 있게 마크다운 형식으로, 구체적으로 작성해줘.

1. ui.md : 페이지 목록, 컴포넌트 구조, 색상·폰트 규칙

2. architecture.md: 기술 스택, 폴더 구조, API 엔드포인트, DB 스키마

3. prompts.md : AI 코딩 규칙, 자주 쓸 프롬프트 템플릿

■ STEP 2 — 생성된 3개 파일 확인



프로젝트이름

/todo-app/

├── ui.md

├── architecture.md

└── prompts.md

architecture.md 예시

```
# architecture.md - 기술 설계도

## 기술 스택
- - Backend : Python 3.x + Flask
- Frontend : 순수 HTML + CSS + JavaScript
- 통신 방식: JavaScript fetch API → Flask JSON 응답
- Database : SQLite (todos.db 파일, 자동 생성)

## 폴더 구조
/todo-app
  app.py          → Flask 앱 메인 파일 (라우트 + DB 처리)
  todos.db        → SQLite 데이터베이스 파일 (자동 생성)
  /static
    index.html    → 단일 페이지 (HTML)
    style.css     → 전체 스타일
    main.js       → 할일 추가·삭제·체크·필터 JavaScript

## DB 스키마 - todos 테이블
id(uuid PK) | title(text) | category(text) | due_date(date)
is_done(boolean DEFAULT false) | created_at(timestamp)

## API 설계
GET /api/todos      → 전체 조회      POST /api/todos      → 추가
PATCH /api/todos/[id] → 완료 토글    DELETE /api/todos/[id] → 삭제
```

prompts.md 예시

```
# prompts.md - AI 지시 규칙 & 프롬프트 레시피

## AI가 항상 지킬 규칙
- Python 코드는 함수 단위로 깔끔하게 분리
- Flask API는 항상 JSON으로 응답 { "data": ..., "error": null }
- 에러는 try-except로 처리, 에러 메시지는 한국어로
- HTML은 /static/index.html 한 파일
- CSS는 /static/style.css 한 파일
## 프롬프트 템플릿

### 새 기능 추가 (Plan 모드 먼저)
@ui.md @architecture.md @prompts.md 를 참고해서
[기능명] 기능을 추가하는 계획을 만들어줘.
- 수정할 파일 목록
- 각 파일에서 변경할 내용 요약
계획이 맞으면 코드를 작성해줄게.
```

ui.md 예시

```
# ui.md - 화면 설계도

## 색상 & 폰트
- 주색: #2563EB (파랑)   배경: #FFFFFF   텍스트: #1A1A1A
- 완료 항목: #94A3B8 (회색)   경고(마감 지남): #DC2626 (빨강)
- 폰트: Pretendard / 제목 20px Bold / 본문 14px Regular

## 메인 페이지 (/)
- 헤더: 로고 "오늘의 할일" (좌) + "+ 추가" 버튼 (우, 파란색)
- 카테고리 탭: [전체] [업무] [개인] [공부]
- 빈 상태: "할일이 없어요! 추가해보세요 😊"

## 할일 카드 컴포넌트
- 체크박스(좌) + 제목 + 카테고리 뱃지 + 마감일(우)
- 완료 시: 취소선 + 전체 회색   마감 지남: 마감일 텍스트 빨간색

## 컴포넌트 규칙
- 버튼: rounded-lg px-4 py-2   카드: shadow-sm border rounded-xl p-4
```

3개 MD 파일 완성! → Chat에서 @참조하며 기능 만들기 시작!

개발 절차

아이디어 기획에서 개발 · 테스트까지

①기획

아이디어 구체화

대략적인 아이디어를
질문을 통해 구체화

grill-me

skill 활용

②설계

설계안 작성

목적 · 사용자 · 핵심
기능 · 제약조건 작성

to-prd

③개발

시스템 구현

작성된 설계안을
바탕으로 구현

plan 모드&skill 활용

④테스트

작동 확인

작성된 설계안을
바탕으로 구현

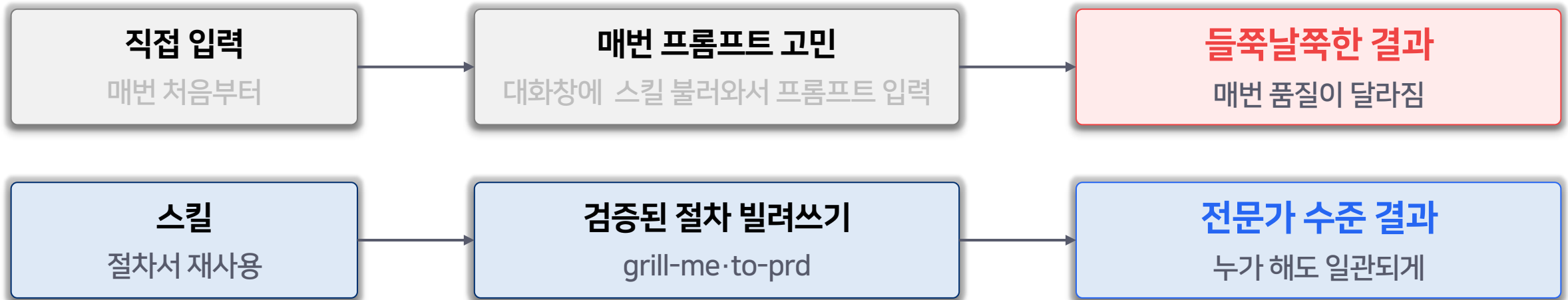
diagnose

⑤배포 · 운영

공유/유지보수

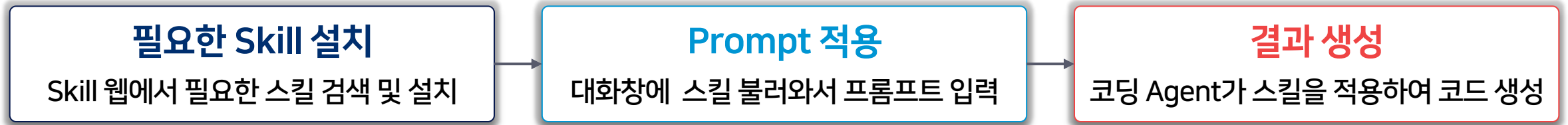
참고. Skill이란?

- AI 코딩 도구(Claude, Codex 등)가 특정 작업을 수행할 때 참조하는 구조화 된 지식 문서
- 사용하는 이유
 - 매번 “어떻게 물어보지? 어떤 프롬프트를 쓰지?” 고민할 필요가 없음. 스킬이 그 역할을 대신 수행
 - 비개발자도 전문가 수준의 질문·문서화를 바로 제작 가능
 - 좋은 절차들은 이미 스킬로 구현되어 있음. 잘 만들어진 것을 활용하는 것도 실력

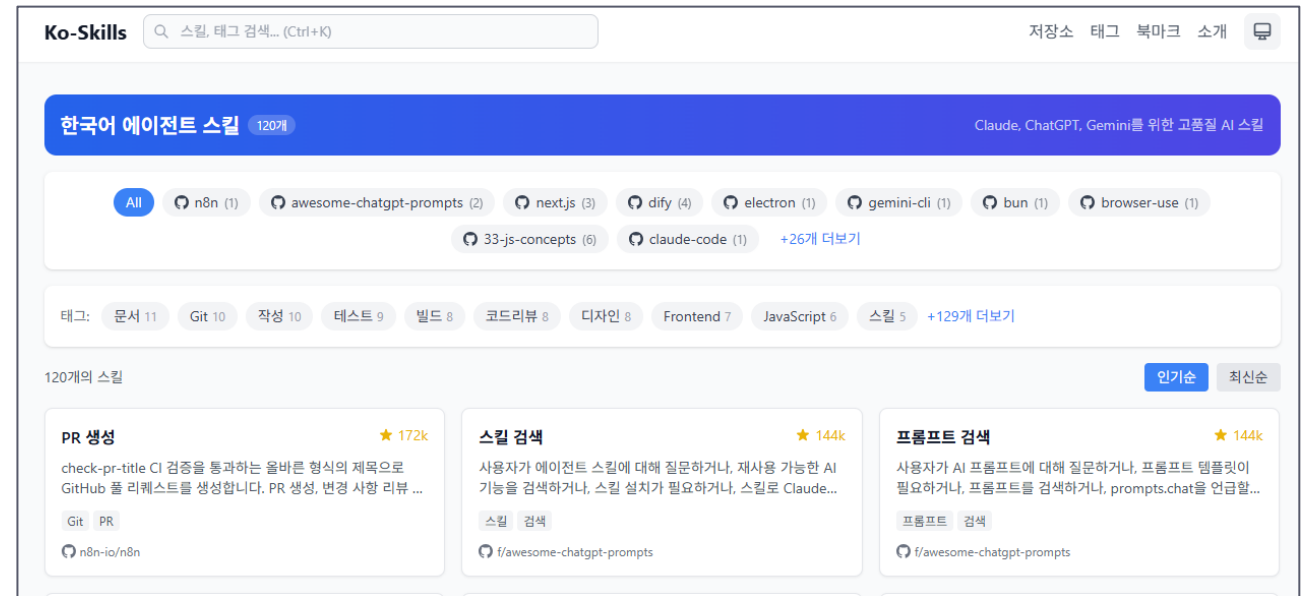
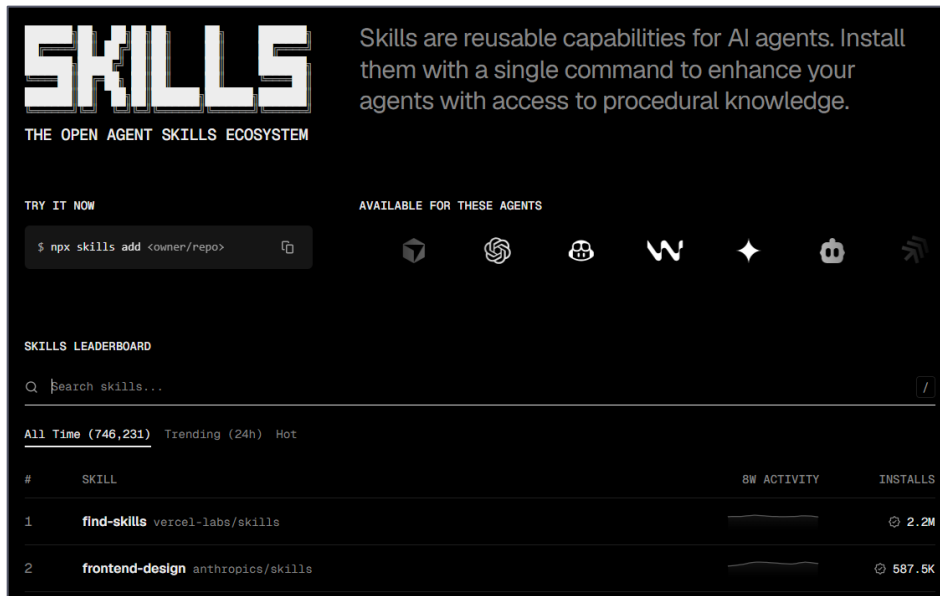


참고. Skill 활용 방법

■ 활용 순서



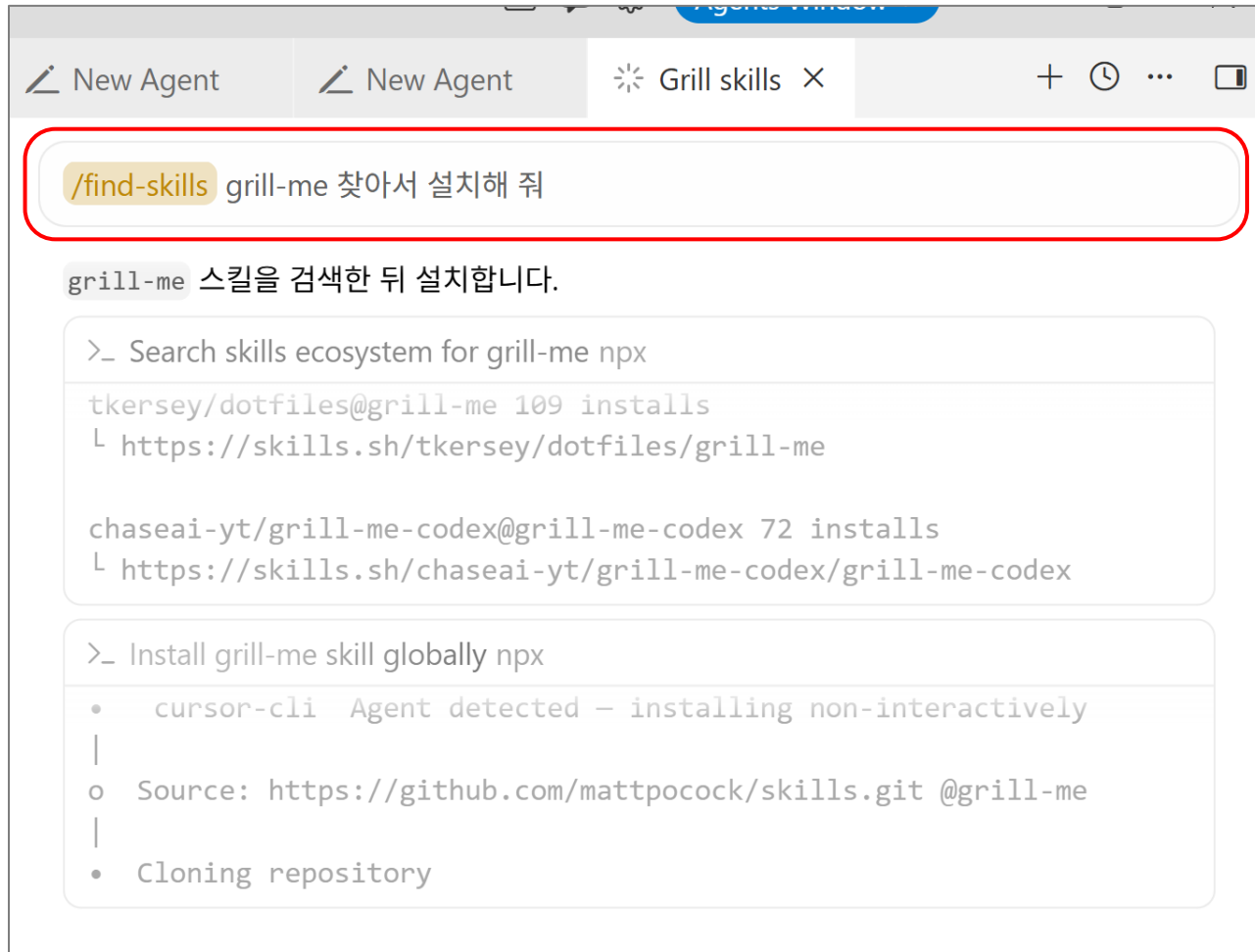
- Skill 검색 사이트 : <https://www.skills.sh>, <https://ko-skills.pages.dev>



참고. Skill 설치 방법

▪ find-skill을 이용한 설치

- Cursor Agent창에 find-skills불러와서 필요한 스킬 설치 요청



참고. 주요 Skill

기능	특징	주요기능	웹주소
grill-me	코드 작성 전 설계를 집요하게 검증하는 사전 검토 도구	① 계획/설계를 질문으로 집요하게 검증 ② 결정 트리 분기별로 하나씩 해소 ③ 코드베이스 탐색으로 중복 질문 자동 생략	skills.sh/mattpocock/skills/grill-me
to-prd	추가 질문 없이 대화만으로 PRD 완성 후 이슈 자동 등록	① 대화 맥락만으로 PRD 자동 생성 ② 사용자 스토리·구현 결정·테스트 계획 포함 ③ 이슈 트래커에 needs-triage 라벨로 자동 게시	skills.sh/mattpocock/skills/to-prd
diagnose	6단계 체계로 어려운 버그를 끝까지 추적하는 디버깅 규율	① 피드백 루프 구축이 핵심 — 재현 가능한 신호 확보에 최우선 ② 3~5개 가설을 먼저 나열하고 순위 매긴 뒤 검증 ③ 수정 전 회귀 테스트 작성 → 수정 → 디버그 코드 전부 제거	skills.sh/mattpocock/skills/diagnose
frontend-design	Anthropic 공식, 디자인 결정을 AI가 자동 판단	① UI 색상/타이포그래피/레이아웃 가이드 적용 ② 디자인 일관성 자동 유지 ③ 컴포넌트 스타일 결정 지원	skills.sh/anthropics/skills/frontend-design
vercel-react-best-practices	Vercel 공식, 배포 환경까지 고려한 React 패턴	① Vercel 최적화 React 패턴 ② 성능/렌더링 모범 사례 ③ 배포 친화적 코드 구조	skills.sh/vercelHabs/agent-skills/vercel-react-best-practices
web-design-guidelines	접근성·반응형·UX를 종합한 웹 디자인 원칙	① 웹 접근성 및 반응형 원칙 ② UX 중심 설계 가이드 ③ 크로스브라우저 호환성 고려	skills.sh/vercelHabs/agent-skills/web-design-guidelines
systematic-debugging	감이 아닌 체계적 가설 검증으로 버그 추적	① 버그 원인을 단계별로 좁혀가는 프로세스 ② 가설 설정 → 검증 → 범위 축소 반복 ③ 디버깅 과정 문서화 지원	skills.sh/obra/superpowers/systematic-debugging

참고. 주요 Skill

기능	특징	주요기능	웹주소
shadcn	shadcn 공식, 컴포넌트 설치부터 커스터마이징까지	① shadcn/ui 컴포넌트 올바른 설치/사용법 ② 테마 커스터마이징 가이드 ③ Radix UI 기반 접근성 패턴	skills.sh/shadcn/ui/shadcn
tdd	테스트를 먼저 쓰는 Red-Green-Refactor 루프 강제	① Red-Green-Refactor 루프 적용 ② 테스트 먼저 작성 후 구 ③ 통합·단위 테스트 전략	skills.sh/mattpocock/skills/tdd
pdf	Anthropic 공식, PDF 생성부터 추출까지 올인원	① PDF 생성/병합/분할 ② 텍스트·이미지 추출 ③ 폼 필드 작성 및 암호화	skills.sh/anthropics/skills/pdf
pptx	Anthropic 공식, PPTX 생성·편집·파싱 전반 지원	① 슬라이드 덱 생성 및 편집 ② 레이아웃·노트·템플릿 처리 ③ 기존 PPTX 파싱 및 내용 추출	skills.sh/anthropics/skills/pptx
seo-audit	SEO 구조를 자동 분석해 우선순위별 개선안 제시	① 페이지 SEO 구조 분석 ② 개선 우선순위 제안 ③ 메타/구조화 데이터 점검	skills.sh/coreyhaines31/marketingskills/seo-audit
docx	Anthropic 공식, Word 문서 생성·편집 전담	① Word 문서 생성 및 서식 적용 ② 목차·페이지번호·헤더 처리 ③ 기존 .docx 편집 및 내용 추출	skills.sh/anthropics/skills/docx

참고. Skill 실제 파일 구조 예시

▪ Markdown 파일(.md) 형태

```
# grill.me Skill
```

```
## 역할
```

당신은 아이디어 검증 전문가임.

```
## 규칙
```

1. 아이디어 입력받으면 7가지 질문 순서대로 함.
2. 질문은 한 번에 하나씩만 함.

```
## 질문 목록
```

1. 누가 씀? 2. 어떤 문제를 해결함?

```
## 출력 형식
```

****문제 정의****: {답변 기반으로 정리}

****대상 사용자****: {답변 기반으로 정리}

```
---
name: to-prd
description: Turn the current conversation into a PRD and publish
disable-model-invocation: true
---

This skill takes the current conversation context and codebase and
The issue tracker and triage label vocabulary should have been pro

## Process

1. Explore the repo to understand the current state of the codebase
2. Sketch out the seams at which you're going to test the feature.
Check with the user that these seams match their expectations.
3. Write the PRD using the template below, then publish it to the

<prd-template>

## Problem Statement

The problem that the user is facing, from the user's perspective.

## Solution

The solution to the problem, from the user's perspective.

## User Stories

A LONG, numbered list of user stories. Each user story should be

1. As an <actor>, I want a <feature>, so that <benefit>
```

참고. 마크다운(MarkDown)

- 특수기호로 문서 구조를 표현하는 가벼운 문법
- 사용문법
 - # 하나면 큰 제목 됨
 - **별표 두 개**면 굵게 됨
 - -(하이픈)이면 목록 됨
 - 문법이 단순해서 실험 기록에 잘 맞음

“글을 꾸미는 것”보다 “구조를 만드는 것”에 가까움

참고. 마크다운 예시

기능	입력	결과
제목(H1)	# 제목	제목
제목(H2)	## 제목	제목
제목(H3)	### 제목	제목
굵게	**굵은 글씨**	굵은 글씨
기울임	<i>*기울임*</i>	<i>기울임</i>
취소선	~~삭제~~	삭제
굵게 + 기울임	<i>***강조***</i>	<i>강조</i>
인라인 코드	`print()`	print()
코드 블록	python print("Hello") 	코드 블록 생성
순서 없는 목록	- 사과	사과
순서 있는 목록	1. 사과	1. 사과

기능	입력	결과
체크박스	- [x] 완료	☒ 완료
체크박스	- [] 미완료	☐ 미완료
링크	[OpenAI](https://openai.com)	OpenAI
이미지	![설명](image.png)	이미지 표시
인용문	> 인용문	> 인용문
수평선	---	_____
표	이름 나이	표 생성
이스케이프	*별표*	*별표*

실습주제 예시

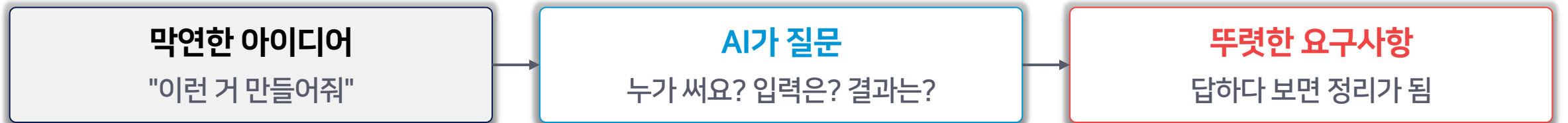
- 메모 웹(DB)
- 레트로게임 웹 : 테트리스, 너구리, 갤러그, 미로찾기 + 알파
- Youtube 링크를 입력하면 영상/음원/영상+음원을 다운로드 및 저장하는 웹
- 일정 관리 웹(달력)
- 오늘날씨~일주일날씨 대시보드: 추천 옷차림+챙길 물품
- 업로드한 파일 내용
- 계산기
- 음식 재료를 입력하면 만들 수 있는 요리를 추천해주는 앱
- 동호회 멤버들끼리 매일 실천하는 챌린지 앱
- 회의록을 바탕으로 회의내용을 정리하고 누가 무엇을 언제까지 해야 하는지 작성

①기획 : 아이디어 구체화하기

- 구체화가 필요한 이유 : 막연한 아이디어 한 줄로 요청시, 빈칸은 AI가 제멋대로 채움



- grill-me 스킬을 활용한 아이디어 구체화 * grill-me는 하나씩 집요하게 물어봄. "질문N개까지"처럼 범위를 정해두면 좋음
 - 내가 AI에게 요청하는 게 아니라 AI가 나에게 필요한 기술을 물어봄



- 스킬 설치 링크: <https://www.skills.sh/mattpocock/skills/grill-me>

참고. grill-me

- 아이디어에 여러 질문을 던져서 아이디어를 구체화하고 검증하는 Skill
- 처음 아이디어는 "이런 거 있으면 좋겠다" 수준임. 근데 실제 개발하려면 누가 쓸지, 무슨 문제 풀건지, 핵심 기능이 뭔지, MVP 범위가 어딘지 정해야 함

▪ 입력, 과정, 그리고 산출물

1. Cursor Chat에 grill-me 스킬을 활용해서 아이디어만 입력하면 됨



/grill-me 냉장고 재료 입력하면 만들 수 있는 요리 추천하는 앱 만들고 싶어

2. grill-me가 던지는 질문들

- 누가 씬?
- 어떤 문제가 있음?
- 기존 서비스랑 뭐가 다름?
- 젤 중요한 기능 하나만 꼽으면?
- 첫 번째 버전(MVP)은 어디까지임?
- 성공 기준이 뭐임?

3. 대답하다 보면 아이디어가 구체화되고, 최종적으로 아이디어 검증 문서가 나옴

- 문제 정의 / 대상 사용자
- 사용자 니즈 / 핵심 가치
- MVP 범위 / 제외 기능
- 성공 기준 / 위험 요소



다음 단계(to-prd)활용시 기초 자료가 됨

②설계 : 기획서 작성하기

- 구체화 한 아이디어를 문서로 정리
- to-prd 스킬을 활용한 문서 제작 *prd : Product Requirements Document
 - PRD: 무엇을 만들지 적은 '주문서', 현장의 작업표준서(SOP)와 같은 역할
 - PRD가 담는 내용

목적

LLM, 코딩 Agent

사용자

누가 쓰나

핵심 기능

어떤 동작이 필요한가

제약 조건

지켜야 할 형식·범위

- 인터뷰 없이 PRD 생성: 사용자에게 추가 질문 없이 현재 대화와 코드베이스만 보고 PRD 작성
- 테스트 포인트 식별: 아키텍처 수준에서 테스트 가능한 지점을 찾아냄
- 도메인 용어 적용: 프로젝트의 기존 용어집(ADR 등) 반영
- 이슈 트래커 자동 게시 — ready-for-agent 라벨을 붙여 AI 에이전트가 바로 작업 픽업 가능하도록 처리
- 스킬 설치 링크: <https://skills.sh/mattpocock/skills/to-prd>

참고. to-prd

- 아이디어 검증 결과를 바탕으로 제품 요구사항 문서(PRD)를 생성해 주는 스킬
- PRD(Product Requirements Document): 개발자가 바로 구현을 시작할 수 있는 설계 문서
- to-prd 산출물에서 코딩까지
 - grill-me에서 나온 결과물을 바탕으로 개발에 필요한 기준들이 정리되어 파일로 나옴

- 누가 씀?
- 어떤 문제가 있음?
- 기존 서비스랑 뭐가 다름?
- 젤 중요한 기능 하나만 꼽으면?
- 첫 번째 버전(MVP)은 어디까지임?
- 성공 기준이 뭐임?

- 위에 나온 파일을 바탕으로 Cursor Chat의 plan모드로 프롬프트 작성



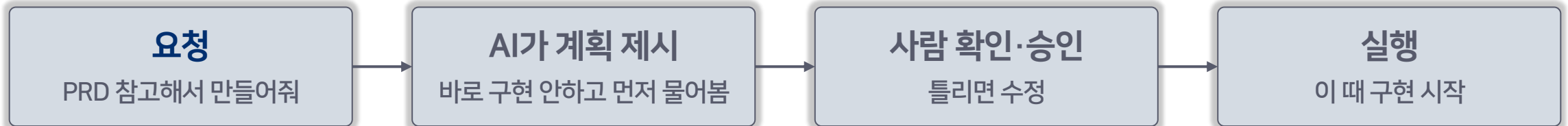
prd파일들을 바탕으로 웹 만들어줘

③개발 : 구현하기

- 구현 순서: 커서 프롬프트창의 Agent 모드를 Plan모드로 변경 후 실행

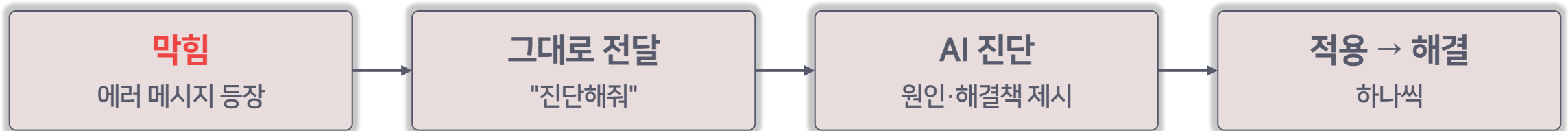


- Plan Mode에서 사람이 확인하면서 구현될 모습 최종 점검



※ 승인한 계획대로 구현 시작 → 가장 핵심 기능 하나부터 동작하게 → 중간중간 눈으로 확인하며 가볍게 다듬기

- diagnose 스킬을 이용해서 막힘 해결



※ 스킬 설치 링크: <https://skills.sh/mattpocock/skills/diagnose>

참고. Plan 모드 사용 및 검토 사항

- AI가 코드를 바로 짜지 않고, "이렇게 만들겠습니다"라는 계획을 먼저 보여주는 모드

- plan모드 사용법

1. Composer 열기 (Ctrl+I)
2. 상단 드롭다운에서 "Plan" 선택
3. @MD 파일 참조 + 프롬프트 입력
4. AI가 계획을 텍스트로 보여줌 (코드는 아직 없음)
5. 계획 검토 → 문제 있으면 수정 요청 → 문제 없으면 "진행해줘" 또는 Agent 모드로 전환
6. 실제 코드 생성

- Plan 모드에서 확인할 사항

- 만들려는 파일 목록이 맞는가?
- 수정하려는 파일이 올바른가? (엉뚱한 파일 건드리면 안 됨)
- 접근 방식이 architecture.md와 일치하는가?
- 빠진 기능이나 조건은 없는가?

참고. Plan 모드 예시

Plan 모드 프롬프트

```
@ui.md @architecture.md @prompts.md
```

할일 추가 기능을 만들 계획을 세워줘.

- 어떤 파일을 만들고 수정하는지
 - 각 파일에서 무엇을 작성하는지
 - Flask API와 JavaScript가 어떻게 연결되는지
- 계획만 먼저 보여줘. 코드는 아직 작성하지 마.

Plan 검토 체크리스트

파일 관련

- [] 만들거나 수정하는 파일 목록이 architecture.md와 일치하는가?
- [] 건드리면 안 되는 파일을 건드리지 않는가?
- [] 새로 만드는 파일이 올바른 폴더에 위치하는가?

기능 관련

- [] 내가 요청한 기능이 계획에 모두 포함됐는가?
- [] 빠진 조건이나 엣지 케이스는 없는가?
- [] API URL이 architecture.md의 설계와 맞는가?

흐름 관련

- [] JavaScript → Flask API → SQLite 흐름이 맞는가?
- [] 성공/실패 처리가 계획에 포함됐는가?
- [] 화면 업데이트(DOM) 방식이 자연스러운가?

체크리스트 검토 후

- 이상 있으면: 3번 항목에서 API URL을 /api/todos로 바꿔줘. 그리고 삭제 확인 창(confirm)이 빠져있는데 추가해줘.
- 이상 없으면: 계획대로 진행해줘

④테스트 : 확인하기

- AI에게 테스트를 맡기고, 실패를 반복 개선해 결과물을 보완
 - 막힌 곳은 함께 진단(diagnose) 활용 → 핵심 기능 하나는 반드시 동작하게
 - 완성 못해도 괜찮음 → "검증할 수 있는 상태"까지만 해도 됨
 - AI에게 프롬프트로 검증 요청



지금 만든 기능에 대해 테스트 코드를 작성하고 실행해줘.
정상 입력, 잘못된 입력(빈칸·글자·음수), 경계값(0·아주 큰 수)도 함께 확인해줘.
결과는 비개발자도 알 수 있게 통과/실패로 요약해줘.



사용자가 쓰는 전체 흐름을 처음부터 끝까지 확인하는 E2E 테스트도 만들어 실행해줘.
실패한 부분이 있으면 원인을 쉽게 설명하고 고쳐줘.



검토 결과를 html문서 대시보드 형태로 만들어줘

M3. 바이트코딩 활용 RAG 구현

7. LangChain 이해

LangChain 맛보기

▶ llm 객체 생성하여 간단한 질의 하기

```
from langchain_openai import ChatOpenAI
# import httpx 사내 사용시

# 객체 생성
llm = ChatOpenAI(
    # api_key= "sk- 실습시 발급한 키 입력"
    # http_client = httpx.Client(verify=False) 사내에서 사용시 입력
)

question = "대한민국의 수도는 어디인가요?"

# 질의
llm.invoke(question)
```

```
AIMessage(content='대한민국의 수도는 서울입니다.', additional_kwargs={'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 16, 'prompt_tokens': 24, 'total_tokens': 40, 'prompt_tokens_details': {'cached_tokens': 0, 'audio_tokens':
0}, 'completion_tokens_details': {'reasoning_tokens': 0, 'audio_tokens': 0, 'accepted_prediction_tokens': 0,
'rejected_prediction_tokens': 0}}, 'model_name': 'gpt-3.5-turbo-0125', 'system_fingerprint': None, 'finish_reason': 'stop',
'logprobs': None}, id='run-eed6d406-51c9-4ee5-8923-f6b4d747f7e9-0', usage_metadata={'input_tokens': 24, 'output_tokens': 16,
'total_tokens': 40}))
```

LangChain 맛보기

▶ 답변을 실시간으로 출력하기

- stream 메소드는 LLM의 출력을 실시간으로 받아오며 for loop 를 통해 출력

```
answer = llm.stream("부산의 아름다운 관광지 5곳과 꼭 맛보아야 할 음식 5곳을 추천해줘")
# 스트리밍 방식으로 각 토큰을 출력 (실시간 출력)
for token in answer:
    print(token.content, end="", flush=True)
```

부산은 아름다운 해안선과 풍부한 문화로 유명한 도시입니다. 다음은 부산의 아름다운 관광지 5곳과 꼭 맛보아야 할 음식 5가지를 추천합니다.

관광지

1. ****해운대 해수욕장****
 - 부산을 대표하는 해변으로, 넓고 깨끗한 백사장과 푸른 바다가 매력적입니다. 주변에는 다양한 레스토랑과 카페가 있어 편리합니다.
2. ****광안리 해수욕장****
 - 광안대교의 아름다운 야경을 감상할 수 있는 곳으로, 밤에는 화려한 조명이 더해져 더욱 매력적입니다. 해변가에서 다양한 해양 스포츠도 즐길 수 있습니다.
3. ****태종대****
 - 자연 경관이 뛰어난 곳으로, 절벽과 바다가 어우러진 멋진 풍경을 감상할 수 있습니다. 태종대 유람선을 타고 주변 섬을 둘러보는 것도 추천합니다.
4. ****감천문화마을****
 - 알록달록한 집들이 언덕을 따라 늘어서 있는 독특한 마을입니다. 예술 작품과 벽화가 곳곳에 있어 사진 찍기 좋은 장소입니다.
5. ****자갈치 시장****
 - 부산의 대표적인 수산 시장으로, 신선한 해산물을 저렴하게 맛볼 수 있습니다. 활기찬 시장 분위기를 경험하며 다양한 해산물을 구경할 수 있습니다.

음식

1. ****돼지국밥****

~~ 이하 생략 ~~

LCEL

▶ LCEL (LangChain Expression Language)

- 워크플로우를 간단히 정의하고 실행할 수 있는 도메인 전용 언어로써 프롬프트, 출력 포맷 등을 언어 모델과 통합



```
# chain = prompt | model | output_parser
# 유닉스 파이프 연사자와 유사하게 서로 다른 구성 요소를 연결하고 한 구성 요소의 출력을 다음 구성 요소의 입력으로 전달

# 예제 코드
prompt = PromptTemplate.from_template("{topic}에 대해 쉽게 설명해주세요.")
model = ChatOpenAI()
output_parser = StrOutputParser()

chain = prompt | model | output_parser
```

Prompt

▶ 프롬프트 템플릿

- 사용자의 입력 변수를 활용하여 완전한 프롬프트 문자열을 만들 때 필요

```
from langchain_core.prompts import PromptTemplate

template = "{city}의 맛집 5곳을 알려주세요?" # 템플릿 정의

# from_template 메소드를 이용하여 PromptTemplate 객체 생성
prompt_template = PromptTemplate.from_template(template)

prompt = prompt_template.format(city="서울")
prompt
```

'서울의 맛집 5곳을 알려주세요?'

```
prompt = prompt_template.format(city="부산")
prompt
```

'부산의 맛집 5곳을 알려주세요?'

LLM 모델

▶ llm 모델 정의

- llm 객체를 생성하며 옵션 설정

```
from langchain_openai import ChatOpenAI

# 객체 생성
llm = ChatOpenAI(
    temperature=0.1,          # 창의성 (0.0 ~ 2.0)
    model_name="gpt-4o",      # 모델명
    max_tokens= 100,
    # api_key= "sk- 실습시 발급한 키 입력"
)

llm.invoke("하늘은 왜 파랳나요?")
# llm.invoke("하늘은 왜 파랳나요?").content # content만 출력
```

```
# model_name list = gpt-4, gpt-4o-mini
# ChatOpenAI 파라미터
llm = ChatOpenAI(
    model="gpt-4o",          # 모델
    temperature=0,          # 창의성
    max_tokens=None,        # 토큰 길이
    api_key= "...",         # api key
    # other params...
)
# invoke : 입력값을 전달하고 답변을 받아옴
```

AIMessage(content='하늘이 파랳게 보이는 이유는 주로 레일리 산란(Rayleigh scattering) 때문입니다. 태양빛은 여러 가지 색의 빛으로 구성되어 있으며, 이 빛은 서로 다른 파장을 가지고 있습니다. 대기 중의 분자와 입자들은 이 빛을 산란시키는데, 파장이 짧은 파란색 빛이 다른 색의 빛보다 더 많이 산란됩니다.\n\n태양빛이 대기를', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 100, 'prompt_tokens': 16, 'total_tokens': 116, 'prompt_tokens_details': {'cached_tokens': 0, 'audio_tokens': 0}, 'completion_tokens_details': {'reasoning_tokens': 0, 'audio_tokens': 0, 'accepted_prediction_tokens': 0, 'rejected_prediction_tokens': 0}}, 'model_name': 'gpt-4o-2024-08-06', 'system_fingerprint': 'fp_4691090a87', 'finish_reason': 'length', 'logprobs': None}, id='run-f475838c-9e7f-4f53-9653-48d2fcedb252-0', usage_metadata={'input_tokens': 16, 'output_tokens': 100, 'total_tokens': 116})

Parser

▶ Llm의 출력을 구조화된 형태로 변환하는 출력파서(Output Parser)

- StrOutputParser() 는 Llm의 메시지에서 .content 속성을 추출하여 최종 출력이 문자열 형식이 되도록 함

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# 객체 생성
llm = ChatOpenAI(
    model_name="gpt-4o",
    max_tokens= 100,
    api_key= "sk- 실습시 발급한 키 입력"
)

prompt = PromptTemplate.from_template("{topic}에 대해 쉽게 설명해줘")
output_parser = StrOutputParser()
chain = prompt | llm | output_parser    # chain 연결
input = {"topic" : "2차전지"}
chain.invoke(input)
```

'2차 전지는 흔히 충전할 수 있는 전지 또는 배터리로 이해하면 됩니다. 1차 전지와 달리, 2차 전지는 에너지를 사용한 후에도 충전 과정을 통해 여러 번 재사용할 수 있습니다. \n\n2차 전지의 기본 원리는 화학 반응을 통해 전기를 저장하고 필요할 때 그 전기를 다시 방출하는 것입니다. 충전 시에는 전기가 배터리에 저장되고, '

LCEL 예제

▶ 프롬프트를 활용하여 영어 회화 출력하기

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# 객체 생성
model = ChatOpenAI(
    model_name="gpt-4o",
    api_key= "sk- 실습시 발급한 키 입력"
)

template = """
당신은 영어 선생님입니다. 요청에 맞춰 [format]에 영어 회화를 작성해 주세요.

요청:
{question}

format:
- 영어 회화:
- 한글 해석:
"""
```

LCEL 예제

▶ 프롬프트를 활용하여 영어 회화 출력하기

```
prompt = PromptTemplate.from_template(template)    # 프롬프트 템플릿을 이용하여 프롬프트 생성

output_parser = StrOutputParser()                # 문자열 출력 파서를 초기화

chain = prompt | model | output_parser            # 체인 구성

scripts = chain.invoke({"question": "비행기표를 예매하고 싶어요"})    # 출력

print(scripts)
```

- 영어 회화:

- A: Hello! How can I help you today?
- B: Hi, I'd like to book a flight ticket, please.
- A: Sure! Where would you like to fly to?
- B: I want to fly to New York.
- A: Great! When are you planning to travel?
- B: I'm planning to travel on the 15th of next month.
- A: Okay, let me check the available flights for that date.
- B: Thank you!

- 한글 해석:

- A: 안녕하세요! 오늘 무엇을 도와드릴까요?
- B: 안녕하세요, 비행기표를 예매하고 싶어요.
- A: 물론이죠! 어디로 가시려고 하시나요?
- B: 뉴욕으로 가고 싶어요.
- A: 좋습니다! 언제 여행하실 계획이신가요?
- B: 다음 달 15일에 여행할 계획이에요.
- A: 알겠습니다, 그 날짜에 가능한 항공편을 확인해드릴게요.
- B: 감사합니다!

M3. 바이트코딩 활용 RAG 구현

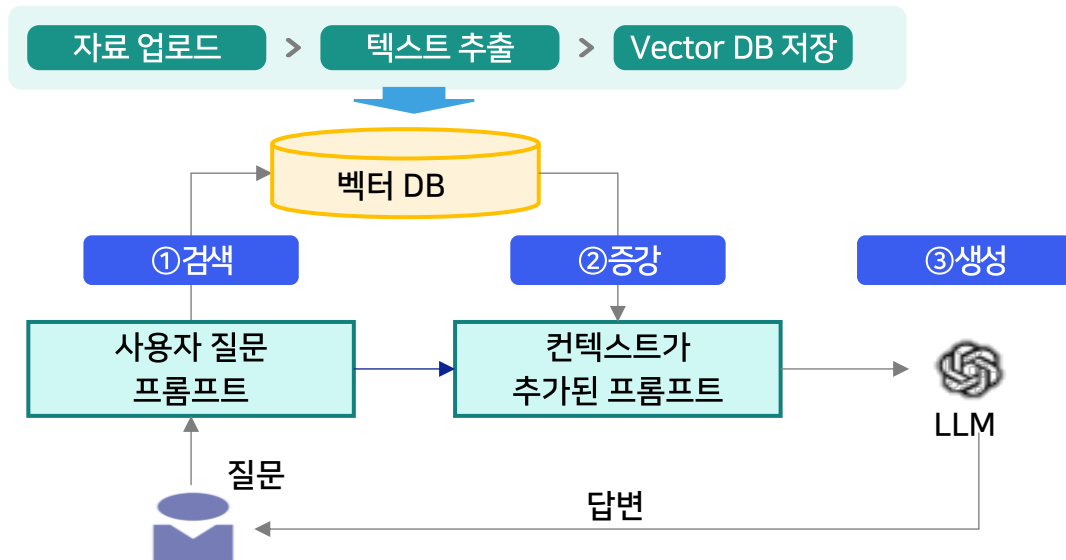
8. RAG 코드리뷰

Rag(Retrieval-Augmented Generation)

▶ Llm의 출력을 구조화된 형태로 변환하는 출력파서(Output Parser)

- StrOutputParser() 는 Llm의 메시지에서 .content 속성을 추출하여 최종 출력이 문자열 형식이 되도록 함

RAG 시스템



☞ 활용 : 지식기반 고객상담 챗봇, 맞춤형 튜터링
특정 법률, 규정, 판례 검색, 맞춤형 보고서 생성

- 검색단계 : 사용자의 질문에 대한 정보를 벡터DB에 저장한 외부 데이터 소스에서 검색
- 증강단계 : 검색된 데이터를 사용자가 입력한 질문과 결합하여 언어모델이 더 풍부하게 답변할 수 있도록 보조
- 생성단계 : 증강된 프롬프트를 바탕으로 사용자의 질문에 대한 최종 답변을 생성

※ 파이썬 패키지 : Langchain, Langgraph, Openai

RAG 장점

- 외부 데이터 소스에서 필요한 정보를 실시간으로 가져올 수 있음
- 일반적인 언어 모델보다 더 정확하고 신뢰할 수 있는 답변을 제공

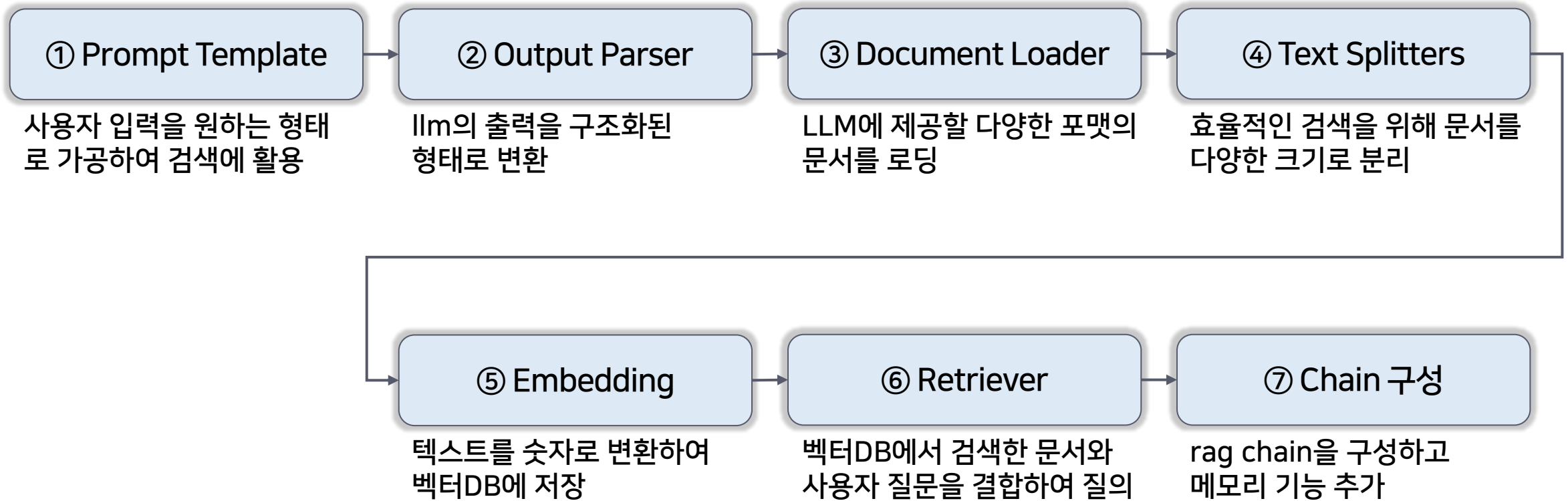
Rag 프레임워크

▶ LangChain과 LlamaIndex

항목	LangChain	LlamaIndex
출시년도	2022년 10월 공개 출시	2022년 11월경
개요	LLM(대형언어모델)을 활용한 애플리케이션 개발을 위해 "체인(Chains)", 도구(Agents), 데이터 + 모델 연계 등 전체 워크플로우를 지원하는 프레임워크	LLM에 외부 지식/데이터(문서, DB 등)를 연결하고, 인덱싱·검색·질의 인터페이스를 제공하여 RAG 및 지식 기반 대화/검색 애플리케이션을 쉽게 만들 수 있도록 돕는 데이터 프레임워크
주요 특징	- 문서 로딩, 청킹(chunking), 벡터 스토어, 검색(Retriever), LLM 호출 등을 모듈화하여 구성 가능 - Agent(에이전트) 패턴, 톨 통합, API 래퍼 등 다양한 구성 요소 제공	- 다양한 데이터 소스(파일, DB, API 등)를 손쉽게 연결 - 여러 인덱스(VectorStoreIndex, TreeIndex, KeywordIndex 등) 지원 - 자연어 질의를 통한 검색엔진 기능(Query Engine) 제공
장점	- LLM 기반 애플리케이션을 빠르게 구성할 수 있음 - 커뮤니티, 생태계가 활발해서 예제/튜토리얼이 많음 - 다양한 도구 및 서비스와 연계 가능하여 확장성 높음	- 데이터 중심의 RAG/검색 기반 애플리케이션에 특화되어 있음 - 복잡한 문서·DB·웹 등의 데이터를 쉽게 구조화 가능 - 인덱스 구조가 유연해 다양한 검색/질의 시나리오에 대응 가능
활용 예시	- 챗봇/가상비서 구성, 문서 요약 및 질의응답 시스템 - 에이전트형 자동화 워크플로우	- 내부 문서(예: PDF, 엑셀, 노션 등)를 LLM으로 질의하는 시스템 - 기업 지식베이스를 활용한 QA 시스템 - 멀티모달 데이터(이미지/텍스트 등)에 기반한 검색/분석앱

Rag 파이프라인

▶ Rag 파이프라인



① Prompt Template

- ▶ Prompt 입력을 위해 사용자 입력을 가공하는 PromptTemplate 과 시스템 메시지를 설정하여 LLM에 역할을 부여할 수 있는 ChatPromptTemplate을 활용

Prompt Template

프롬프트
[매개변수]

ChatPromptTemplate

SystemMessage
[매개변수]

HumanMessage
[매개변수]

AIMessage
[매개변수]

■ PromptTemplate

: PromptTemplate은 수행해야 할 작업을 설명하는 문장들을 모두 템플릿으로 선언하고, 사용자가 입력할 때마다 다른 값이 들어올 부분은 괄호를 감싸서 매개변수로 입력

■ ChatPromptTemplate

: SystemMessage에 LLM이 사용자와 상호작용할 때 필요한 내용을 미리 설정할 수 있으며 HumanMessage, AIMessage에서는 LLM이 사용자와 나눈 대화에 맥락을 더해 줄 수 있음

① Prompt Template

▶ PromptTemplate 예제

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate # PromptTemplate class 임포트
from langchain_core.output_parsers import StrOutputParser
llm = ChatOpenAI(
    model_name="gpt-4o",
    api_key= "sk- 실습시 발급한 키 입력"
)
prompt= (
    PromptTemplate.from_template(
        """
        너는 요리사야. 내가 가진 재료들을 갖고 만들 수 있는 요리를 {개수}추천하고,
        그 요리의 레시피를 설명해줘. 내가 가진 재료는 아래와 같아.
        <재료>
        {재료}
        """
    )
)
output_parser = StrOutputParser() # Outparser 개체 생성
chain = prompt | llm | output_parser # chain 생성
print(chain.invoke({"개수":"3", "재료" : "사과,양파,햄"})) # 매개변수에 딕셔너리 자료형으로 데이터 입력

```

① Prompt Template

▶ PromptTemplate 예제

재료인 사과, 양파, 햄을 활용하여 만들 수 있는 요리를 세 가지 추천하고, 각각의 레시피를 제시해 드리겠습니다.

1. **사과 햄 샐러드**

- ****재료****: 사과 1개, 양파 1/4개, 햄 100g, 샐러드 채소(상추, 루꼴라 등), 마요네즈 2큰술, 레몬즙 1큰술, 소금과 후추 약간
- ****레시피****:
 1. 사과는 껍질을 벗기고 얇게 슬라이스합니다.
 2. 양파는 얇게 채썰고 찬물에 잠시 담갔다가 물기를 제거합니다.
 3. 햄은 적당한 크기로 자릅니다.
 4. 샐러드 채소를 손으로 잘게 뜯어 준비합니다.
 5. 큰 볼에 사과, 양파, 햄, 샐러드 채소를 넣고 섞습니다.
 6. 마요네즈와 레몬즙을 넣고 잘 버무린 뒤, 소금과 후추로 간을 맞춥니다.

2. **햄과 사과를 곁들인 양파 스프**

- ****재료****: 양파 2개, 사과 1/2개, 햄 100g, 버터 1큰술, 치킨 스톡 500ml, 소금과 후추 약간
- ****레시피****:
 1. 양파는 얇게 채썰고, 사과는 깍둑썰기 합니다.
 2. 냄비에 버터를 녹이고 양파를 넣어 중약불에서 갈색이 나도록 볶습니다.
 3. 양파가 갈색이 되면 햄을 넣고 조금 더 볶습니다.
 4. 사과를 넣고 치킨 스톡을 부어 끓입니다.
 5. 끓기 시작하면 불을 줄이고 약 15분간 더 끓입니다.
 6. 소금과 후추로 간을 맞추고 완성합니다.

② Output Parser

▶ Output Parser는 LLM의 답변을 받아 다양한 종류의 출력 데이터로 변환하는 역할 수행

StrOutputParser

- LLM의 출력 중 .content 속성을 추출하여 최종 문자열만 출력

PydanticOutputParser

- LLM의 출력을 좀 더 구조화된 정보로 변환

CommaSeparatedListOutputParser

- 사용자가 입력한 데이터나 요청한 정보를 쉼표로 구분

StructuredOutputParser

- LLM의 출력을 딕셔너리 유형으로 변환하여 반환

JsonOutputParser

- 사용자가 원하는 JSON 스키마로 출력

PandasDataFrameOutputParser

- 사용자가 지정한 Pandas DataFrame 형태로 출력

DatetimeOutputParser

- LLM의 출력을 datetime 형식으로 출력

② Output Parser

▶ CommaSeparatedListOutputParser 예제 (사용자가 지정한 숫자만큼 대상을 CSV 형태로 출력)

```

from langchain_core.output_parsers import CommaSeparatedListOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

output_parser = CommaSeparatedListOutputParser()           # 콤마로 구분된 리스트 출력 파서 초기화

format_instructions = output_parser.get_format_instructions() # 출력 형식 지침 가져오기
prompt = PromptTemplate(                                   # 프롬프트 템플릿 설정
    template="List five {subject}.\n{format_instructions}", # 주제에 대한 다섯 가지를 나열하라는 템플릿
    input_variables=["subject"],                          # 입력 변수로 'subject' 사용
    partial_variables={"format_instructions": format_instructions}, # 부분 변수로 형식 지침 사용
)

# ChatOpenAI 모델 초기화
model = ChatOpenAI(
    model_name="gpt-4o",
    api_key= "sk- 실습시 발급한 키 입력"
)

```

② Output Parser

▶ CommaSeparatedListOutputParser 예제 (사용자가 지정한 숫자만큼 대상을 CSV 형태로 출력)

```
chain = prompt | model | output_parser
```

프롬프트, 모델, 출력 파서를 연결하여 체인 생성

```
chain.invoke({"subject": "공포 영화"})
```

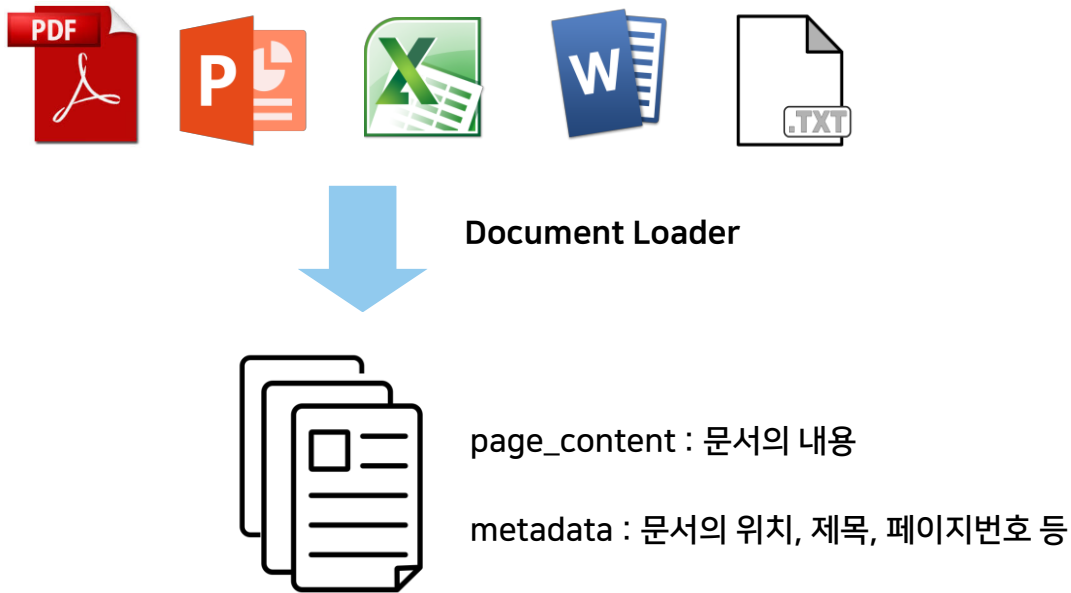
"공포 영화"에 대한 chain 호출 실행

```
['곡성', '겟 아웃', '사일런스 오브 더 램스', '셰이프 오브 워터', '더 빅터']
```

- CommaSeparatedListOutputParser의 `get_format_instructions`를 프롬프트 템플릿의 `partial_variables` 지정해야 프롬프트 템플릿안에 심표로 구분된 텍스트 형식의 답변을 하도록 프롬프트가 변경 됨
 - chain을 구성할 때 CommaSeparatedListOutputParser를 `output_parser`로 연결하면 리스트 형식의 답변을 출력할 수 있음
- ※ `format_instructions` 내용 : "Your response should be a list of comma separated values, eg: 'foo, bar, baz'"

③ Document Loader

▶ 다양한 유형의 문서를 불러와 rag에서 사용할 수 있도록 하나의 통일된 형태로 관리



Document Loader의 필요성

- rag의 답변 근거
: rag에서 LLM은 근거가 명확한 답변을 생성해야 하는데,
이때 Document Loader로 불러온 문서와 메타데이터를 활용
- 메타데이터 기반 필터링
: 업로드된 다양한 문서 중 특정 문서의 내용만 참고하려 할 때
메타 데이터를 활용하여 필요한 문서를 빠르게 필터링

③ Document Loader

▶ PDF 파일 불러오기 (PyPDFLoader)

```
from langchain_community.document_loaders import PyPDFLoader # PyPDFLoader 불러오기
file = "../실습파일/2022년 버전 선재 카탈로그_국문_최종.pdf"
loader = PyPDFLoader(file) # PDF파일 불러올 객체 PyPDFLoader 선언

pages = loader.load_and_split() # PDF파일 로드 및 페이지별로 자르기
print(pages[5].page_content) # 5페이지 내용 출력
```

```
10
POSCO Wire RodPOSCO Wire Rod 11
주요 용도
연강선재
C 성분 0.22% 이하의 저탄소강으로서 각종 아연도금
철선 등의 제작에 사용됩니다.
~~~
```

```
print(pages[0].page_content) # 0페이지의 이미지 글자를 불러오지 못함
```

```
Wire Rod선재
```

③ Document Loader

▶ CSV 파일 불러오기 (CSVLoader)

```
from langchain_community.document_loaders.csv_loader import CSVLoader # CSVLoader 불러오기

loader = CSVLoader(file_path="../실습파일/fitness.csv")

data = loader.load() # 파일 업로드

data[0].page_content # 0번째 출력

'Id: 1\nSepalLengthCm: 5.1\nSepalWidthCm: 3.5\nPetalLengthCm: 1.4\nPetalWidthCm: 0.2\nSpecies: Iris-setosa'
```

CSVLoader로 파일을 읽어 오면 행별로 document 객체를 구성하고 page_content와 metadata를 제공함

③ Document Loader

▶ TXT 파일 불러오기 (TextLoader)

```
from langchain_community.document_loaders import TextLoader # TextLoader 불러오기

loader = TextLoader("../실습파일/news.txt", encoding= "utf-8")

data = loader.load() # 파일 업로드

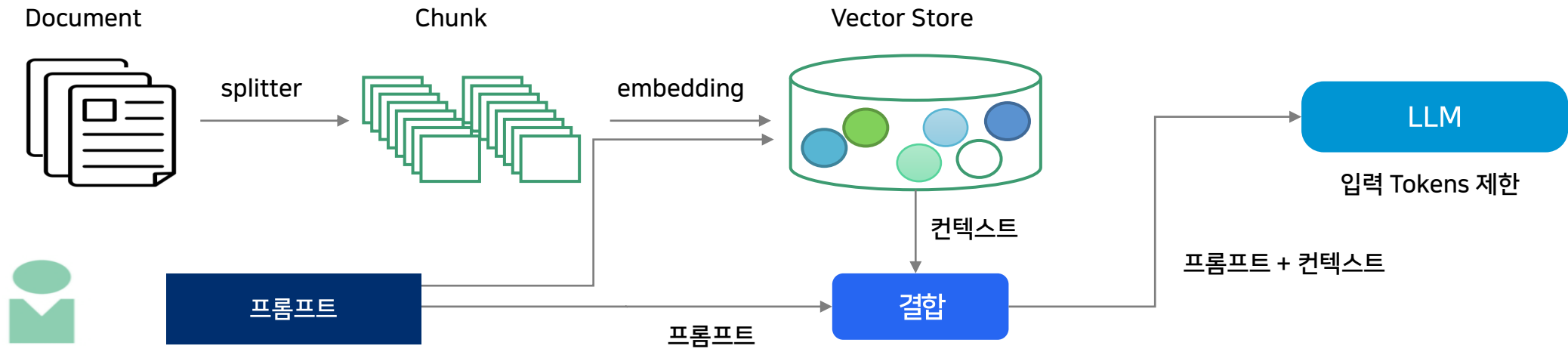
data[0].page_content # 0번째 출력
```

'포스코DX, '전국 청소년 AI창의 경진대회' 개최해 미래IT인재 육성\n전국 청소년 대상 AI 활용한 산업분야 개선 아이디어, 프로그램 등 공모\nI 본선 진출 6개 팀은 AI 전문가의 멘토링 통해 역량향상 기회 제공 받아...\n\n포스코DX(대표 심민석)가 AI를 기반으로 하는 중고교 학생들의 창의적인 개선 아이디어, 프로그램 등을 발굴해 시상하는 '전국 청소년 AI 창의 경진대회('2025 AI Youth Challenge)'를 개최하며 미래 IT 인재 육성에 나서고 있다.\n\n올해로 6회째를 맞이하는 '전국 청소년 AI 창의 경진대회'는 포스코DX와 한국인공지능산업협회가 공동 주최하고, 교육부가 후원하는 전국 최대 규모의 행사다. 올해는 『인공지능과 함께하는 인간 중심의 AI』라는 주제로 열린다 ~~~'

txt 파일은 하나의 행에 모든 내용이 업로드됨

④ Text Splitters

▶ 업로드한 문서를 일정 크기로 잘라내어 질의시 사용할 컨텍스트를 LLM의 최대 입력 길이내로 유지



④ Text Splitters

▶ 지정된 Chunk 길이를 유지할 수 있는 RecursiveCharacterTextSplitter

```

from langchain_community.document_loaders import PyPDFLoader # PyPDFLoader 불러오기
file = "../실습파일/이슈리포트_2024-09호.pdf"
loader = PyPDFLoader(file) # PDF파일 불러올 객체 PyPDFLoader 선언
pages = loader.load() # PDF파일 로드

from langchain_text_splitters import RecursiveCharacterTextSplitter # RecursiveCharacterTextSplitter 불러오기
text_splitter = RecursiveCharacterTextSplitter(
    separators=["\nn","\n"," ",""], # separator 리스트 설정
    chunk_size=500, # 텍스트 분할 글자수, 청크의 최대크기
    chunk_overlap=100, # 청크간 겹치는 글자수로 맥락 유지에 필요
    length_function=len # 텍스트의 길이를 계산하는 함수 지정
)

texts = text_splitter.split_documents(pages) # 텍스트 분할

len(texts) # 청크 수 확인

107

print(texts[29]) # 29번째 청크 확인

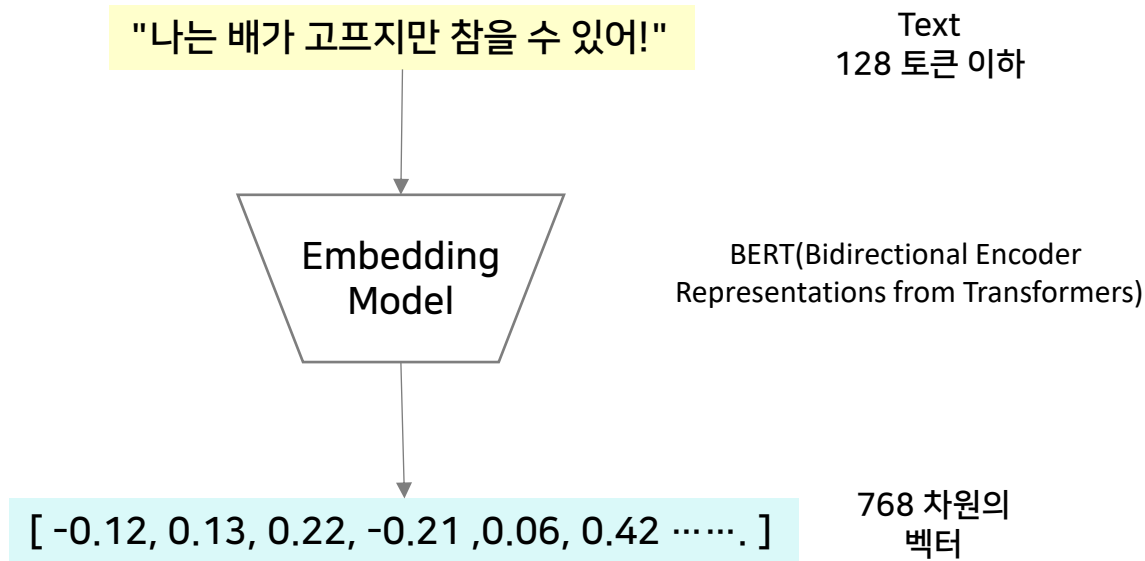
```

⑤ Embedding

▶ 문서 청크 텍스트를 숫자 벡터로 변환하여 컴퓨터가 이해하고 처리할 수 있도록 하는 것

- 문서 청크를 벡터 DB에 저장할 때, 사용자의 질문에 답할 근거를 벡터 DB에서 검색할 때 활용

임베딩 모델의 동작 과정 (BERT 모델 예시)



임베딩 모델 종류

- Open source Model
 - 무료지만 사용방법이 어려움, 로컬 실행 가능하며 보안 우수
 - 대표 모델 (허깅페이스)
 - . bge-m3
 - . multilingual-e5-large
- Closed source Model
 - 사용이 편리하나 비용 발생, API 통신으로 보안 취약
 - 대표모델 (OpenAI)
 - . text-embedding-ada-002
 - . Text-embedding-3-small

⑤ Embedding

▶ 간단한 문장 임베딩 하기

```
import os
from langchain_openai import OpenAIEmbeddings

embeddings_model = OpenAIEmbeddings(
    model = "text-embedding-3-small",
    #api_key= "sk- "
)
embeddings = embeddings_model.embed_documents(
    [
        "Hi there!",
        "Oh, hello!",
        "What's your name?",
        "My friends call me World",
        "Hello World!"
    ]
)
len(embeddings), len(embeddings[0])
```

임베딩 모델 불러오기

임베딩 객체 생성

text-embedding-3-small 임베딩 모델은
한 문장을 1536개의 숫자 벡터로 변환

⑤ Embedding

▶ 임베딩 완료 후 생성된 벡터를 벡터 DB에 저장

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma

embeddings_model = OpenAIEmbeddings(model = "text-embedding-3-small") # 임베딩 객체 생성
loader = PyPDFLoader("../실습파일/2022년 버전 선재 카탈로그_국문_최종.pdf") # PDF 문서 로드
pages = loader.load()

text_splitter = RecursiveCharacterTextSplitter( # PDF 문서를 여러 청크로 분할
    chunk_size=500,
    chunk_overlap=100 )

texts = text_splitter.split_documents(pages)

# OpenAI 임베딩 모델로 청크들을 임베딩 하면서 벡터 DB에 저장
db = Chroma.from_documents(texts, embeddings_model)
```

⑤ Embedding

▶ 임베딩 DB에서 저장된 문서 검색해 보기

```
query = "쾌삭강"
docs = db.similarity_search(query)      # 유사도 검색을 통해 db에서 query와 유사한 chunk 추출
print(docs[0])                        # 첫번째 문서 출력, 총 문서수는 len(docs)로 확인
```

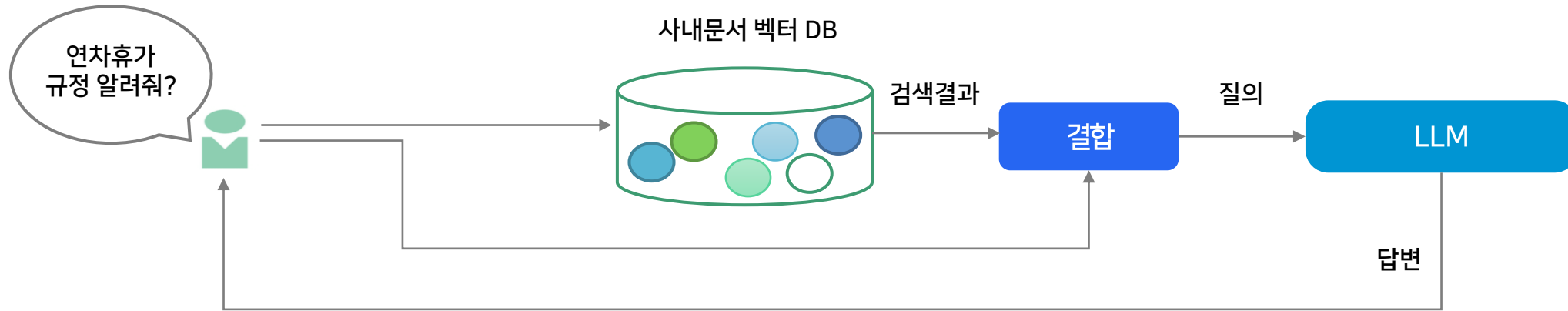
page_content='쾌삭강
 쾌삭강은 일반강에 MnS 개재물 또는 Pb, Bi 등의 피삭성
 원소를 단독 및 복합으로 첨가하여 피삭성을 향상시킨
 제품입니다. 절삭공구의 마모 감소로 공구수명이 향상되며,
 ~~~~~~

```
# 데이터베이스를 하드디스크에 저장
db1 = Chroma.from_documents(texts, embeddings_model, persist_directory=".\\database")

# 데이터베이스를 하드디스크에서 불러오기기
db2 = Chroma(persist_directory=".\\database", embedding_function=embeddings_model )
result = db2.similarity_search("쾌삭강")
print(result[0])                # 첫번째 문서 출력, 총 문서수는 len(docs)로 확인
```

## ⑥ retriever

▶ retriever란 벡터 DB에서 정보를 검색하는 방법으로 LLM이 좀 더 나은 답변을 생성하도록 지원



# 벡터 DB에서 검색시 발생 가능한 상황

- 벡터 DB에서 얼마나 많은 문서를 가져올 것인가? → 유사도 순위기준 등을 고려 적절한 문서를 가져와야함
- 프롬프트와 유사한 내용이 벡터 DB에 없다면? → 사용자의 질의를 다양한 방식으로 확장하여 검색

## ⑥ retriever

### ▶ 벡터 DB 기반 retriever

```

from langchain_community.document_loaders import PyPDFLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma

# 임베딩 모델 API 호출
embeddings_model = OpenAIEmbeddings(model = "text-embedding-3-small")
loader = PyPDFLoader("../실습파일/2022년 PosMAC1.5 카탈로그_강건재산업용_국문_최종.pdf")
pages = loader.load()

# 임베딩 객체 생성
# PDF 문서 로드

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
texts = text_splitter.split_documents(pages)

# 문서분할

# OpenAI 임베딩 모델로 청크들을 임베딩 하면서 벡터 DB에 저장
db = Chroma.from_documents(texts, embeddings_model)

retriever = db.as_retriever()
retriever.invoke("내식성")

# retriever 객체 생성
# 내식성으로 문서 검색(기본 4건 출력)

```

```

[Document(metadata={'page': 0, 'source': '.\\data\\PosMAC1.5 카탈로그_강건재산업용_국문_최종.pdf'}, page_content='PosMAC\n®\n1.5\nPOSCO Magnesium Aluminium alloy Coating product\n포스맥1.5'),
 Document(metadata={'page': 1, 'source': '.\\data\\PosMAC1.5 카탈로그_강건재산업용_국문_최종.pdf'}, page_content='포스맥1.5\nPosMAC®1.5은 포스
~~~~~

```

## ⑥ retriever

### ▶ 벡터 DB 기반 retriever

```
result_score = db.similarity_search_with_relevance_scores("posmac") # 값이 높을 수록 유사성 높음
print(result_score[0][1], result_score[1][1], result_score[2][1], result_score[3][1])
```

```
0.2342658567167376 0.02343771098378522 0.014769031475761585 0.003140547384211878
```

```
retriever = db.as_retriever(search_kwargs={"k":6}) # 유사도 높은 문서 6개 선택
retriever.invoke("갈바륨")
```

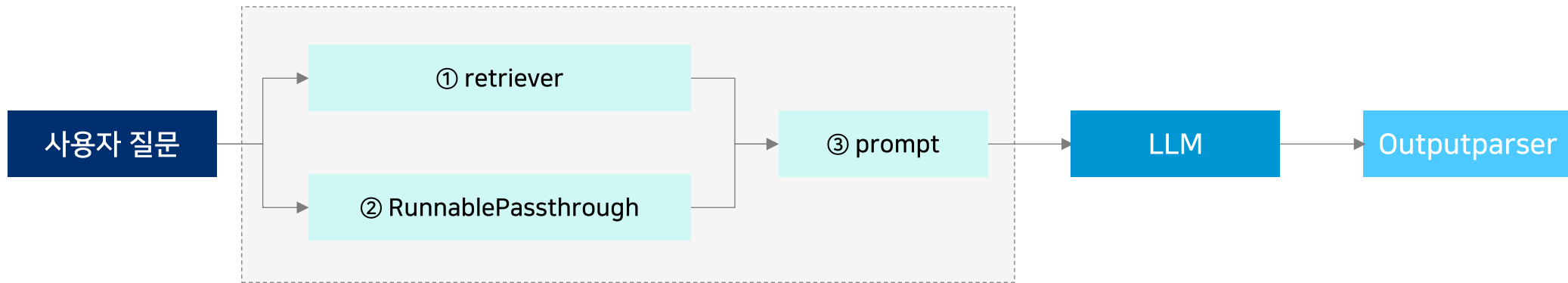
```
[Document(metadata={'page': 4, 'source': '.\\data\\PosMAC1.5 카탈로그_강건재산업용_국문_최종.pdf'}, page_content='후처리\n도금량 양면합
\n구분 GI 갈바륨\nCr6+유기피막 Cr6+유기피막 Cr6+유기피막 Cr3+ Cr6+\n500hr\n1000hr\n1500hr\n2000hr\nPosMAC1.5\n180g/m 2 90g/m 2 80g/m 2
120g/m 2 120g/m 2 120g/m 2\nCr3+\n갈바륨과의 평판 내식성 비교 (Edge부 노출후 시험)\n· PosMAC1.5는 단면부 내식성이 우수하여 Edge부가 노출된 상
태에서도 우수한 평판부 내식성을 나타냅니다.\n· 갈바륨은 단면부 내식성이 열위하여 Edge부가 노출된 상태에서 염수분무시험을 실시하면 단면부로부터 적청이
평면부로 빠르게 전이됩니다.\n테스트 방법 : 염수분무시험 (SST, Salt Spray Test) [ISO 9227, JIS Z2371, ASTM B117] 5%NaCl, 35 °C\n후처리\n도
금량 양면합\n구분 갈바륨\nCr6+유기피막 Cr6+유기피막 Cr6+유기피막 Cr3+ Cr6+\n500hr\n750hr\nPosMAC1.5'),
```

```
~~~~~
```

## ⑦ Chain 구성

### ▶ Rag chain 구성

- 사용자 질문과 retriever로 검색한 context를 prompt로 결합하여 LLM에 입력하는 Chain 구성



# 벡터 DB에서 검색시 발생 가능한 상황

- 벡터 DB에서 얼마나 많은 문서를 가져올 것인가? → 유사도 순위기준 등을 고려 적절한 문서를 가져와야함
- 프롬프트와 유사한 내용이 벡터 DB에 없다면? → 사용자의 질의를 다양한 방식으로 확장하여 검색

## ⑦ Chain 구성

### ▶ Rag chain 구성

# 모듈 불러오기기

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import PromptTemplate
```

# 헌법 PDF 파일 로드

```
loader = PyPDFLoader(r"..../실습파일/대한민국헌법(헌법)(제00010호)(19880225).pdf")
pages = loader.load_and_split()
```

# PDF 파일을 500자 청크로 분할

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0)
docs = text_splitter.split_documents(pages)
```

# ChromaDB에 청크들을 벡터 임베딩으로 저장

```
db = Chroma.from_documents(docs, OpenAIEmbeddings(model = "text-embedding-3-small",
                                                    api_key= "sk-"))
```

## ⑦ Chain 구성

### ▶ Rag chain 구성

```
# retriever 생성
retriever = db.as_retriever()
# prompt 생성
prompt = PromptTemplate.from_template(
    """
    당신은 질문-답변 작업을 위한 어시스턴트입니다.
    다음에 제공된 맥락을 사용하여 질문에 답하세요.
    만약 답을 모른다면, 그냥 모른다고 말하세요.
    한국어로 답변하세요.
    # question:
    {question}
    # context:
    {context}
    # answer: """
)

# LLM 생성
llm = ChatOpenAI(model_name="gpt-4o",
                  #api_key= "sk- ",
                  temperature=0)
```

## ⑦ Chain 구성

### ▶ Rag chain 구성

```
# Chain 생성
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

```
# Chain 시각화 출력
chain.get_graph().print_ascii()
```

```
# 문서에 대한 질의를 입력하고, 답변을 출력
question = "국회의원의 임기는 몇 년인가?"
response = chain.invoke(question)
print(response)
```

국회의원의 임기는 4년입니다.

```
# 문서에 없는 질문을 했을 때 답변
question = "미국 국회의원의 임기는 몇년인가?"; response = chain.invoke(question); print(response)
-----
미국 국회의원의 임기에 대한 정보는 제공된 맥락에 포함되어 있지 않습니다. 따라서 답변을 드릴 수 없습니다.
```

# M3. 바이트코딩 활용 RAG 구현

## 9. RAG 구현

# 실습 1 : General Rag 구현

## ▶ 개요 : PDF 파일을 업로드하면 벡터 DB를 구성하고, 사용자의 질문에 대해 Chunk를 찾아서 답변 생성

- GPT와 유사한 웹으로 구성, 텍스트는 500글자로 분할, 임베딩 모델 text-embedding-3-small , llm gpt-4o-mini
- chunk를 찾지 못하면 '모르겠습니다'라고 답변, 답변과 관련한 chunk 정보 출력, 대화 메모리 기능 사용

### RAG Assistant

파일 업로드

2022년 버전 냉연 카탈로그\_국문\_최종.pdf

대화 초기화

메시지를 입력하세요...

1개의 파일이 성공적으로 업로드되었습니다.

냉연에 대해 설명해줘

냉연은 금속을 상온에서 압연하여 원하는 두께와 형태로 가공하는 과정입니다. 이 과정은 일반적으로 40%에서 90%의 압하율로 진행되며, 자동두께제어 및 자동형상제어와 같은 첨단 제어 기기를 사용하여 정밀한 품질을 유지합니다. 냉연 과정에서는 산세와 전해청정 등의 공정을 통해 표면 결함을 제거하고, 소둔 과정을 통해 강도를 높이는 등의 추가적인 처리가 이루어집니다. 이러한 과정을 통해 생산된 냉연강판은 다양한 용도로 사용됩니다.

참고 자료

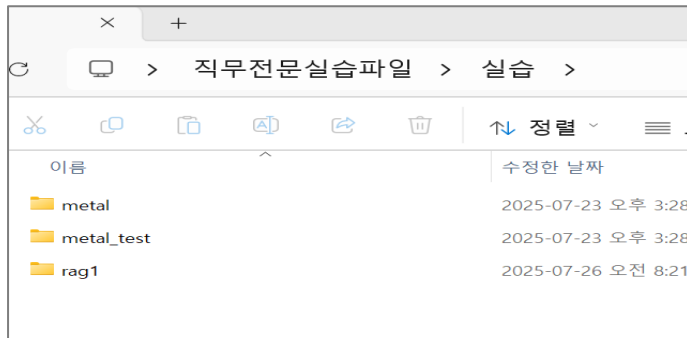
- **페이지 1** - 2022\_...pdf: Cold Rolled Steel 냉연강판
- **페이지 11** - 2022\_...pdf: 전기집진기 (EP) 황산/염산 내식성 ANCOR-CS 탈황설비 Duct 기타원소 첨가 보일러 황산내식강 적용 황산/염산 복합내식강 적용 배연 탈황 설비 전기 집진기 배연 탈황설비 Air Pre- Heater GGH (입구) GGH (출구) Temp. range : 50~450°C SOx level : 25~1,000ppm I/D Fan Stack F/D F...
- **페이지 10** - 2022\_...pdf: 18 POSCO Cold Rolled Steel POSCO Cold Rolled Steel 19 용접봉용 냉연강판 법랑용 냉연강판 일반사항 FCW(Flux Cored Wire) 용접봉 : 냉연강판을 소폭으로 Slitting한 후, Roll Forming 하면서 Flux를 충전하고, Die 신선하여 Wire로 만든 제품입니다. 화학성분 CSP2-WB CS...
- **페이지 6** - 2022\_...pdf: 10 POSCO Cold Rolled Steel POSCO Cold Rolled Steel 11 제조공정 & 설비(냉연) 포스코는 최신설비와 기술로 제품을 생산하여 고객의 요구에 맞는 제품의 품질을 생산하기 위하여 전 공정을 컴퓨터로 제어하며 정밀도와 품질특성을 향상시킬 수 있는 완전자동화 설비를 갖추고 있습니다. 냉간압연 용도에 맞는 두께와 재질확보를 ...

174

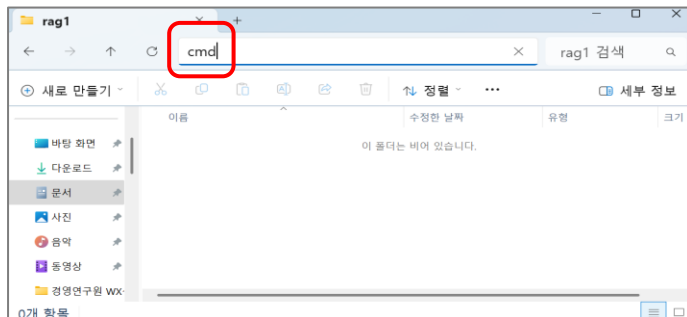
# 실습 1 : General Rag 구현

## ▶ 개발 환경 설정

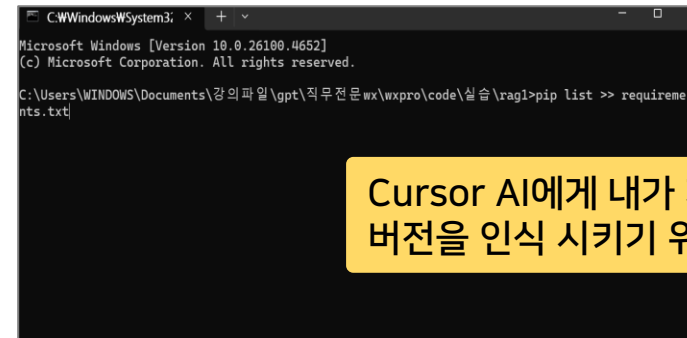
### ① 탐색기 : 실습 폴더 아래 'rag1' 폴더 생성



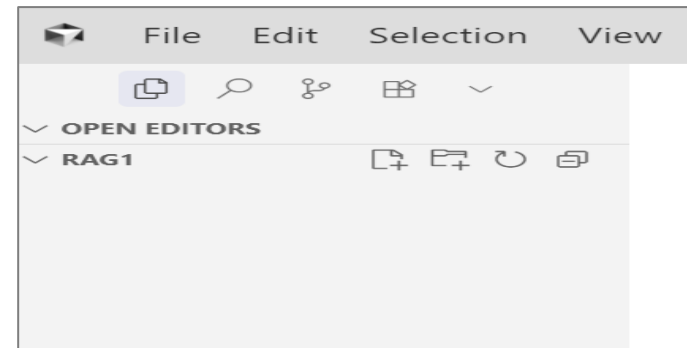
### ② 탐색기로 실습 폴더로 이동 → 주소창에 "cmd" 입력



### ③ "pip list >> requirements.txt" 입력



### ④ cursor AI : file → Open folder → "rag1" 폴더 클릭



# 실습 1 : General Rag 구현

## ⑤ 대화창에 프롬프트 입력

The screenshot shows the Cursor IDE with a Python prompt for building a RAG application. The prompt is highlighted with a blue box and a green arrow points to the 'New Chat' button in the chat interface on the right.

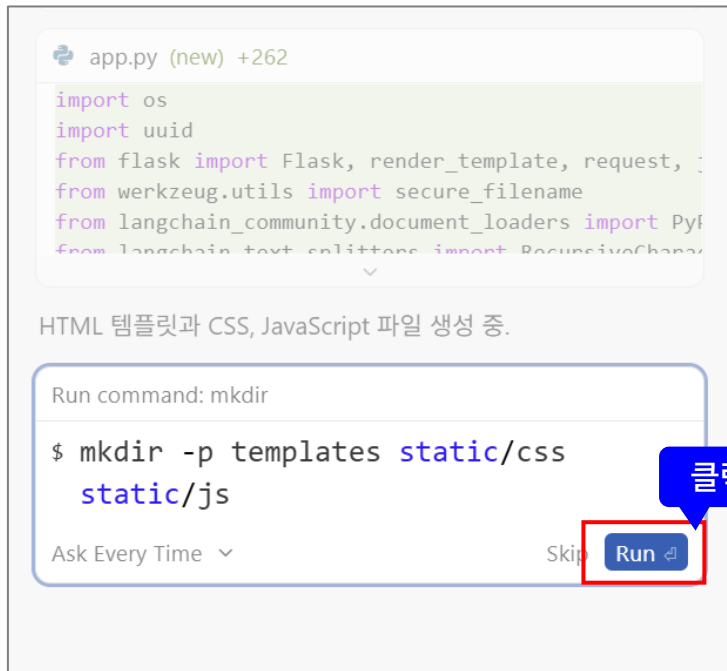
**Prompt**

- 파이썬 langchain 패키지를 이용해서 rag를 만들어줘
- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할거야
- llm은 gpt-4o-mini 모델을 사용해야 해
- api-key는 나의 시스템 환경에 있으니 설정 안 해도 돼
- 이 파일을 PyPDFLoader 로 로딩해줘
- RecursiveCharacterTextSplitter로 500글자 분할, 100글자 오버랩
- text-embedding-3-small로 임베딩하고 chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- 답변 출력 후, 참고한 chunk 내용 및 페이지번호를 요약하여 출력해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다"라고 대답해야 해
- 대화 메모리 기능을 사용해주

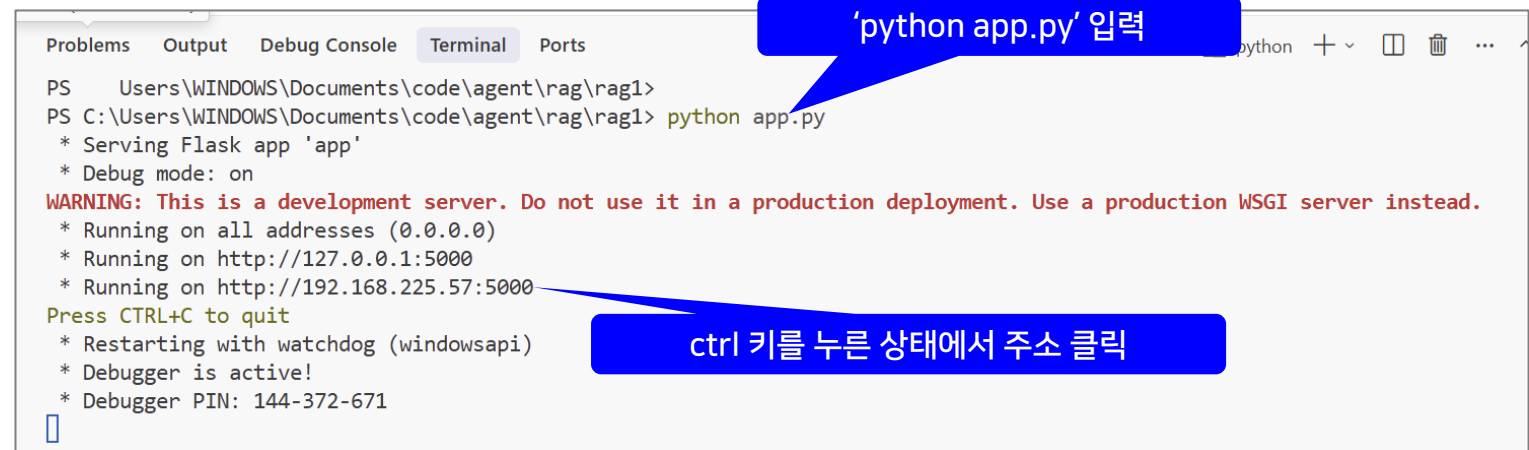
The chat interface on the right shows a 'New Chat' button and a 'Composer 1' dropdown menu. A green arrow points from the 'New Chat' button to the 'Composer 1' dropdown menu.

# 실습 1 : General Rag 구현

## ⑥ 폴더생성, 패키지 설치 등 승인



## ⑦ 코딩이 완료되면 터미널 창에서 웹 실행



# 실습 1 : General Rag 구현

## ⑧ 웹 동작 상태 확인, 에러 발생시 수정

The screenshot shows a web browser window with the title 'RAG Chat Assistant'. The address bar shows '127.0.0.1:5000'. The page has a light blue header with the title 'RAG Assistant'. On the left side, there is a sidebar with a 'PDF 파일 업로드' button and a list of uploaded files, including '2022년 버전 선재 카탈로그\_국문\_최종.pdf'. Below this is a '대화 기록 초기화' button. The main content area displays a chat response to a query about piano wire. The response text is: '피아노선재는 고강도 Bead Wire, PC강선, 교량케이블 등에 주로 사용되는 고탄소 청정강입니다. 이 선재는 극세선 신선성, 고강도, 내피로성이 우수한 특성을 가지고 있습니다. 주요 규격으로는 POSMICRO62, POSCABLE82, POSCABLE86, POSCABLE90, POSCABLE92, POSCABLE98 등이 있으며, JIS 규격으로는 SWRS62A/B부터 SWRS92A 까지 다양합니다. 피아노선재는 주로 스프링강과 같은 고탄소강 제품으로, 높은 강도와 신선 가공성을 필요로 하는 용도에 적합합니다.' Below the response is a '참고 자료:' section with three references. At the bottom, there is a text input field with the placeholder '메시지를 입력하세요' and a '전송' button.

**2022년 버전 선재 카탈로그\_국문\_최종.pdf** 업로드

**PDF 파일 업로드**

2022년 버전 선재 카탈로그\_국문\_최종.pdf

대화 기록 초기화

피아노선재에 대해 알려줘

피아노선재는 고강도 Bead Wire, PC강선, 교량케이블 등에 주로 사용되는 고탄소 청정강입니다. 이 선재는 극세선 신선성, 고강도, 내피로성이 우수한 특성을 가지고 있습니다. 주요 규격으로는 POSMICRO62, POSCABLE82, POSCABLE86, POSCABLE90, POSCABLE92, POSCABLE98 등이 있으며, JIS 규격으로는 SWRS62A/B부터 SWRS92A 까지 다양합니다. 피아노선재는 주로 스프링강과 같은 고탄소강 제품으로, 높은 강도와 신선 가공성을 필요로 하는 용도에 적합합니다.

**참고 자료:**

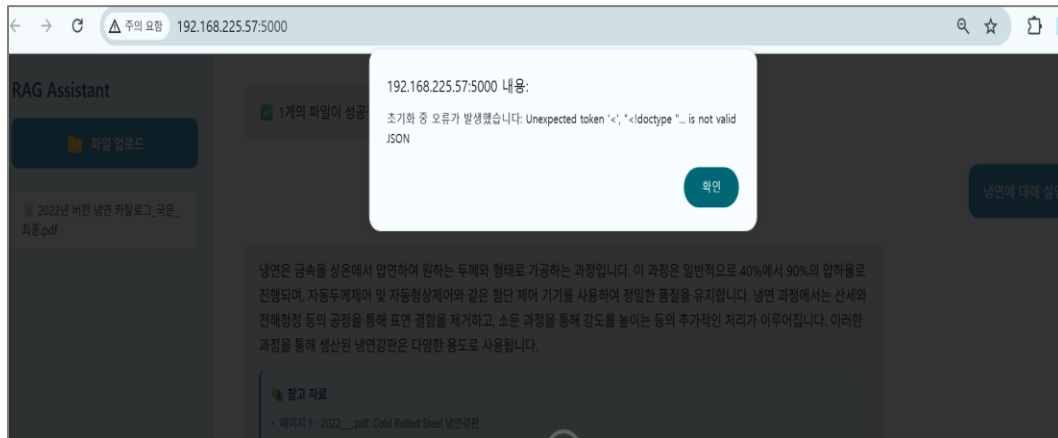
- 페이지 13: ■ Micro Cable용 피아노 선재 ■ 케이블용 선재 ■ 타이어코드용 선재 규격기호 화학성분(%) C Si Mn P S Cu Cr POSMICRO62 0.58~0.68 0.10~0.30 0.35~0.65 0.020 Max. 0.020 Max. 0.20 Max. 0.20 Max. 규격기호 화학성분(%) C Si Mn P S Cu Cr POSCABLE82 ...
- 페이지 10: 16 POSCO Wire Rod POSCO Wire Rod 17 제조가능 규격 피아노 선재 스프링강 쾌삭강 연자성 선재 ■ JIS ■ JIS ■ SAE/DIN 규격기호 화학성분(%) C Si Mn P S Cu Cr SWRS62A 0.60~0.65 0.12~0.32 0.30~0.60 0.025 Max. 0.025 Max. 0.20 Max. - SWRS62B ...
- 페이지 6: 8 POSCO Wire Rod POSCO Wire Rod 9 선재공장 소개 시험 항목 우수한 품질의 제품을 생산하기 위해 최선에 설비구축 및 기술 개발 포스코의 선재제품은 산업의 가장 기초적인 소재로부터 공장의 산업설비까지 다양한 분야에 사용되어 그 품질을 인정받고 있습니다. 최선에 설비와 자동화된 시스템으로 보다 우수한 품질의 선재제품을 개발하고 생산하...
- 페이지 7: 10 POSCO Wire Rod POSCO Wire Rod 11 주요 용도 연강선재 C 성분 0.22% 이하의 저탄소강으로서 각종 아연도금 철선 등의 제작에 사용됩니다. ■ POSCO POSFIS5M1, 6M1, 6B ■ JIS SWRM6~22 경강선재 고탄소강 제품으로서 고강도 유지와 신선 가공성 확보를 위해 Fine Pearlite 조직제어를 필요로...

메시지를 입력하세요 전송

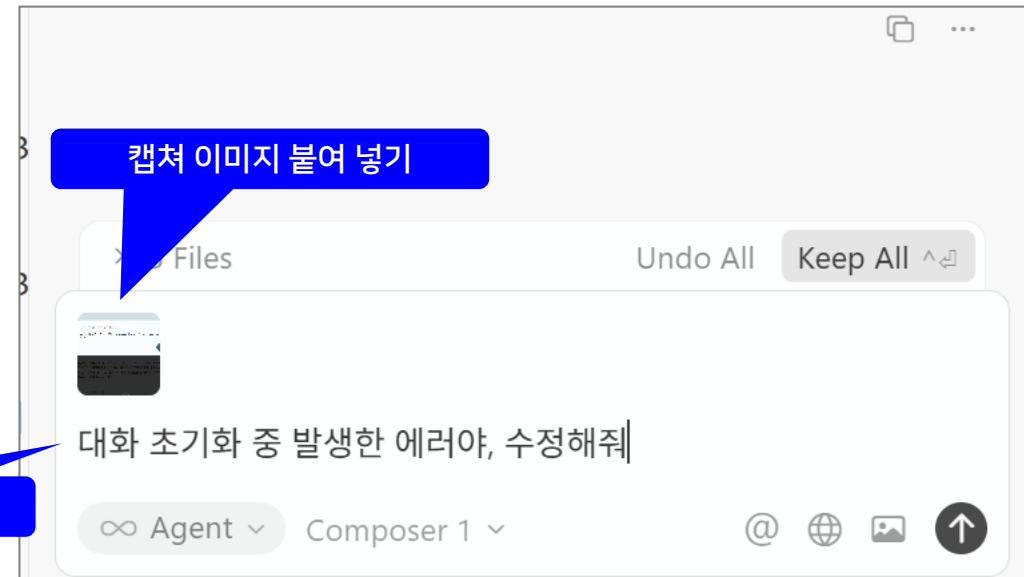
# 실습 1 : General Rag 구현

## ⑨ 오류 수정

- 오류가 발생한 화면을 캡처



- cursor AI 대화창에 캡처 이미지 붙여 넣고 오류 상황 입력



※ 코드 수정 완료 후 웹 페이지 새로고침하여 수정 여부 확인

# 실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

## ▶ Local LLM, 임베딩 모델



- 모델명 : [BAAI/bge-m3](#) (임베딩)
- 크기 : 1GB 이내
- 문서 벡터화, 검색, RAG Retriever
- 다국어, 한국어 지원 우수
- CPU/GPU 임베딩 사용
- 빠르고 정확한 검색 품질, 가벼움



- 모델명 : [EXAONE-3.5-2.4B-Instruct-GGUF](#) (LLM)
- 크기 : 1.8GB 이내
- 대화형 Instruct LLM, 오프라인 대화
- 한국어 강점, 한국어, 영어 이중언어 최적화
- GGUF 포맷, CPU 에서도 로컬 실행 가능
- 소형 모델이라 복잡한 추론에 한계

# 실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

## ▶ 모델 다운로드 및 개발 환경 설정

- ① 폴더 생성 : Rag 코드 생성할 폴더에 models 폴더 생성 ex) Rag 폴더 : `.\local_rag` , 모델 저장 폴더 : `.\local_rag\models`
- ② 터미널 실행 : 탐색기로 `.\local_rag\models` 폴더로 이동 후, 주소창에 'cmd' 입력
- ③ 허깅페이스 패키지 설치 : 터미널에 'pip install huggingface\_hub' 입력
- ④ 임베딩 모델 다운로드 : 터미널에 명령어 입력
  - `.\local_rag\models\hf download BAAI/bge-m3 --local-dir bge-m3`
- ⑤ llm 다운로드 : 터미널에 명령어 입력
  - `.\local_rag\models\hf download LGAI-EXAONE/EXAONE-3.5-2.4B-Instruct-GGUF --include "*Q5_K_M.gguf" --local-dir exaone_2.4b`
- ⑥ requirements 파일 생성 : `.\local_rag` 폴더에서 명령어 입력
  - `.\local_rag\pip list >> requirements.txt`
- ⑦ Cursor AI 실행 : file → Open folder → "`.\local_rag`" 폴더 클릭

# 실습 2 : Local LLM, 임베딩 모델 활용 Rag 구현하기

## ▶ Local rag 구현



### Prompt

파이썬 langchain 패키지를 이용해서 rag를 만들어줘

- flask 패키지를 이용해 웹페이지를 만들어줘
- user interface는 chatgpt와 유사하게 만들어줘
- 버튼, 창 스타일은 연한 청색, 회색 컬러로 산뜻하게 만들어줘
- 왼쪽 메뉴창 상단에 파일 업로드 버튼표시 (여러개 파일 업로드)
- pdf 파일만 업로드 할꺼야
- llama-cpp-python 사용해주 (llm 과 임베딩 모델 모두 local 에 저장되어 있음)
- llm은 "./models/exaone\_2.4b/EXAONE-3.5-2.4B-Instruct-Q5\_K\_M.gguf" 이용해
- 임베딩 모델은 "./models/bge-m3" 을 이용해서 500글자 분할, 100글자 오버랩, chromadb에 저장
- 사용자가 질문을 할 때 유사한 chunk를 4개까지 검색해줘
- chunk가 검색되지 않으면 "정보가 없어서 답변할 수 없습니다" 라고 대답해야해
- 대화 메모리 기능을 사용해주
- 답변을 출력한 후, 참고한 chunk 정보도 같이 출력해줘

# M4. 종합실습

## 10. 종합실습

AGENT 입문 교육 · 종합실습

# 바이트코딩 + RAG 종합실습 3선

LangGraph 에이전트 · RAG 1 + 비RAG 2 · PyQt5 1 + Flask 2

01

영업마케팅

비RAG · PyQt5

02

전략재무

비RAG · Flask

03

HR경영지원

RAG · Flask

# 3개 실습 한눈에 보기

GUI · PyQt5

RAG 미사용

영업마케팅·전략재무는 비RAG형, HR경영지원은 RAG형이며, GUI는 PyQt5 1개·Flask 2개로 구성했습니다.

01

## 영업마케팅 고객 리뷰(VOC) 분석

비RAG

고객 리뷰를 업로드하면 감성·이슈를 분류하고, 내장된 응대 원칙으로 대응전략을 도출해 보고서를 작성하는 에이전트

입력 customer\_reviews.csv · GUI PyQt5

02

## 전략재무 재무제표 분석 리포트

비RAG

재무제표를 업로드하면 재무비율을 계산하고, 내장된 해석 기준으로 이상징후를 도출해 재무분석 보고서를 작성하는 에이전트

입력 financial\_data.csv · GUI Flask

03

## HR경영지원 채용 적합성 검토 보조

RAG

지원자 데이터를 업로드하면 직무 적합도를 검토하고 채용 검토 보고서를 작성하는 의사결정 보조 에이전트

입력 인사규정.pdf + applicants.csv · GUI Flask

### 공통 구조 · 기술 스택

- 에이전트 LangGraph · LLM gpt-4o-mini · 보고서 docx
- 흐름: 분석 → 기준 적용 → 보고서 생성
- RAG: HR경영지원만 사용 (영업마케팅·전략재무는 비RAG)
- GUI: PyQt5(영업마케팅) · Flask(전략재무·HR)

## 고객 리뷰(VOC) 분석

VOC 분석 및 대응전략 시스템

데이터 업로드 분석 다운로드

업로드 리뷰 데이터 (CSV 미리보기)

|    | review_id | date       | product  | rating | review_text                          |
|----|-----------|------------|----------|--------|--------------------------------------|
| 1  | R0001     | 2025-05-07 | 노트북 거치대  | 5      | 배송도 빠르고 마감이 깔끔해요. 안정감 최고입니다.         |
| 2  | R0002     | 2025-05-08 | 무선 이어폰   | 1      | 2주 만에 한쪽 소리가 안 나요. 불량인 것 같습니다.       |
| 3  | R0003     | 2025-05-17 | 텀블러      | 2      | 주문한 색이랑 실제 색이 너무 달라요. 사진이랑 다릅니다.     |
| 4  | R0004     | 2025-05-20 | 블루투스 키보드 | 4      | 타건감 좋아요. 다만 설명서가 부실해서 페어링이 헛갈렸어요.    |
| 5  | R0005     | 2025-05-01 | 캠핑 의자    | 1      | 배송이 5일이나 늦었고 포장도 찢어진 채로 왔습니다.        |
| 6  | R0006     | 2025-05-18 | 가습기      | 3      | 제품은 괜찮은데 가격이 타사보다 비싼 편이에요.           |
| 7  | R0007     | 2025-05-07 | 보조배터리    | 5      | 충전 속도 빠르고 가벼워서 만족합니다. 재구매 의사 있어요.    |
| 8  | R0008     | 2025-05-23 | 러닝화      | 2      | 사이즈가 정사이즈보다 작아요. 교환하려는데 절차가 복잡합니다.   |
| 9  | R0009     | 2025-05-21 | 전기포트     | 1      | 사용 3일 만에 누수가 발생했어요. 안전 문제 같습니다.      |
| 10 | R0010     | 2025-05-23 | 백팩       | 4      | 수납공간 많고 튼튼해요. 색상이 화면보다 약간 어둡네요.      |
| 11 | R0011     | 2025-05-18 | 선풍기      | 2      | 소음이 생각보다 커요. 밤에 쓰기 부담됩니다.            |
| 12 | R0012     | 2025-05-14 | 무드등      | 5      | 감성 조명으로 딱이에요. 선물용으로 좋습니다.            |
| 13 | R0013     | 2025-05-08 | 청소기      | 1      | 흡입력이 약하고 금방 멈춰요. 환불 요청했는데 연락이 안 됩니다. |
| 14 | R0014     | 2025-05-15 | 모니터암     | 3      | 설치가 좀 까다로웠어요. 영상 가이드가 있으면 좋겠어요.      |
| 15 | R0015     | 2025-05-19 | 가죽지갑     | 4      | 품질 좋아요. 다만 쿠션 적용이 결제 단계에서 안 됐어요.     |
| 16 | R0016     | 2025-05-09 | 공기청정기    | 1      | 필터에서 냄새가 나요. 새 제품인데 실망합니다.           |
| 17 | R0017     | 2025-05-26 | 손목 보호대   | 5      | 착용감 편하고 효과 좋아요. 강력 추천합니다.            |
| 18 | R0018     | 2025-05-28 | 우산       | 2      | 바람에 한 번에 뒤집어졌어요. 내구성이 약합니다.          |
| 19 | R0019     | 2025-05-01 | 도마       | 3      | 무난해요. 배송은 빨랐습니다.                     |
| 20 | R0020     | 2025-05-25 | 전동 칫솔    | 1      | 충전 단자가 헛거워서 충전이 안 될 때가 많아요.          |
| 21 | R0021     | 2025-05-26 | 무선 마우스   | 4      | 가성비 좋아요. 클릭감도 만족스럽습니다.               |
| 22 | R0022     | 2025-05-06 | 커피머신     | 2      | 사용법이 너무 어려워요. 매뉴얼만으로는 모르겠어요.         |

Agent 동작 로그 및 분석 결과

VOC 분석 시스템이 시작되었습니다.  
메뉴에서 '데이터 업로드'를 선택하여 CSV 파일을 업로드하세요.  
데이터 업로드 원료: customer\_reviews.csv  
→ 총 36행 로드됨

=== AI 분석 시작 ===

[1/3] review\_analysis 노드 실행 중...  
리뷰 분석이 지연되고 있습니다. 기다려 주세요!!  
[1/3] review\_analysis 노드 완료 → 36건 리뷰 분석

[2/3] response\_strategy 노드 실행 중...  
[2/3] response\_strategy 노드 완료 → 12건 대응전략 도출

[3/3] report\_writer 노드 실행 중...  
[3/3] report\_writer 노드 완료 → 보고서 생성: /var/folders/ws/jxvfrysd1gj3b\_z9ph2r5d7r0000gn/T/voc\_analysis\_report.docx

=== VOC 분석 결과 (카테고리별 요약) ===

| 이슈 카테고리 | 부정 건수 | 우선순위 대응방안                     | 관련정책                 |
|---------|-------|-------------------------------|----------------------|
| 배송      | 1     | 보통 검토 필요                      | 검토 필요                |
| 품질      | 10    | 높음 사과 → 사진 요청 → 무상 교환/환불      | 수령 후 30일 이내 무상 교환·환불 |
| 디자인     | 1     | 확인 필요 검토 필요                   | 검토 필요                |
| 사용성     | 3     | 보통 단계별 사용법 안내 → 영상 링크 → AS 연결 | 미해결 시 AS 접수          |
| 배송      | 1     | 높음 배송 추적 확인 → 지연 사유 안내 → 보상쿠폰 | 지연 3일 초과 시 3,000원 쿠폰 |
| 가격      | 1     | 확인 필요 검토 필요                   | 검토 필요                |
| 사용성     | 3     | 확인 필요 검토 필요                   | 검토 필요                |
| 디자인     | 1     | 보통 상세페이지 기준 안내 → 변심 교환 안내     | 수령 후 7일 이내 변심 교환     |
| 품질      | 10    | 보통 검토 필요                      | 검토 필요                |
| 품질      | 10    | 확인 필요 검토 필요                   | 검토 필요                |
| 가격      | 1     | 보통 정책 안내 → 쿠폰 안내 → 차액 환급 검토   | 적용 가능 쿠폰 우선 안내       |
| CS      | 1     | 확인 필요 검토 필요                   | 검토 필요                |

=== 마케팅 개선 제언 ===

- \*\*품질 개선\*\*: 고객의 품질 관련 부정 리뷰가 많으므로, 제품 품질 관리 시스템을 강화하고, 고객 피드백을 반영한 품질 개선 프로세스를 도입합니다. 정기적인 품질 점검 및 테스트를 통해 문제를 사전에 예방합니다.
- \*\*사용성 안내 강화\*\*: 사용성 관련 부정 리뷰에 대응하기 위해, 제품 사용법을 단계별로 안내하는 콘텐츠(영상, 매뉴얼 등)를 제작하고, 고객이 쉽게 접근할 수 있도록 웹사이트 및 패키지에 포함시킵니다.
- \*\*배송 지연 대응 정책 개선\*\*: 배송 지연에 대한 고객 불만을 줄이기 위해, 배송 추적 시스템을 개선하고, 지연 발생 시 고객에게 즉시 알림을 제공하며, 보상 쿠폰 정책을 명확히 안내합니다.
- \*\*디자인 피드백 수집\*\*: 디자인 관련 부정 리뷰를 줄이기 위해, 고객의 디자인 선호도를 조사하고, 이를 바탕으로 제품 디자인을 개선하는 프로세스를 마련합니다. 고객 참여를 유도하는 디자인 공모전 등을 개최할 수 있습니다.
- \*\*가격 정책 투명화\*\*: 가격 관련 부정 리뷰에 대한 대응으로, 가격 정책을 명확히 안내하고, 할인 및 쿠폰 사용 방법을 고객에게 쉽게 이해할 수 있도록 설명하는 자료를 제공합니다.

=== 분석 완료 ===

메뉴에서 '다운로드'를 선택하여 보고서를 저장하세요.

고객 리뷰를 업로드하면 감성·이슈를 분류하고, 내장된 응대 원칙으로 대응전략을 도출해 보고서를 작성하는 에이전트

### 데이터 / 입력

#### 분석 데이터

customer\_reviews.csv — 고객 리뷰 36건

#### 대응 기준

사내 응대 원칙·보상정책을 프롬프트에 내장 (문서 검색 없음)

### 개발하고자 하는 것

- 리뷰 감성·이슈 카테고리·우선순위 분류 (pandas+LLM)
- 이슈별 대응방안을 LLM이 직접 생성 (내장 응대 원칙 기반)
- VOC 분석·대응전략 보고서(docx) 생성

**산출물** 분석 결과 표 형식의 보고서 (docx · 14pt · 가로)

Agent 노드 구성 (3노드 · RAG 미사용) · GUI PyQt5

**review**  
리뷰 분석



**strategy**  
대응 도출



**writer**  
보고서

## 전략재무 분석 시스템

데이터 업로드 분석 다운로드

## 재무제표 미리보기

| 계정과목    | 2023   | 2024   | 2025   |
|---------|--------|--------|--------|
| 매출액     | 52,000 | 58,000 | 61,000 |
| 매출원가    | 39,000 | 44,500 | 48,200 |
| 매출총이익   | 13,000 | 13,500 | 12,800 |
| 판매비와관리비 | 8,200  | 9,100  | 10,300 |
| 영업이익    | 4,800  | 4,400  | 2,500  |
| 이자비용    | 900    | 1,200  | 1,600  |
| 당기순이익   | 3,100  | 2,400  | 700    |
| 유동자산    | 18,000 | 19,500 | 18,800 |
| 유동부채    | 12,000 | 14,500 | 17,900 |
| 자산총계    | 46,000 | 50,000 | 53,000 |
| 부채총계    | 24,000 | 29,000 | 34,500 |
| 자본총계    | 22,000 | 21,000 | 18,500 |

## Agent 동작 로그

재무비율 계산이 지연되고 있습니다. 기다려 주세요!!

[financial\_analysis] 재무제표 CSV 로드 및 재무비율 계산 시작

[financial\_analysis] 8개 지표 계산 완료

[financial\_analysis] 이상징후 2건 감지

[interpretation] 재무비율 해석 및 등급 판정 시작

[interpretation] 8개 지표 해석 완료

[report\_writer] 재무분석 보고서 생성 시작

[report\_writer] 보고서(docx) 생성 완료

## 재무비율 분석 결과

| 구분  | 지표명    | 값      | 추세            | 등급 | 해석                             |
|-----|--------|--------|---------------|----|--------------------------------|
| 안정성 | 유동비율   | 105.0% | 전기 대비 하락 (악화) | 주의 | 100~150% 구간, 제조업 평균(130%) 하회   |
| 안정성 | 부채비율   | 186.5% | 전기 대비 상승 (악화) | 주의 | 100% 초과 주의 구간, 제조업 평균(110%) 상회 |
| 안정성 | 자기자본비율 | 34.9%  | 전기 대비 하락 (악화) | 주의 | 30% 미만, 재무 안정성 저하              |
| 안정성 | 이자보상배율 | 1.56배  | 전기 대비 하락 (악화) | 주의 | 1배 미만, 재무 리스크                  |
| 수익성 | 영업이익률  | 4.1%   | 전기 대비 하락 (악화) | 주의 | 3% 미만, 수익성 저하                  |
| 수익성 | 순이익률   | 1.1%   | 전기 대비 하락 (악화) | 위험 | 적자, 재무 상태 악화                   |
| 수익성 | ROE    | 3.8%   | 전기 대비 하락 (악화) | 위험 | 5% 미만, 투자 수익성 저하               |
| 성장성 | 매출성장률  | 5.2%   | 전기 대비 성장      | 주의 | 0~10% 구간, 지속적인 성장 필요           |

## 이상징후

- 매출 증가에도 영업이익률이 하락하여 원가-판매비 점검이 필요합니다.
- 부채비율 상승과 이자보상배율 하락이 동반되어 재무 리스크가 있습니다.

## 종합 평가

메인으로

보고서 다운로드

재무제표를 업로드하면 재무비율을 계산하고, 내장된 해석 기준치로 이상징후를 도출해 재무분석 보고서를 작성하는 에이전트

### 데이터 / 입력

#### 분석 데이터

financial\_data.csv — 3개년 재무제표

#### 판단 기준

재무비율 해석 기준치를 프롬프트에 내장 (문서 검색 없음)

### 개발하고자 하는 것

- 재무비율 계산·추세·이상징후 도출 (pandas)
- 내장 기준치로 LLM이 지표 해석·종합평가
- 재무분석 보고서(docx) 생성

**산출물** 분석 결과 표 형식의 보고서 (docx · 14pt · 가로)

Agent 노드 구성 (3노드 · RAG 미사용) · GUI Flask

**finance**  
비율 계산



**interpret**  
지표 해석



**writer**  
보고서

## HR 채용 검토 Agent — 분석 결과

메인으로

보고서 다운로드

## 분석 로그

```
[RAG] 기존 vectordb 확인됨 - 분석 준비 완료
```

```
[applicant_parse] CSV 로드 중
```

```
[applicant_parse] 지원자 10명 데이터 분석
```

## 지원자 적합성 검토 결과

| 지원자  | 지원직무        | 적합도   | 직무요건 충족 | 권고사항  |
|------|-------------|-------|---------|-------|
| 지원자A | 소프트웨어개발     | 높음    | 충족      | 면접 추천 |
| 지원자B | 소프트웨어개발     | 낮음    | 요건 미달   | 보류    |
| 지원자C | 영업마케팅       | 높음    | 충족      | 면접 추천 |
| 지원자D | 영업마케팅       | 낮음    | 요건 미달   | 보류    |
| 지원자E | 생산기술        | 높음    | 충족      | 면접 추천 |
| 지원자F | 생산기술        | 낮음    | 요건 미달   | 보류    |
| 지원자G | 경영지원(HR/재무) | 높음    | 충족      | 면접 추천 |
| 지원자H | 경영지원(HR/재무) | 보통    | 충족      | 확인 필요 |
| 지원자I | 소프트웨어개발     | 높음    | 충족      | 면접 추천 |
| 지원자J | 영업마케팅       | 확인 필요 | 확인 필요   | 확인 필요 |

채용 검토 보고서 다운로드 (DOCX)

본 결과는 보조자료이며 최종 판단은 채용담당자가 수행함

## 채용 적합성 검토 보조

GUI · Flask

RAG 사용

지원자 데이터를 업로드하면 직무 적합도를 검토하고 채용 검토 보고서를 작성하는 의사결정 보조 에이전트

## 데이터 / 입력

RAG 문서

인사규정\_직무기술서.pdf — 직무 요건·평가 배점

분석 데이터

applicants.csv — 가상 지원자 10명

## 개발하고자 하는 것

- 지원자 핵심정보·직무 적합도 도출
- 직무별 요건·평가기준 RAG 검색
- 채용 검토 보고서(docx) 생성 — 최종 판단은 사람이

산출물 분석 결과 표 형식의 보고서 (docx · 14pt · 가로)

Agent 노드 구성 (4노드) · GUI Flask

rag  
규정 RAG



parse  
지원자 분석



criteria  
기준 검색



writer  
보고서